Master document: n = 200 SMC-BVS study — methodology, simulation design, benchmarks, execution plan

Felipe (project); compiled 2026-10-19

The Markdown file `MASTER_Methodology_and_Plan.md` is the master document. This PDF is generated from it by a script (`latex_build/md2tex.py`) and is a rendering only: if the two ever differ, the Markdown file governs.

Markdown title: Master document: n = 200 SMC-BVS study (methodology, simulation design, benchmarks, execution plan)

Source file SHA-256: `7d77dddb968135d86a6b6431a7a3e04e122eb0c09f5ec9a6a8329e9a20757b6e`

Source lines: 2,759. The Markdown table of contents is replaced by the one on the next page; relative file links are shown as paths.

Table of contents

Revision log . . . 1
Reading guide, tags and project paths . . . 4

Part I. Methodology under evaluation: SMC-BVS v3 as deployed (2026-10-22) 5
I.1 Inputs, preprocessing and budgets . . . 5
I.2 Target posterior and numerical scorer . . . 7
I.3 Exact branch, blocks, tables and composite start . . . 9
I.4 Neighbour graph, elementary steps, resampling and randomness . . . 11
I.5 Calibration, candidate grid and fresh final run . . . 14
I.6 Local search, HPM, PIPs, evidence and statuses . . . 16
I.7 Connection to the benchmark study . . . 19
I.8 Complete settings table . . . 19
I.9 Lead rulings and limits of deployment claims . . . 24

Part II. Simulation design 26
II.1 Regimes and scope . . . 26
II.2 Common data-generating model . . . 26
II.3 Truth by p . . . 26
II.4 The six covariance scenarios . . . 27
II.5 Nominal signal-to-noise ratio . . . 28
II.6 Size of the study, scales and pairing . . . 28
II.7 Seeds . . . 29
II.8 Conventions for every method . . . 30
II.9 Generator specification . . . 31
II.10 Files and separation of truth . . . 32
II.11 Size on disk . . . 32
II.12 Storage location on NOTS . . . 32
II.13 Verification checks (gate G2) . . . 33

Part III. Benchmarks 34
III.1 The final panel . . . 34
III.2 Why each method was chosen . . . 34
Evidence base . . . 35
F1 Lasso . . . 35
F2 Elastic net . . . 36
F3 Adaptive lasso . . . 36
F4 MCP (SCAD as a sensitivity) . . . 36
F5 Best subset (ABESS) . . . 37
A1 SuSiE . . . 37
A2 Spike-and-slab lasso . . . 37
A3 Horseshoe . . . 38
Why A1–A3 . . . 38
III.3 Run specification for each panel method . . . 38
III.3.0 Common rules for all eight methods . . . 38
III.3.1 F1 Lasso . . . 42
III.3.2 F2 Elastic net . . . 49

III.3.3 F3 Adaptive lasso . . . 53
III.3.4 F4 MCP (SCAD as a sensitivity) . . . 55
III.3.5 F5 Best subset . . . 59
III.3.6 A1 SuSiE . . . 63
III.3.7 A2 Spike-and-slab lasso . . . 71
III.3.8 A3 Horseshoe (`bayesreg`) . . . 79
III.3.9 No-oracle audit (acceptance item) . . . 88
III.4 Considered but not selected . . . 88
III.4.1 Earlier candidate panel members not selected . . . 88
III.4.2 Sensitivity variants not adopted . . . 89
III.4.3 Considered and rejected (frequentist) . . . 90
III.4.4 Considered and rejected (Bayesian) . . . 91
III.4.5 Dropped by decision D1 (2026-10-19) . . . 92

Part IV. Execution plan **94**
IV.1 Outputs and metrics . . . 94
IV.1.1 Per (case, method) files . . . 94
IV.1.2 Rule-row schema . . . 94
IV.1.3 Aggregate tables . . . 95
IV.1.4 Pairing interface for the SMC-BVS arm . . . 95
IV.2 Compute plan on NOTS . . . 96
IV.2.1 Job structure . . . 96
IV.2.2 Runtime pilot (gate G6) before any budget is fixed . . . 97
IV.2.3 Estimates (EST; to be replaced by pilot measurements) . . . 97
IV.2.4 Waves, monitoring and retries . . . 99
IV.3 Gates and verification . . . 100
IV.3.1 Run records (policy §8) . . . 102
IV.3.2 CODE_LEDGER.md entries (project root), all "AI, unreviewed" until G5 . . . 103
IV.4 Timeline and checklist (EST durations) . . . 103
IV.5 Risks and mitigations . . . 105
IV.6 UNVERIFIED items (summary) . . . 106

Appendix A. Open decisions **108**
A.1 Study-design and execution decisions (O1–O14) . . . 108
A.2 Decisions recorded with the scenario specification . . . 111
A.3 Items that belong to the SMC-BVS arm and are not settled here . . . 111
A.4 Unverified items to close before production . . . 112

Appendix B. Reconciled differences **113**

Appendix C. Evidence and tools index **117**

Master document — supersedes BENCHMARK_PLAN.md, SCENARIOS.md, BENCHMARKS.md; last updated 2026-10-22 (Part I: SMC-BVS v3 with CORE_SPEC rulings and DP2 D2/D4; benchmark plan preserved; see the Revision log)

Status of the work: **plan only.** Nothing has been generated, installed or run. The design (Part II) comes from the former scenario specification, the method settings (Part III) from the former benchmark summary and the two evidence files `evidence/FREQUENTIST.md` and `evidence/BAYESIAN.md`. Where this master departs from the evidence files, it says so and gives the reason. The NOTS facts quoted below were checked read-only on 2026-10-19; the raw output is in `evidence/nots_checks_20261019.txt`. Where the superseded documents disagreed, the newer or more specific statement is kept and the difference is listed in Appendix B.

Revision log

2026-10-22, SMC-BVS v3 deployment methodology:

- Part I replaced by SMC-BVS v3 as deployed from `METHODOLOGY_v3.md`, SHA-256 `93043758058a6d422cca6ca4b6be8f3a9e8149b6f6de2e1bab1ce31c44b8019c`, with the CORE_SPEC rulings and variate conventions, DP2 D2 (primary RB PIP only), DP2 D4 (`cut_group` on the uncapped threshold graph, defined by the lead ruling 2026-10-22, DP2_DECISIONS addendum), and the retained warning semantics. Profiling-only features and pair moves are excluded. The lead ruling 2026-10-22, DP2_DECISIONS addendum also resolves R9/R16 by deploying the prototype's heuristic numerical certificate, labelled "heuristic (not a validated interval bound)". Both former source OPEN items are closed: fidelity is 105 mapped, 6 excluded, 0 OPEN. Source-location qualifications for R14–R18 and remaining prototype discrepancies are in `deployment_20261022_v3/CHANGES.md`.
- Round 2: applied all 14 findings (4 gaps and 10 minor) under **lead ruling 2026-10-22 (RULINGS_2)** in §I.1–§I.6 and §I.8–§I.9: explicit memory preflight/default allocation/runtime guard; identical inverse-CDF guide-table resampling and zero guard; RC-2 endpoint snap with all checks retained; complete prototype draw order; scorer rebuild, ordering, near-tie predicate, local certification, exact-branch NA and stage diagnostics. Updated CHANGES.md; fidelity remains 105 mapped, 6 excluded, 0 OPEN. §I.7 and everything outside Part I and this log are unchanged by Round 2. No build was attempted, as instructed.
- Round 3: applied N1–N6 under **lead ruling 2026-10-22 (RULINGS_3)** in §I.1, §I.4–§I.5 and §I.8–§I.9: prototype-exact RC-2 snapping and grid-check order; removal of the redundant pilot progress instruction; repeated backward inverse-CDF lookup; prototype 200-score timing procedure; specified convolution/exact-store coefficients with no extra term; and guard checkpoints after every island bridge stage, grid construction and local search. Updated CHANGES.md, including the RULINGS_2/G3 description. Source fidelity remains 105 mapped, 6 excluded, 0 OPEN. All other master content is unchanged by Round 3; no build was attempted.
- The untouched pre-v3 Markdown, LaTeX and PDF backup is `deployment_20261022_v3/backup_before_v3/`.
- Other locations touched: header; table of contents; this revision log and the historical 2026-10-19 D2 supersession label; reading guide; §II.8 (SMC preprocessing, primary PIP and evidence-scale references only); §III.1; §IV.1.4; §IV.2.1 (SMC scope sentence only); §IV.2.3 (SMC cost paragraph only); §IV.4 (SMC exclusion sentence only); §IV.5 (SMC prior/block rows and prior-arm sentence); §IV.6 item 9; Appendix A introduction, O13/O14 and A.3; Appendix B entries 14

and 18–20 and the historical Part-I renumbering reference; Appendix C. The benchmark panel, scenarios, seeds and NOTS run plan are preserved. The same-day lead addendum updates the Part-I authority/scope, §I.2, §I.3, §I.6, §I.8–§I.9, their contents link, Appendix A.3 and CHANGES.md to close D4 and R16; all other content is retained.

2026-10-19 (fifth revision), production launch:

- **All eight benchmark methods were launched in production on NOTS on 2026-10-19 between 21:57 and 23:08 UTC** (c = 1, 1,200 cases each, 120 tasks of 10 datasets, each from a frozen code snapshot whose tree hash equals the tested one). Run ids, jobs and resources: study `RUNS.md`. Every method passed, before launch, its unit tests, a 12-dataset dev re-run on the final code, a one-dataset production smoke test, a frozen dry run, a GPT-6.1 Sol code review and a GPT-6 Astra production-readiness consult. Truth folders were set to mode 000 during the fitting waves; scoring reads the read-only copy `$STUDY/truth_scoring/`.
- **A3 MCMC budget (O10 re-plan):** 10,000 burn-in + 15,000 kept draws, thin 5 (85,000 iterations). The pre-declared doubled budget (2,000 + 10,000) still flagged 12/12 dev fits; the chosen budget gave 0/12 flagged, ESS(log τ^2) 158–569, split-half $\hat{R} \leq 1.007$ and two-chain `ci95` Jaccard 1.0 (12/12), with p = 1000 fits below the 900 s re-plan line. Decided from diagnostics only, by the lead under the person's delegation of 2026-10-19; this departs from O10's "stop and re-plan with Felipe" clause.
- **Unit-test gate exceptions (F1, F2, F3):** a few G3 checks fail as worded in this master but not in substance: CV-min over-selects on the orthogonal canary (the EBIC rules recover the five signals exactly); the no-intercept path tolerance 1e-8 and the F2 generic-backend coefficient tolerance 1e-6 are unattainable near interpolation or at small λ, while supports are identical and KKT and RSS checks pass. The checks were kept as failing checks; the lead granted gate exceptions. Proposed wording changes are in each method's `MASTER_CHANGES.md` and will be folded in after G5.
- **Validation structure:** four further threads check the production outputs: scoring and tables (S), an independent Python recount (G9) by a different model provider, and independent G5 verification of F1–F5 (Python, different provider) and A1–A3 (separate thread). Thread list: `02_Experiments/F_2026-10-19_benchmark_study_n200/Benchmark_Study_20261019/thread_briefs/THREADS.md`. Results remain provisional until G5 and G9 pass.
- **Sealed held-out set:** 1,200 further datasets with the same design and a disjoint seed range (data 7,610,000 / 7,710,000, folds 8,610,000 / 8,710,000; run `20261019T2208Z_bench20261019_holdout_gen`, all checks passed) under `$STUDY/holdout/`, truth locked. Reserved for the final comparison of the person's own algorithm with the baselines; never used for tuning.

2026-10-19 (fourth revision), decisions for the production run:

- **D3 (Felipe).** The benchmark production run fits **scale c = 1 only** (`Y[, "c100"]`): 1,200 cases per method, 600 at p = 500 and 600 at p = 1000 (S1–S6 × 100 replicates). This is the alternative of O2. The datasets keep all five Y columns (generated and sealed as designed); the other four scales are not fitted. Counts that change: benchmark cases per method 6,000 → 1,200; primary fits 48,000 → 9,600.
- **Production data.** Run `20261019T2038Z_bench20261019_gen` (NOTS job 2277158) generated and sealed the 1,200 Regime-H datasets under `$STUDY/inputs`, `truth`, `manifests`; every G2 check of §II.13 passed, including bit-identity of all 600 p = 500 X and noise draws with Oct-14. Record in the study `RUNS.md`.

- **Pilot actions taken from the dev runs** (12 dev datasets per method, c = 1; diagnostics only, §IV.2.2): `GLMNET$maxit` 1e6 → **1e7** for F1–F3 (dev fits reached the pass limit; `thresh` stays 1e-16); A2 `capped_intermediate` becomes an **informational code**, not a flag (it fired on 24/24 dev fits from the package's σ^2 reset cycle; a capped final point still fails the fit); A3 O10 is **triggered** (12/12 dev fits flagged `ess_tau`; the new budget is recorded in §III.3.8 once fixed). Per-method dev reports: `02_Experiments/F_2026-10-19_benchmark_study_n200/Benchmark_Study_20261019/methods_dev/<method>/`.

2026-10-19 (third revision), benchmark deployment specification:

- §III.3.1–§III.3.8 are replaced by deployment-ready specifications, one per method, each with items a–l (model and algorithm, software, inputs and preprocessing, parameter table with `config.R` names, step-by-step calls, rules, outputs, failure handling, verification, compute, code to write, open items). They were written by two independent Opus agents (F1–F5 and A1–A3) from the CRAN source tarballs and read-only NOTS checks, and each was reviewed in two rounds by GPT-6 Astra (frequentist: 15 findings, 14 accepted, 1 partly; Bayesian: 27 findings, 26 accepted, 1 partly). Working files, change logs and Astra logs: `deployment_20261019/` (Appendix C).
- Resolved from package source: bayesreg's sampler (Bhattacharya et al. when $p/n \geq 2$, so at both p), its unit-length standardisation (not neutral, §III.3.8 c), `fit$beta` orientation ($p \times$ n.samples) and RNG use; the SSL native route at $p > n$; SuSiE's PIP, credible-set and stopping semantics; the F2 response scaling; the abess RNG side effect and the inert `max.splicing.iter`.
- Shared sections changed to match: reading guide ([V-src] tag), §II.7 (one method seed), §III.2 (A1, A3 caveats), §III.3.0 (interface, failure rows, configuration and code layout, reseeding, EBIC tie-break, versions), §III.3.9, §IV.1.2, §IV.2.2, §IV.2.3, §IV.3 (G1, G4, unit tests 3–5), §IV.3.2, §IV.5, §IV.6, Appendix A (O5, O10), Appendix C. The study design (Part II scope, panel, rules' roles, budget ceiling) is unchanged.
- A closing Fable 5.1 review (`deployment_20261019/FINAL_REVIEW/FABLE_REVIEW.md`; decisions in `RESOLUTION.md`) found the derivations correct and the shared layer inconsistent. Applied: one meaning of `status`, `diag_flag` (closed list of flag codes) and `converged` for all methods; a method registry, the rule-frame columns and one capture layer; timeout and not-run rows; the A3 common-random-numbers check moved to scoring; SuSiE ELBO decreases below 1e-3 flag rather than fail; `ebic05` added to F2 and F3 and the F1–F3 `ebic1` companion column in T1; O10 Jaccard at c = 1; several diagnostics dropped (SSL σ^2-state reconstruction, horseshoe near-boundary ESS and split R, SuSiE raw-set classification).

2026-10-19 (second revision), decisions by Felipe:

- **D1.** Regime L (n = 200, p = 30) is used **only** to assess the accuracy of the SMC-BVS estimates of the evidence Z and of the PIPs, against exact enumeration of all 2^{30} models. Bayesian variable selection (support recovery) and the benchmark comparison with F1–F5 and A1–A3 are run **only** for $p > n$, i.e. Regime H (n = 200, p = 500 and 1000).
- **D2 (historical; superseded for deployment by v3 on 2026-10-22).** Composite-start (SMC sampler) blocks: at most 5 variables per block at p = 30 ($2^5 = 32$ sub-models per block), and at most 10 variables per block at p = 500 and p = 1000 ($2^{10} = 1{,}024$ sub-models per block). These are the sampler blocks only; the data-generating population blocks of five in S1–S6 (§II.3, §II.4) are unchanged.

Sections changed: header; Part I preamble, §I.1, §I.3, §I.4, §I.5, §I.6, §I.7; §II.1, §II.6, §II.7, §II.8; Part III introduction, §III.1, §III.2 (evidence base, F4, F5), §III.3.0, §III.3.1, §III.3.4, §III.3.5, §III.3.6, §III.3.7, §III.3.8, §III.3.9, §III.4 (intro, §III.4.2, new §III.4.5); §IV.1.4, §IV.2.1–§IV.2.4, §IV.3 (G4, G6, G8, unit tests), §IV.4, §IV.5, §IV.6; Appendix A (introduction, O2, O9, new O12–O14, A.2, A.3); Appendix B (entries 17–22). Unchanged: §I.2, §II.2–§II.5, §II.9–§II.13, §III.3.2, §III.3.3, §IV.1.1–§IV.1.3, §IV.3.1, §IV.3.2, Appendix C. Counts that change: benchmark cases per method 9,000 → 6,000; primary fits 72,000 → 48,000; benchmark array tasks 1,800 → 1,200 (§II.6, §IV.2.1).

Reading guide, tags and project paths

- **Part I** is the methodology under evaluation (SMC-BVS v3 as deployed, with DP2 D2/D4). **Part II** is the simulation design. **Part III** is the benchmark panel: what it is, why, how each method is run, and what was not selected. **Part IV** is the execution plan (outputs, compute, gates, timeline, risks, unverified items). The appendices hold the open decisions, the reconciled differences and the evidence and tools index.
- Section references are written §I.x to §IV.x.x. Part I uses M for METHODOLOGY_v3, CORE R# for the ambiguity rulings, and DP2 D# for the 2026-10-22 decisions. Historical R1, DEC, FB, FA and FBS references denote the superseded round-1 sources in Setup_20261019 (Appendix C); they are not CORE R# rulings or sections of this document. The 2026-10-19 study decisions D1/D2 remain dated historical decisions; DP2 D2/D4 are different decisions. "This plan" in the text means this master document.

Tags used throughout:

- **[V-doc]**: checked against the CRAN reference manual or package source saved under `evidence/notes_frequentist/` or `evidence/notes_bayesian/`.
- **[V-src]**: read in the CRAN source tarball (or, for SSLASSO, the Oct-7 vendor copy), identified by its SHA-256, at the file and line given ("l." is a line of that file). The hashes and line references are listed in `evidence/FREQUENTIST.md` and `evidence/BAYESIAN.md`, sections "Deployment study 2026-10-19". "Derived here" marks algebra done for this plan from a [V-src] fact.
- **[V-NOTS]**: checked against the package installed on NOTS (`formals()` printed on 2026-10-19).
- **[V-run]**: already executed in a completed project campaign (Oct-7 `02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks`, Oct-14 `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14_Benchmarks`).
- **UNVERIFIED**: not checked yet.
- **EST**: an estimate, to be replaced by pilot measurements.

Project paths are written relative to the project root `Local_to_global_Bayesian/`, in the layout after the 2026-10-19 reorganisation (map: `00_Admin/Reorganization_20261019/MOVE_MAP.csv` and `PATH_MAP.md` at the root). This master document, with its `evidence/`, `tools/` and `latex_build/` folders, lives in `01_Methodology_and_Theory/MASTER_Methodology_and_Plan/`. The study workspace, where runs, `RUNS.md` and benchmark code go, is `02_Experiments/F_2026-10-19_benchmark_study_n200/Benchmark_Study_20261019/` (called STUDY in the policy). `CODE_LEDGER.md` stays at the project root. Campaign names used below: Oct-7 = `02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks`; Oct-14 = the `Experiments_October_14*` folders (Benchmarks, WeakSignals, Mixture and the base `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14`).

Part I. Methodology under evaluation: SMC-BVS v3 as deployed (2026-10-22)

What this section is. This is the operational specification of the SMC-BVS arm. It replaces the round-1 methodology. The benchmark methods, data-generating scenarios, study seeds and benchmark execution plan in Parts II–IV are separate decisions and are not changed by this replacement. In Regime L the study compares SMC evidence and PIPs with an external exact reference; in Regime H it compares selection with the benchmark panel (§II.8). The production exact branch below is part of the method; the external study reference is not an input to the fit.

Sources and authority. `M` denotes `../Methodology/SMC_BVS_v3_20261019/METHODOLOGY_v3.md`, SHA-256 `93043758058a6d422cca6ca4b6be8f3a9e8149b6f6de2e1bab1ce31c44b8019c`. `C` denotes the supplied `CORE_SPEC.md`, whose rulings resolve implementation ambiguities without changing M's target. `DP2` denotes `DP2_DECISIONS.md`, adopted 2026-10-22: D2 removes the secondary PIP output; D4 adds the cut-group report flag without changing B. DP2's warning decision retains E1, E3 and `local_limit`. D1 and D3 are theory changes and do not change this method. The later `RULINGS_OPEN.md` is the lead ruling 2026-10-22, DP2_DECISIONS addendum: it defines D4's uncapped threshold-graph predicate and adopts the prototype's labelled heuristic certificate for R9/R16, superseding the earlier validated-bound requirement. The exact source locations and fidelity mapping are recorded in `deployment_20261022_v3/CHANGES.md`.

Where the sources and the reference implementation are. Snapshots of every source named here are in `deployment_20261022_v3/source_snapshots/`. `prototype_core/` means `deployment_20261022_v3/source_snapshots/prototype_core/`, with checksums in `prototype_core.sha256` there. It is the independently verified R + C++ prototype of v3; the original is on NOTS at `/projects/ml75/fb31/BVS/v3_profiling_20261020/code/core/`, logged in CODE_LEDGER.md. It is a test reference only. Where this Part and the prototype differ, this Part governs, and the deliberate differences are listed in `deployment_20261022_v3/CHANGES.md`. These differences are: no secondary PIP, the cut-group flag, the `local_certified` tie rule, the memory-only preflight with an in-run time guard, and no profiling features. Everything else, including the order of random draws, is meant to reproduce the prototype's output bit for bit given the same seed.

Scope of implementation. Fit inputs are X, y, one integer seed, and the job's memory and wall-time allocation. There is no secondary prior arm, no pair-addition/deletion move, no profiling observer, watch list or watch set, no trace-replay interface, no exact-reference oracle, and no user options `force_smc`, `B_override` or profiling budget overrides. The production exact branch remains enabled under its stated rule. The two former source OPEN items are resolved by the lead addendum (§I.9); the separate study-adapter item in §I.7 remains unchanged.

I.1 Inputs, preprocessing and budgets

Input and index convention (M §2, §4; C R12). X is a numeric n × p input matrix; y is a numeric vector of length n. The integer seed s is in $[1, 2^{31} - 1]$. Every model and tie comparison uses sorted, 1-based **original column indices**, including after columns are dropped. Lexicographic comparison puts the smaller original index first at the first difference, and a proper prefix before the longer vector (R14; lead ruling 2026-10-22 (RULINGS_2)). Use p below for the number of retained predictors, except during initial input validation.

Apply these checks in this order (C §1):

1. Return `invalid_input` for a non-finite entry in X or y, $n < 3$, or fewer than one input predictor. The interface must also accept only the stated numeric dimensions and seed domain.
2. Drop a column only when all its entries are **exactly equal**. Record its original index; it can never be selected. If no predictor remains, return the empty selected model and `no_predictors`; evidence and PIPs are unavailable.
3. If all entries of y are exactly equal, return the empty selected model and `y_constant`; evidence and PIPs are unavailable. The uncomputed exact PIPs are not asserted to be zero. M §4 identifies the empty model as the HPM in this case.
4. Standardise each retained column stably: divide by its maximum absolute entry, compute its mean on that scale, centre, then divide by the sample SD with denominator $n - 1$. Apply the same procedure to y. Thus every retained column and y is centred and has squared norm $n - 1$. Preserve original indices, means, maximum-absolute scaling factors, SDs and the response scale s_y, where $s_y^2 = \|Cy_raw\|^2/(n-1)$ and $C = I - 11^T/n$. This map supports reporting and back-transformation.
5. Report the connected components of the graph with edges $|r_jl| \geq 1 - 10^{-10}$, using correlations of standardised columns. Keep duplicate and sign-duplicate predictors. Their posterior mass can split, so the MPM can exclude both members of a pair.
6. Perform the branch-specific resource preflight described below before allocating the branch's large objects.

These rules make the target invariant to nonzero affine changes of y or individual columns of X, and to column permutation apart from the stated original-index tie rules (M §4). The preprocessing record is therefore part of the fit, not an optional display transformation.

Budgets (M §3). All budgets use retained p. They are declared resource limits, not accuracy guarantees.

Symbol	Rule	At n = 200, p = 30 / 500 / 1000
K	$\min(p, \lfloor n/2 \rfloor)$	30 / 100 / 100
B	$\min(\lfloor \sqrt{n} \rfloor, \lfloor \sqrt{p} \rfloor, 16)$	5 / 14 / 14
ℓ	$\lceil \log_2(n + p + 1) \rceil$	8 / 10 / 11
L	$\max(4, \lceil \log_2(n + 1) \rceil)$	8 / 8 / 8
N_0	$\max(2500, n, p)$, final particles per island	2500 / 2500 / 2500
H_0	$\lceil 512\, N_0/2500 \rceil$, pilot and each diagnostic population	512 / 512 / 512
T_0	$\max(1000, \ell^2)$, stage ceiling	1000 / 1000 / 1000
R	4 final islands and 4 diagnostic islands per rebuild	4 / 4 / 4
e	tier 0 or tier 1; $(N_e, H_e, T_e) = 4^{\frown}e(N_0, H_0, T_0)$	one possible escalation

The sampler blocks in this table are built from X; they are unrelated to the population blocks used to generate study data. In particular, the previous B = 10 at p = 500 or 1000 is superseded by B = 14 at n = 200. The implementation ceiling $B^* = 16$ is fixed by M §3 and §8.7.

Resource preflight and time guard — lead ruling 2026-10-22: operational guard, not a statistical setting (RULINGS_2 G1; M §8.6; C §7). Use the job's declared allocation,

normally its actual memory and wall-time request. When absent, use `mem_gb = 8` (GiB) and `hours = 24`. Before allocating each branch's large objects, and again before tier 1, estimate memory from known dimensions and storage capacities. The SMC preflight does not predict the unknown number of stages or wall time.

For sequential islands, the following byte accounting is derived from `prototype_core/v3_fit.R`, `v3_preflight`. Put C_cache = 500p, W = ⌈p/64⌉ and N = N_e = 4^e N_0 (which also covers H_e). Replace the prototype's typical model size by the known cap K. Use the prototype's convolution coefficient ⌈p/B⌉ and exact-branch store size min(K,⌈p/2⌉), as required by lead ruling 2026-10-22 (RULINGS_3), N5. The table-score bound Σ_b 2^{|C_b|} ≤ p · 2^B/B (M §8.2) permits the rounded table term below.

```
M_base,SMC = 8(3np + 3p²)
           + 16⌈p/B⌉2^B
           + 8⌈p/B⌉(K+1)(B+1)
           + 29·2^⌈log₂(2C_cache)⌉ + 4C_cache K
           + (80p+1)(4K+64)
           + 2N(8W+4K+96)
M_base,exact = 8(3np+p²) + 8(40p+1)(min(K,⌈p/2⌉)+8) + 32p
M_est = M_base,branch.
```

The SMC terms account, respectively, for data copies and Gram/correlation/distance matrices, the two block-score tables, convolution/sampler tables, cache slots and model keys, the two 20p leader stores plus the (40p+1) near-tie store, and the population plus its resampled copy. The byte coefficients describe the prototype's storage layout, not statistical parameters. The prototype has no M_extra term, so none is added (lead ruling 2026-10-22 (RULINGS_3), N5). This is a documented implementation memory estimate, not a rigorous peak-memory guarantee. Return `resource_limit` if M_est > mem_gb · 2^{30}, retaining the estimate and its component record.

On the exact branch also project seconds as E × measured s(k*), with known E from §I.3 and k* = min(K, ⌈p/2⌉), the prototype's dimension-based timing size. Use the prototype's timing procedure (`v3_fit.R`, `v3_time_scores`): time 200 scores at size k* on a timing context built from the first min(p, max(64, k*)) standardised columns. Label this "operational measurement, not a statistical setting"* (lead ruling 2026-10-22 (RULINGS_3), N4). Record the measured per-score cost, timing procedure and environment; return `resource_limit` if this projection exceeds hours · 3600. Do not import the prototype's 50-planned-stage or min(K,10) SMC planning assumptions.

During the run, check the time guard after every bridge stage of each island (pilot, diagnostic islands and final run), after each grid construction, and after each local search (lead ruling 2026-10-22 (RULINGS_3), N6). At each checkpoint compare elapsed wall time since fit start plus the duration of the just-completed operation (bridge stage, grid construction or local search) with hours · 3600. If the sum is strictly greater, stop with `resource_limit` and keep everything produced so far, with partial results explicitly labelled. Record both times, the allocation and the stopping phase/checkpoint. This operational guard leaves all statistical budgets unchanged.

I.2 Target posterior and numerical scorer

Working model (M §5). All quantities below use the standardised X and y. There is no fitted intercept. For S ⊆ {1, …, p}, k = |S| ≤ K, set G_S = $X_S^T X_S$ and z_S = $X_S^T y$.

```
y | β_S, σ², S ~ N_n(X_S β_S, σ² I_n)
β_S | σ², S    ~ N_k(0, n σ² (G_S + I_k)^(-1))
σ²             ~ IG(a₀, b₀),  a₀ = b₀ = 0.01

w(k) = Beta(1+k, 2p-k)/Beta(1,p) = (1/2)/choose(2p-1,k)
W_K = Σ_{j=0}^K choose(p,j) w(j)
ϖ(S) = w(k)/W_K,  k ≤ K
```

The inverse-gamma convention is density proportional to $(\sigma^2)^{-(a_0-1)} \exp(-b_0/\sigma^2)$. The cap and normalisation are part of the target. This is the sole prior arm: g = n, unit ridge $\tau = 1$, IG(0.01, 0.01), and capped BB(1, p).

M §5 fixes g = n as unit information and $\tau = 1$ as one pseudo-observation on the standardised scale. The proper inverse-gamma prior keeps the integrated model proper. Before capping, BB(1,p) has mean size p/(p+1), near 1; moving from size k to k+1 costs $\log[(2p-k-1)/(k+1)]$ nats per individual model. The cap excludes near-interpolating models; its fraction 1/2 is a declared restriction, not a condition needed for propriety. These are modelling choices retained by Felipe, not parameters selected from the unknown support or study outcomes.

Integrated score (M §5).

```
H_S = (n+1) G_S + I_k
Q_S = y^T y - n z_S^T H_S^(-1) z_S
log C_n = a₀ log(2b₀) + log Γ(a₀+n/2) - (n/2)log π - log Γ(a₀)
log m(S) = log C_n
           - (1/2)[log det H_S - log det(G_S+I_k)]
           - (a₀+n/2)log(2b₀+Q_S)
F(S) = log ϖ(S) + log m(S)
π(S | y) = ϖ(S)m(S)/Z,  Z = Σ_{|S|≤K} ϖ(S)m(S)
```

For the empty model the determinant factors are 1 and the quadratic correction is zero. Keep W_K exactly once. Compute log w(k) with log-gamma binomial coefficients and log W_K by log-sum-exp (C §2). The evidence is the evidence of this **standardised working model**. It is not labelled the raw-data marginal likelihood. The exponent remains $a_0 + n/2$ after centring and scaling y; it is not changed to $a_0 + (n-1)/2$.

M §5 explains the reduction to contrast regression with n−1 Gaussian errors, raw response coefficient $\theta = s_y \beta$ and variance $v = s_y^2 \sigma^2$, and induced IG(0.51, 0.01 s_y^2). That reduction was a stated theory target in M; it is not an extra fitting step or a different deployed prior. On the standardised scale the ridge keeps all model scores finite even under collinearity, and $Q_S \geq y^T y/(n+1)$.

Numerics and failure (M §6.13; C §2). Use Cholesky factors for solves and log-determinants; never form an explicit inverse or determinant. Initially calculate Q_S by subtraction. Whenever $Q_S < 10^{-8}\, y^T y$, recompute it in the non-negative form

```
β̂ = n H_S^(-1) z_S
Q_S = ‖y - X_S β̂‖² + (‖X_S β̂‖² + ‖β̂‖²)/n.
```

Do not clip Q_S. Use log-sum-exp for every sum of weights, size profiles, normalisers and evidence. On Cholesky failure, rebuild $G_S = X_S^T X_S$ from X_S, then refactor from scratch; if that fails,

try augmented QR on X_S (lead ruling 2026-10-22 (RULINGS_2), finding 5). The prototype resolves the augmentation explicitly: QR of [√(n+1)X_S; I_k] yields a factor of H_S, and QR of [X_S; I_k] yields a factor of G_S+I_k (`prototype_core/cpp/v3_score.h`, `try_qr`). Obtain log-determinants from twice the sum of log absolute diagonal entries and solve with the triangular factors. If that also fails, stop with `numerical_failure`; do not substitute $-\infty$ for the score. Never send a model larger than K to the scorer. The output records numerical routes and the comparison-error information used for any certificate (§I.9).

Numerical certificate rule (lead ruling 2026-10-22, DP2_DECISIONS addendum). Deploy the verified prototype's heuristic from `prototype_core/cpp/v3_score.h`, `error_bound`, unchanged. Let $\nu = a_0+n/2$, ε = the machine epsilon of double precision (2^{-52}), and A_S = G_S+I_k. With the factors L_H and L_A used by the scorer, define $\kappa_H = [\max_i(L_H)_{ii}/\min_i(L_H)_{ii}]^2$ and κ_A analogously. These are cheap diagonal-ratio condition estimates, not exact matrix condition numbers. On the QR fallback use the positive-diagonal triangular factors obtained from the two augmentations above; for the empty model set $\kappa_H = \kappa_A = 1$ and $Q = y^Ty$, as the prototype does. C_n and Q are exactly those defined above. The heuristic for the error in log m(S) is

$$\eta(S) = 64\,\varepsilon\,[\nu\,(y^Ty/Q_S)(1 + k\,\kappa_H) + k(\kappa_H + \kappa_A) + |\log C_n| + \nu\,|\log(2b_0 + Q_S)|].$$

On the exact branch only, use the prototype's comparison rule (`v3_exact.R`, `cpp_enumerate`). For the returned HPM S*, set `global_certified` true iff

$$F(S^*) - \eta(S^*) > \max_{S\neq S^*,\ |S|\le K} [F(S) + \eta(S)].$$

Accumulate the maximum over all other enumerated models, so every competing score has its own η(S). Report η(S*), the unadjusted margin F(S*)−max_{S≠S*}F(S), and `certificate_bound` = F(S*)−max_{S≠S*}[F(S)+η(S)] with each certificate. Label the bound *"heuristic (not a validated interval bound)"**. Exact-F ties still use the HPM size/lexicographic rule and cannot pass this strict separation test. This is a numerical safety rule, not a statistical tuning constant; the certificate is not a rigorous interval proof or a statistical accuracy guarantee. It does not alter scores, target, HPM ties, statuses or budgets.

I.3 Exact branch, blocks, tables and composite start

Production exact branch (M §6.2). Compute E = Σ_{k=0}^K choose(p,k) with saturating integer arithmetic, sufficient to decide whether

$$E \le (5H_0 + 4N_0)p.$$

If so, score every feasible model in a streaming pass. Track the HPM under §I.6, accumulate log Z and each PIP numerator by log-sum-exp, and report exact posterior PIPs, MPM and evidence with status `exact`. No blocks or SMC are built. Completed enumeration has already scored every feasible neighbour of the returned HPM, so it may set `local_certified` directly; the equivalent full feasible flip/swap scan is optional (lead ruling 2026-10-22 (RULINGS_2), finding 9). `global_`

`certified` additionally requires the heuristic separation test in §I.2, adopted by the lead ruling 2026-10-22, DP2_DECISIONS addendum; completion of enumeration alone does not set it. Report its bound as "heuristic (not a validated interval bound)". This branch is mandatory, and there is no production `force_smc` option. At the n = 200, p = 30 study setting, $E = 2^{30}$ exceeds the branch threshold; the production fit therefore uses SMC. The study's separate all-model reference in §II.8 is not this branch decision.

Correlation blocks (M §6.3; C R13). When the SMC branch is selected:

1. Cluster retained predictors by complete linkage with distance $1 - |r_{jl}|$. The distance between clusters is the maximum pairwise distance. Among equal-distance candidate merges, choose the merge whose union has the lexicographically smallest sorted original-index list.
2. At every internal node place the child with smaller minimum original index on the left. Dendrogram leaf order means left-to-right order under this convention.
3. Cut top-down from the root: split any node containing more than B predictors into its children, visiting the left child first, until every retained cluster has at most B members.
4. Pack whole retained clusters by next-fit in dendrogram leaf order into C_1, …, C_J, each of size at most B. Never split a retained cluster to fill a block. This can leave unused block slots; $J \geq \lceil p/B \rceil$.

Cut-group report (DP2 D4; lead ruling 2026-10-22, DP2_DECISIONS addendum). For this report, a correlated group is a connected component of the **uncapped threshold graph** on retained predictors, with an edge j–l exactly when $j \neq l$ and $|r_{jl}| \geq \rho_n = \sqrt{(2 \log p/n)}$, using standardised sample correlations. This reporting graph differs from the move graph in §I.4: it uses $\geq$, has no per-variable B cap, and does not force a largest-correlation partner. The neighbour-swap graph and its strict $>$ rule are unchanged.

After constructing the composite-start blocks, set `cut_group` true iff some component C spans two or more blocks. For every such component report its size |C|, its number of spanned blocks, and its members in sorted original-column-index order; also report the total count of flagged components. This includes any split component of size at most B; every component with $|C| > B$ is necessarily split and flagged. If no component is split, `cut_group` is false and the flagged-component count is zero. On the exact branch set `cut_group = NA` (not applicable: no blocks are used; lead ruling 2026-10-22 (RULINGS_2), finding 10). The flag is informational and changes no setting, status or budget; B, clustering and packing are unchanged.

Block tables (M §6.3; C R2–R3). For every block C_b score every submodel $U \subseteq C_b$ against the full response y, and store $\log r_b(U) = \log m(U) - \log m(\emptyset)$. The table exponent is exactly 1. Since $B \leq K$ for $n \geq 3$, table submodels are feasible. Every table score enters the all-scored HPM and both candidate-leader stores. Table scores are not inserted into the score cache.

Composite start (M §6.3). Define

$$\gamma_0(S) = \varpi(S) \prod_{b=1}^{J} r_b(S \cap C_b), \quad |S| \leq K$$
$$Z_0 = \Sigma_{|S| \leq K} \gamma_0(S)$$
$$\pi_0(S) = \gamma_0(S)/Z_0.$$

This start has an exactly computable normaliser and an exact iid sampler. To implement the size convolution, let $e_b(j) = \Sigma_{U \subseteq C_b, |U|=j} r_b(U)$, and set $A_0(0) = 1$ and $A_0(k) = 0$ for $k > 0$. For $b = 1, \ldots, J$ and $k \leq K$,

```
A_b(k) = Σ_{j=0}^{min(|C_b|,k)} e_b(j) A_{b-1}(k-j)
Z_0 = Σ_{k=0}^K [w(k)/W_K] A_J(k).
```

Carry these calculations in log space. Draw the total size k with probabilities proportional to [w(k)/W_K]A_J(k). Backwards for b = J, …, 1, given remaining size k, draw j with probabilities proportional to e_b(j)A_{b−1}(k−j), then replace k by k−j. Within each block's drawn size j, draw U with probabilities proportional to r_b(U). The union of those U is an exact π_0 draw. This recurrence spells out the convolution/backward sampling rule of M §6.3; it does not introduce another approximation.

For reproducible variate consumption, the prototype's compatible convention (`cpp/v3_start.h`, `draw_pi0`) uses one uniform for total size in ascending order 0,…,K, then one for each backward block-size draw while the remaining size is positive; all remaining block sizes are zero once it reaches zero. Finally visit blocks in forward order and use one uniform per nonempty block to select its submodel. Order each block's members by original index and its within-size submodels by increasing binary inclusion code (the first member has the least significant bit). Each of these categorical draws chooses the first cumulative probability strictly greater than u; the final cumulative probability is exactly 1. Empty blocks require no submodel draw. This start-sampler convention is distinct from resampling's prescribed ≥u convention (§I.4).

Discrepancy and bridge (M §6.3).

```
BF(S) = m(S)/m(∅)
d(S) = log BF(S) - Σ_b log r_b(S∩C_b)
γ_φ(S) = γ_0(S) exp{φ d(S)},  0≤φ≤1
q_φ(S) = γ_φ(S)/Σ_A γ_φ(A).
```

Thus $q_0 = \pi_0$ and q_1 is the target posterior. $\gamma_1 = \varpi(S)BF(S)$, so its normaliser is Z/m(∅). Candidate ranking by log γ_1 equals ranking by F(S), since log m(∅) is common to every model; retain the scale distinction when reporting evidence.

I.4 Neighbour graph, elementary steps, resampling and randomness

Neighbour graph (M §6.4). Set ρ_n = $\sqrt{(2 \log p/n)}$. For each j with p > 1, consider partners l ≠ j with |r_jl| > ρ_n, always also including j's single largest-|r| partner even if it does not exceed ρ_n. Keep at most the B largest-|r| partners, breaking ties by smaller original index first (lead ruling 2026-10-22 (RULINGS_2), finding 12). Take the symmetric union: l ∈ N(j) iff j ∈ N(l). The resulting degree can exceed B. For p = 1 there are no edges.

Round (M §6.5; C §4). At temperature φ, a round contains the p labelled Gibbs updates G1, …, Gp, one uniform swap U, B labelled neighbour swaps N1, …, NB, and one independence proposal I: m = p + B + 2 elementary steps. Draw a uniformly random permutation of all these labels afresh for every particle and every round, independently of its state. Apply the steps in that order. Repeated N labels are distinct positions with identical kernels. There is no pair-addition/deletion move and no state-dependent repetition count.

For all steps, use log γ_φ to calculate target ratios stably. Every feasible scored configuration or proposal updates the HPM and leader stores, whether or not it becomes the particle's next state.

- **Gibbs coordinate j (C R1, R8).** Put A = S\{j}. Score A and, if |A| < K, A∪{j}; both feasible configurations count as scored, including the held state. With a = γ_φ(A∪{j})/γ_φ(A),

set o_j = a/(1+a), evaluated stably in log space. If |A| = K, addition is infeasible: o_j = 0 and only A is scored. Immediately before this coordinate's draw, record o_j at the particle state current at that point in the random order. Include j iff the Bernoulli draw succeeds.

- **Uniform swap.** Self-loop if |S| is 0 or p. Otherwise choose j uniformly from S and l uniformly from its complement; propose S′ = S\{j}∪{l}. Accept with min(1, $\gamma_{-}\varphi$(S′)/$\gamma_{-}\varphi$(S)).
- **Neighbour swap.** Self-loop if S is empty. Otherwise choose j uniformly from S. If N(j)\S is empty, reject without scoring a proposal. Otherwise choose l uniformly from N(j)\S, propose S′ = S\{j}∪{l}, and accept with min(1, [$\gamma_{-}\varphi$(S′)/$\gamma_{-}\varphi$(S)] · |N(j)\S|/|N(l)\S′|). The symmetric graph ensures the reverse edge exists.
- **Independence proposal.** Draw S′ exactly from π_0. Score it even if rejected. Accept with min(1, exp{φ[d(S′)−d(S)]}).

Each elementary step preserves its tempered target. The law of the random-order round is invariant under reversal; an individual realised ordering is not asserted to be self-adjoint. Kernel validity does not certify adequate mixing at the finite budgets.

One bridge stage (M §6.6, §6.8, §6.12; C R7). With incoming equally weighted particles S_i from the preceding stage's move round (or initial iid draws for stage 1), and $\Delta\varphi = \varphi_{-}t - \varphi_{-}\{t-1\}$:

1. Compute log weights $\Delta\varphi$ d(S_i) and the log increment log[(1/N)Σ_i exp{$\Delta\varphi$ d(S_i)}] **before** resampling. Record empirical ESS = (Σ_i w_i)²/Σ_i w_i², stably in log space.
2. Perform fresh multinomial resampling at every stage, including the terminal φ_t = 1 stage. The resampled population is equally weighted. Use the Chen–Asau inverse-CDF guide table specified below: O(N) construction and O(1) expected lookup per draw, hence O(N) expected total time, with exactly the prescribed ancestor for each uniform (lead ruling 2026-10-22 (RULINGS_2), G2).
3. Apply one full move round at φ_t to every resampled particle.

There is no conditional resampling threshold and no systematic resampling. Pilot and diagnostic stages use H_e particles; final stages use N_e.

Random streams (M §6.1, §6.8; C R6, §5).

Stream	Use
0	tier-0 pilot
1–4	tier-0 diagnostic islands 1–4
5	tier-1 pilot
6–9	tier-1 diagnostic islands 1–4
10–13	final islands 1–4

Derive reproducible independent PRNG streams from (seed, stream), and also particle identifier if parallelising by particle, using a documented seeding function. Save that function and PRNG convention in the stream map. For the same implementation and numerical environment, the same seed must reproduce bit-identical outputs. Diagnostic islands continue their respective streams across rebuild iterations; never re-initialise an island to repeat previous randomness. Each final island uses fresh iid start draws and touches none of streams 0–9.

Documented prototype convention (C §5, resolved by `prototype_core/cpp/v3_util.h`). C requires documented seeding and gives SplitMix64 → xoshiro256** as an example. The verified

prototype selects that example, with one serial stream per pilot/island. Retain this explicitly identified implementation convention for reproducibility. It is a generator definition, not a new statistical tuning choice. Use unsigned 64-bit arithmetic modulo 2^{64}, zero-based state words a[0..3], logical right shift `>>`, xor `^`, and rotation `rotl`:

```
x = (uint64(seed) << 32) + stream
repeat four times, assigning successive a[0],…,a[3]:
    x = x + 0x9E3779B97F4A7C15
    z = x
    z = (z ^ (z >> 30)) * 0xBF58476D1CE4E5B9
    z = (z ^ (z >> 27)) * 0x94D049BB133111EB
    a[next] = z ^ (z >> 31)

next_uint64():
    result = rotl(a[1]*5, 7)*9
    t = a[1] << 17
    a[2] ^= a[0]; a[3] ^= a[1]; a[1] ^= a[2]; a[0] ^= a[3]
    a[2] ^= t; a[3] = rotl(a[3], 45)
    return result
uniform() = (next_uint64() >> 11) * 2^(-53)
```

Here `rotl(z,k) = (z << k) | (z >> (64-k))`. Serial particle processing preserves the prototype's consumption order. Introducing parallel particle streams requires documenting their derivation and changes the reproduction convention; M/C do not prescribe that extension.

Variate conventions (C §4.1; lead ruling 2026-10-22 (RULINGS_2), G2 and finding 13). A uniform choice from m candidates uses one $u \in [0,1)$ and position $\lfloor um \rfloor + 1$ in the candidate list sorted by original index. A Bernoulli with probability a succeeds iff $u < a$; this includes Gibbs inclusion and MH acceptance. Draw round permutations by Fisher–Yates from the canonical labels G1, …, Gp, U, N1, …, NB, I: for i=m down to 2, draw u and swap positions i and $\lfloor ui \rfloor + 1$. Initial draws and independence proposals use the exact π_0 sampler in §I.3.

For multinomial resampling, construct the cumulative weights exactly as in `cpp/v3_engine.h`: with lw_i = $\Delta\varphi$ d(S_i), accumulate c_k = Σ_{i=1}^k exp(lw_i−max_j lw_j) in particle order in double precision; set W_k = c_k/c_N, then W_N = 1 exactly. Draw N fresh uniforms in stream order. Replace u = 0 by the smallest positive double, 2^{-1074} (`denorm_min`), without consuming another draw; call the resulting value u_eff. The ancestor is the **smallest** k with W_k ≥ u_eff, including equality. Thus a zero-weight particle is never selected.

Implement that same first-index search with a Chen–Asau guide (index) table. Build the N equal-width probability-bin guides in one forward scan of W, storing for bin q = 0,…,N−1 the first index with W_k ≥ q/N. For u_eff, start at index k from the guide for q = min(N−1, ⌊N u_eff⌋), step forward while W_k < u_eff, then step back while k > 1 and W_{k−1} ≥ u_eff. The result is the first k with W_k ≥ u_eff, bit-identical to `lower_bound` even across zero-weight runs (lead ruling 2026-10-22 (RULINGS_3), N3). This takes O(N) setup and O(1) expected lookup per draw. It changes neither W's arithmetic nor the selected ancestor and consumes no extra randomness: it is bit-identical to the prototype's inverse-CDF search in the same numerical environment. Alias sampling, systematic resampling and any scheme mapping the same uniform to a different ancestor are forbidden.

Random-draw consumption order (lead ruling 2026-10-22 (RULINGS_2), G4). Retain the order in `prototype_core/v3_run.R`, `variate_order_doc`, checked against `cpp/v3_engine.h`, `cpp/v3_start.h` and `cpp/v3_util.h`. In each pilot/island stream, first process particles 1,…,N, making

each initial π_0 draw completely: `pi0_size`; `pi0_block_size` for b = J,…,1 while remaining size > 0; then `pi0_submodel` for b = 1,…,J with k_b > 0 (§I.3). Here N is H_e during calibration and N_e in the final run.

At each bridge stage consume all N `resample` uniforms first. Then, for particles 1,…,N, perform one whole round per particle. Each round first consumes m−1 `order` uniforms for Fisher–Yates, i = m,…,2; then execute the steps in that drawn order with the following consumption:

Step	Uniforms consumed, in order
G	Exactly one `gibbs` uniform per coordinate step, including deterministic probabilities and an infeasible cap addition with o_j = 0.
U	None when size is 0 or p; otherwise `uswap_j`, `uswap_l`, `uswap_accept`.
N	None when S is empty; otherwise `nswap_j`, then, only if N(j)\S is nonempty, `nswap_l`, `nswap_accept`.
I	A full π_0 draw in the stated start-sampler order, then `indep_accept`.

Do not omit an acceptance uniform when its probability is 1, or when an independence proposal leaves the state unchanged. After the bridge, run L burn-in rounds and four measured rounds in that order; each round processes particles 1,…,N with the same permutation and step-draw order and no resampling. This consumption order makes the prototype a bit-level test reference for these operations under the same PRNG and numerical environment. It does not import prescribed-draw/order hooks, a trace recorder or replay facilities.

I.5 Calibration, candidate grid and fresh final run

Execution order (M §6.1; C R4). After preprocessing and preflight, take the exact branch if its condition holds. Otherwise construct the blocks, tables, start and neighbour graph. Calibrate at tier 0. If any calibration trigger fires, preflight the tier-1 work and repeat calibration at tier 1, preserving the all-scored best and both candidate-leader stores. There is at most one escalation. Freeze the selected grid and budgets before drawing any final randomness. Run the four fresh final islands, then one final local search and the final checks. Calibration estimates are archived and labelled; they never substitute for final estimates. Final observations never revise the grid, change the budget or select replacement estimates.

Pilot (M §6.6; lead ruling 2026-10-22 (RULINGS_2), finding 14; lead ruling 2026-10-22 (RULINGS_3), N1–N2). Draw H_e iid π_0 particles and score them. Starting at $\varphi = 0$, first test $\varphi' = 1$: if the ESS of weights $\exp\{(1-\varphi)\mathrm{d}\}$ is at least 0.90H_e, take 1 directly and do not count a forced step. Otherwise bisect on $(\varphi,1]$ with lo = φ and hi = 1. For each of 60 halvings, test mid = (lo+hi)/2 using the current-particle ESS; if ESS $\geq$ 0.90H_e, keep that passing point as the lower end (lo = mid), otherwise set hi = mid. Use the final lo as the bisected endpoint. The step is forced iff lo$-\varphi < 1/$T_e, strictly: in that case replace the endpoint by min(1, φ+1/T_e) and increment `forced_pilot_steps`; otherwise retain lo.

Snap only a forced step. When the step is forced, set the endpoint to min(1, φ+1/T_e); if this value is $\geq 1-10^{-12}$, set it to exactly 1 (RC-2, ASSUMPTIONS §17; `cpp/v3_calib.h`, `pilot_next_phi`). Never snap a non-forced bisected endpoint lo. The direct endpoint 1 needs no snap. A snapped forced step still counts in `forced_pilot_steps`. All applicable progress and interval checks remain in force, including the pilot's stated exploratory floor exception to its ESS target. Perform weighting, multinomial resampling and one move round at each selected temperature. End exactly at 1. Then run L burn-in rounds and four measured rounds at 1, followed by **one** local search

from the global best at that moment. The pilot floor is confined to exploration; it is not used in final production stages. The snap prevents an extra rounding-only stage that can cause `too_many_intervals` or `schedule_limit` at T_e ∈ {2000,4000,16000}; it bypasses no check.

Candidate leaders and V (M §6.7; C R2–R3). Maintain two distinct stores over every model scored during the current fit, including tables, rejected proposals, local searches and cached or later-evicted models: the top 20p by log γ_0 and the top 20p by target score F (equivalently log γ_1). Break ranking ties lexicographically on sorted original indices. Preserve these stores across tiers. They are distinct from the HPM running best and from the top-(40p+1) near-tie store. Set V to their deduplicated union with the empty model, so |V| ≤ 40p+1. Freeze V during each grid construction.

Restricted grid (M §6.7). For frozen V define

```
F_V(t) = log Σ_{S∈V} γ0(S) exp{t d(S)}
ρ_V(u,v) = exp{2F_V(v) − F_V(u) − F_V(2v−u)},  u<v.
```

ρ_V is the population ESS fraction under the restricted law q_u(· |V), not the empirical ESS of a particle population and not a full-space ESS guarantee. Evaluation at 2v−u > 1 is a finite-sum moment calculation; it does not add an over-tempered production stage.

M §6.7 bounds each grid construction by 60 · |V| · T_e scalar log-sum-exp evaluations. Record this cost rather than assuming it is negligible relative to model scoring.

Begin at u = 0 with the successive pilot temperatures as targets b. If ρ_V(u,b) ≥ 0.90, choose b as the next endpoint. Otherwise bisect on (u,b] with u held fixed, at most 60 halvings, and choose the largest tested passing endpoint lo. For each endpoint v about to be appended (the target b or the bisected lo), if v ≥ $1-10^{-12}$, set v = 1. Then require v > u and ρ_V(u,v) ≥ 0.90 at the snapped v, append v, and fail if the grid then has more than T_e intervals. Non-progress or failed validation makes construction fail. Continue from the appended point towards the same target until it is reached, then proceed to the next pilot target. This is the prototype's RC-2 rule (lead ruling 2026-10-22 (RULINGS_3), N1). Do not introduce a final-run step floor to rescue a failed grid.

Diagnostic rebuilds (M §6.7–§6.8; C R4, R6). Construct a preliminary grid from the pilot and V. For at most ℓ rebuild iterations:

1. Run four diagnostic islands with H_e fresh iid π_0 draws each on the current grid. Use one move round per stage, then L burn-in rounds and four measured rounds at 1. Initial draws and all proposals are scored.
2. After **all four** diagnostic islands, perform one local search from the then-current global best; do not perform four island-specific local searches.
3. Update V from the persistent leaders and rebuild the grid from the pilot targets. Stop if the rebuilt temperature vector is exactly equal to the preceding vector. Equality of V is not required.

After ℓ iterations without equality, set `grid_unconverged`. At tier 0 this triggers escalation; at tier 1 use the last successfully rebuilt grid and report the flag. Failed construction triggers escalation at tier 0; at tier 1 it returns `schedule_limit` with the best scored model and unavailable evidence/PIPs. A forced pilot step triggers escalation but does not itself invalidate a successfully constructed candidate grid.

Checks and escalation (M §6.9; C R5, §10). Any of the following triggers tier 1 if currently at tier 0:

Trigger	Rule
E1	Some diagnostic island's own best F is more than 1 nat below the overall best. Its own best includes its initial draws and proposals, but excludes local searches and inherited leaders.
E2	The sample SD of the four diagnostic log-evidence estimates exceeds 1 nat; denominator 4−1.
E3	A calibration local search strictly increases the global best F, comparing immediately before and after that search. An equal-F tie-rule change does not trigger E3.
Forced pilot	At least one floor-forced pilot step.
Grid	Failed construction or `grid_unconverged`.
Local budget	`local_limit` from a calibration local search.

Aggregation convention resolved by the prototype. E1 and E2 are taken from the **last** completed diagnostic rebuild iteration, with E1 compared against that iteration's post-local-search global best. E3 and `local_limit` accumulate by logical OR over the pilot local search and every diagnostic local search in that tier. E2 uses the usual sample SD, denominator 3 for four islands. `checks_passed_calibration` is true iff none of the listed calibration triggers is set in the selected tier. Preserve triggers by tier and sum `forced_pilot_steps` over attempted tiers. These conventions resolve M/C's aggregation wording using `prototype_core/v3_calib.R`; they do not change trigger thresholds.

At tier 1 no further escalation is available. Retain the checks and flags even when the run proceeds. E1, E3 and `local_limit` stay in the implementation; DP2 documents them as **rarely active under production budgets**, with no guarantee that they detect an error. Passing any collection of checks does not certify unseen modes, global optimality, mixing, or PIP/evidence accuracy.

Final run (M §6.8; C R4, R7–R8). Freeze the grid, chosen tier e, N_e and L before using streams 10–13. In each of four final islands, draw and score N_e new iid π_0 models. On every frozen interval reweight, record the evidence increment, multinomially resample and perform one round, including the terminal interval. At $\varphi = 1$ run L burn-in rounds followed by four measured rounds. Do not include burn-in recordings in the PIPs. After all islands, perform one local search from the global best. Recompute E1–E3 and the empirical-ESS check using the final outputs as **flags only**; do not rerun, escalate or replace reported estimates in response.

For final E1 compare each final island's own best with the global best after final local search. Final E2 is the sample SD of the four final $\log\hat{Z}_r$, with denominator 3. Final E3 is the strict increase in global best F immediately across the final local search. Set `checks_passed_final = !(E1 or E2 or E3 or low_empirical_ESS)`; `local_limit` remains a separate flag. These are the prototype's explicit check conventions (`prototype_core/v3_run.R`) resolving M §6.9.

I.6 Local search, HPM, PIPs, evidence and statuses

Local search (M §6.10; C R4–R5). Freeze the current global best as centre and score every feasible single flip and every included/excluded swap. Skip additions that would exceed K. All scored neighbours update the global HPM even if they do not become the next centre. Choose the best **strict F improvement** as the next centre; among equally scoring improving candidates use

lexicographic order on sorted original-index vectors, with the proper prefix first. An exact-F tie with the centre is not a local improvement. Take at most ℓ such moves, then scan once more. Stop earlier at a scan with no strict improvement. If the extra scan still improves F, set `local_limit`; there is no restart. The best model scored in that scan still enters the returned HPM. On the SMC branch, set `local_certified` only if the returned selected model was itself a scanned centre with no improving neighbour. If an exact-F tie changes the reported HPM away from that certified centre, leave `local_certified` false. Completed exact enumeration may set it directly as in §I.3 (lead ruling 2026-10-22 (RULINGS_2), findings 8–9). The search never changes particle states. Archive each invocation's phase, tier, centres, path, completed scans, termination and certification. R14, recovered in `ASSUMPTIONS.md` §10, restates these strict-improvement and tie conventions; it adds no new move.

HPM and ties (M §6.10; C R9, R12). The selected model is the maximum of F(S) over **every model ever scored in the fit**: tables, initial draws, both feasible Gibbs configurations, rejected proposals, pilot, diagnostics, final stages and terminal rounds, and all local searches. Keep it as a running best independently of cache eviction. Scores tie only at exact floating-point equality. Then prefer smaller |S| and then the lexicographically smallest sorted original-index vector, comparing smaller original indices first and a proper prefix before the longer vector. No tolerance reorders the HPM.

Maintain a separate top-(40p+1) store for the informational list of models within 10^{-9} nat of the best: include F(S) $\geq$ best F $- 10^{-9}$, with equality included. This list never changes the strict ordering. Set `near_tie_truncated` iff the store is full and its smallest stored F is $\geq$ best F $- 10^{-9}$ (lead ruling 2026-10-22 (RULINGS_2), finding 7). Report the selected model, score and first-score provenance, including phase and mechanism. For island diagnostics, an island's own maximum excludes inherited leaders and local-search scores (C R5).

One PIP output (M §6.11 amended by DP2 D2; C R8). In each of the four measured final rounds, record o_j immediately before j's Gibbs draw at the state current at that point in the round. For each island r and variable j, average its recordings over all N_e particles and all four measured rounds to obtain PIP_rj. Report

```
PIP_j = (1/4) Σ_{r=1}^4 PIP_rj
MPM = {j : PIP_j > 1/2}.
```

This equal-island Rao–Blackwellised PIP is the sole pooled PIP estimate. The secondary evidence-weighted pooling is dropped entirely from reported outputs. Island components may be retained as diagnostics for this primary estimate. The MPM is reported as computed even if it contains more than K variables; it is not the HPM decision. The normalised PIP estimate is not asserted unbiased at finite budgets. Dropped columns are marked never selectable; report retained results through the preprocessing index map.

Evidence (M §6.12; C R7). For final island r,

```
log Ẑ_r = log m(∅) + log Z_0
          + Σ_t log[(1/N_e) Σ_i exp{(φ_t-φ_{t-1})d(S_i^(t-1))}]
log Ẑ = log-sum-exp(log Ẑ_1,…,log Ẑ_4) - log 4.
```

Report island and pooled evidence on the full working scale and relative to m(∅), with the 95% one-sided lower bound $0.05\hat{\mathbf{Z}}$ (or log $\hat{\mathbf{Z}}$ + log 0.05). This Markov bound is not a two-sided accuracy

interval. W_K occurs inside γ_0 exactly once, and the m(∅) multiplier occurs exactly once on the full scale. The per-stage log increments are retained. Conditional on the frozen calibration, unbiasedness is a statement about each **linear-scale** $\hat{Z}_r$ under the prescribed SMC construction; $\log \hat{Z}$ is not unbiased. The project proof status remains separate from implementation and numerical verification.

Statuses (M §2). Return exactly one of the following:

Status	Meaning and available results
`exact`	All feasible models enumerated; exact posterior outputs subject to reported numerical validation.
`completed`	Fresh final SMC run completed; diagnostic flags can still be set.
`schedule_limit`	No admissible grid within the stage ceiling at tier 1; return the best scored model; evidence and PIPs unavailable.
`numerical_failure`	Scorer failed after refactor/QR fallback; estimates unavailable; return any best scored model explicitly labelled incomplete.
`resource_limit`	Memory preflight, exact-branch time projection, or the during-run time guard in §I.1 exceeds the allocation; return resource estimates/timings and keep all results produced so far, explicitly labelling partial results.
`y_constant`	Constant response after valid-input and retained-predictor checks; empty selected model; evidence/PIPs unavailable.
`no_predictors`	All predictors dropped; empty selected model; evidence/PIPs unavailable.
`invalid_input`	Inputs fail the stated input checks.

Flags and certification (M §2, §6.7–§6.10; DP2 D4). Save `escalated` when tier 1 is actually used; `checks_passed_calibration` and `checks_passed_final`; the `forced_pilot_steps` count; `grid_unconverged`; `local_limit`; `low_empirical_ESS` when any final-stage ESS is below 0.5N_e; `near_tie_truncated`; `local_certified`; and `global_certified`. Also retain the individual E1–E3 check outcomes and `cut_group`, the flagged-component count and component size/block-span/original-index records (§I.3). `global_certified` is possible only under `exact`, when the prototype's heuristic comparison-margin rule in §I.2 passes; report its bound with the label "heuristic (not a validated interval bound)" (lead ruling 2026-10-22, DP2_DECISIONS addendum). Diagnostic success never sets it. A local certificate concerns the returned selected model's feasible neighbourhood; completed exact enumeration can establish it directly (§I.3). An exact-F tie changing the reported HPM away from a certified SMC search centre leaves `local_certified` false.

Required fit record (M §2; C §6–§7). In addition to the selected model, near ties, primary PIPs, MPM, evidence, status and flags, save the blocks and neighbour graph, pilot/final grids and rebuild grids, selected tier and budgets actually used, stream map and seeding convention, score counts, per-stage empirical ESS and restricted ρ_V, distinct particle counts after resampling and after moving, acceptance per mechanism, evidence increments, candidate-set sizes and rebuild count, phase timings, resource estimates, numerical fallback routes, local-search records, and the preprocessing map. Save calibration estimates with explicit calibration labels. Preserve enough saved source/configuration, seed, environment and output provenance to audit the fit. This is operational bookkeeping; it does not add the excluded profiling observer or scoring ledger. If simulation truth is supplied separately for evaluation, score and compare it only after the fit/HPM is frozen; it never enters fitting, leaders, V or searches.

For every stage record `distinct_resampled` and `distinct_moved`. Following C §6 and `v3_run.R`, `stage_table`, report Gibbs state-change count/max(Gibbs steps,1), U acceptances/max(U executions,1), N acceptances/max(N executions,1), and I acceptances/max(I executions,1), retaining the counts and boundary self-loop/no-score-rejection counts (lead ruling 2026-10-22 (RULINGS_2), finding 11). U executions exclude size-0/p self-loops; N executions exclude empty-state self-loops but include nonempty-state attempts whose candidate list is empty. Gibbs reports state changes, not an MH acceptance rate. Label phase, tier, rebuild/island and stage so calibration records cannot be confused with final records.

I.7 Connection to the benchmark study

The method operates on the paired X and y identified by the study interface (§IV.1.4), applying the complete preprocessing in §I.1 even when stored X was already centred and scaled. It now standardises y as well as X. Retained p determines all method budgets and thresholds. The SMC arm reports its v3 version/source hash, target scale, preprocessing map, seed, status and flags alongside the study identifiers, so the external exact reference and the SMC estimate can be compared on the same working-evidence scale.

- Read the same sealed `inputs/` files keyed by `dataset_id` and Y column `scale_key`, without consulting the fitting-sealed truth or using cross-validation folds. Preserve `dataset_id = p{pppp}_S{s}_r{rrr}` and `case_id = dataset_id_scale_key` (§IV.1.1, §IV.1.4).
- Write the study's rule rows with `method_id = "SMCBVS"` and `rule_id = "hpm"` for the selected HPM or `rule_id = "pip05"` for the MPM from the sole primary RB PIP. Use the arm's own `results/<run_id>/` tree. The join key remains (`case_id`, `method_id`, `rule_id`).
- Preserve `x_sha256` and `y_sha256` of the **stored input** on every joined row. Internal standardisation does not change these input hashes. Keep the preprocessing map separately. PIP columns for this arm come from the primary RB PIPs, including the active-coordinate PIP fields and `max_pip_null` when the external scoring step supplies the truth mapping.
- Freeze the fit/HPM before the scoring step fills `tp`, `fp`, `fn`, `exact`, `error_type` or truth-versus-HPM diagnostics. Truth never changes a fit result. Record whether truth is present in the retained top-model set only during this separate evaluation.
- `runtime_s` covers the whole fit: preprocessing, tables, calibration/rebuilds and any escalation, four fresh final islands, and local searches. Preserve method-specific status and flags. **OPEN — study adapter:** the sources do not adopt the `role` mapping of hpm/pip05 or the exact mapping of v3 statuses/flags to the benchmark `converged` and `diag_flag` fields; do not infer these from a diagnostic warning. Resolve the adapter without changing the method or the fixed study failure-denominator rules.

At n = 200 the production settings are K = 30 and B = 5 for p = 30, and K = 100 and B = 14 for p = 500 and 1000. All three settings have $N_0 = 2500$, $H_0 = 512$ and $T_0 = 1000$; their ℓ budgets are 8, 10 and 11 respectively. Four final islands follow calibration. These changes define the SMC arm; they do not change the fixed benchmark methods F1–F5 or A1–A3, scenarios, seed pairing, number of datasets, or benchmark NOTS jobs. Earlier output produced under round-1 settings is historical output and must not be relabelled v3.

I.8 Complete settings table

The table collects values and decision rules, including boundary, diagnostic and numerical settings. Section references give the authoritative provenance; derived recurrences only expand a stated exact algorithm.

Component	Value or rule	Source
Fit input	Numeric X, y; one integer s ∈ [1, $2^{31}-1$]; job memory/wall-time allocation	M §2
Input checks and precedence	Finite entries, n ≥ 3, input p ≥ 1; then exact-constant column removal; then exact-constant y	M §4.1–§4.3; C §1
Predictor/response scaling	Maximum-absolute rescale, mean-centre, divide by sample SD (n−1)	M §4.4
Original indices	Sorted 1-based original indices everywhere, including ties and dropped-column map	C R12
Duplicate report	Connected components at $\lvert r\rvert \geq 1-10^{-10}$; duplicates retained	M §4.5
Slab	g = n, ridge τ = 1	M §5
Noise prior	IG(a_0,b_0), a_0=b_0=0.01	M §5
Model prior	BB(1,p), w(k)=(1/2)/choose(2p−1,k), normalised by W_K	M §5; C §2
Cap K	min(p,⌊n/2⌋)	M §3, §5
Intercept/exponent	No intercept; centred/scaled y; exponent a_0+n/2	M §4, §5
Score	F=logϖ+logm; H=(n+1)G+I; Q and C_n as §I.2	M §5
B	min(⌊√n⌋,⌊√p⌋,16)	M §3
ℓ	$\lceil \log_2(n+p+1) \rceil$ local moves and rebuild iterations	M §3
L	max(4,$\lceil \log_2(n+1) \rceil$) burn-in rounds at 1	M §3
N_0	max(2500,n,p) final particles/island	M §3
H_0	$\lceil 512N_0/2500 \rceil$ pilot and diagnostic particles/island	M §3
T_0	max(1000,ℓ^2) stage ceiling	M §3
Islands	Four diagnostic islands/rebuild; four final islands	M §3, §6.8
Tier	e∈{0,1}; particles and stage ceiling multiply by 4^e	M §3, §6.9
Exact branch	$\Sigma_{k\leq K}$choose(p,k)≤$(5H_0+4N_0)$p, saturating integer decision	M §6.2
Clustering	Complete linkage on 1−|r|; equal-distance merges ordered by lexicographic union	M §6.3; C R13
Dendrogram	Left child has smaller minimum original index; left-to-right leaf order	C R13
Cut and packing	Split nodes >B top-down left first; whole-cluster next-fit packing	M §6.3; C R13
Block tables	All U⊆C_b, full response; log r_b=logm(U)−logm(∅); exponent 1	M §6.3
Table scoring	HPM and both leaders updated; table scores bypass score cache	C R2–R3

Component	Value or rule	Source
Start	$\gamma_0=\varpi\prod r_b$; exact size convolution through K, exact iid backward draw	M §6.3
Start variate order	Total size, backward sizes while positive, forward nonempty submodels by binary code; categorical cumulative >u (§I.3)	C §4.1; prototype `cpp/v3_start.h`
Discrepancy/bridge	$d=\log BF-\Sigma\log r_b$; $\gamma_\varphi=\gamma_0\exp(\varphi d)$, $\varphi\in[0,1]$	M §6.3
Neighbour threshold	$\rho_n=\sqrt{(2\log p/n)}$, strict exceedance	M §6.4
Neighbour list	Always largest-|r| partner, ≤B before symmetric union; smaller original index first on ties; none at p=1	M §6.4; lead ruling 2026-10-22 (RULINGS_2), finding 12
Round	p Gibbs + 1 uniform swap + B neighbour swaps + 1 independence proposal	M §6.5
Step ordering	Independent state-independent uniform permutation for each particle/round	M §6.5; C §4
Gibbs scored set	Both feasible configurations; infeasible addition omitted; $o_j=0$ at $\|S\backslash j\|=K$	C R1
Gibbs recording	Immediately before own draw, at current within-round state	C R8
Swap boundaries	Uniform self-loop at size 0 or p; neighbour self-loop at empty; empty proposal list rejects unscored	M §6.5; C §4
MH ratios	Uniform target ratio; neighbour target ratio×forward/reverse list sizes; independence $\exp(\varphi\Delta d)$	M §6.5
Resampling	Fresh multinomial every reweighting including terminal; inverse-CDF guide table O(N) setup, O(1) expected per draw; same ancestor as prototype; alias/systematic forbidden	M §6.6; C R7; lead ruling 2026-10-22 (RULINGS_2), G2
Evidence timing	Increment from incoming particles before resampling, then one move round	C R7
Pilot ESS	Direct 1 if ESS≥0.90H_e; else 60 halvings on $(\varphi,1]$, lo=φ, hi=1, passing midpoint becomes lo	M §6.6; lead ruling 2026-10-22 (RULINGS_2), finding 14
Pilot floor	Forced iff bisected lo−φ<1/T_e; use $\min(1,\varphi+1/T_e)$, count it; direct endpoint 1 is not forced; no final floor	M §6.6, §6.8; lead ruling 2026-10-22 (RULINGS_2), finding 14
Candidate leaders	Separate top 20p by $\log\gamma_0$ and top 20p by F, over all scored; survive tiers	M §6.7; C R3
V	Deduplicated leaders plus ∅; ≤40p+1; frozen per construction	M §6.7
Candidate grid	$\rho_V(u,v)=\exp[2F_V(v)-F_V(u)-F_V(2v-u)]\geq 0.90$	M §6.7
Grid bisection/failure	≤60 halvings; after endpoint snap require v>u and $\rho_V\geq 0.90$; >T_e intervals/non-progress/ failed validation still fail	M §6.7; lead ruling 2026-10-22 (RULINGS_2), G3

Component	Value or rule	Source
Endpoint snap	A forced pilot floor step or a grid point about to be appended, with value $\geq 1-10^{-12}$, becomes exactly 1. Non-forced pilot endpoints are never snapped. Grid checks apply after the snap.	ASSUMPTIONS §17 RC-2; lead ruling 2026-10-22 (RULINGS_3), N1–N2
Rebuild stop	Exact equality of successive temperature vectors; $\leq \ell$ iterations	M §6.7; C §10
Diagnostic streams	Continue across rebuilds; no re-initialisation	C R6
E1	Diagnostic-island own best more than 1 nat below overall best	M §6.9; C R5
E2	Sample SD (denominator 3) of four diagnostic $\log\hat{\mathbf{Z}}$ exceeds 1 nat	M §6.9; prototype `v3_calib.R`
E3	Strict global-best F gain over a calibration local search	M §6.9; C R5
Other escalation triggers	Forced pilot steps, grid failure/non-convergence, local_limit	M §6.9
Check aggregation	Last diagnostic iteration for E1/E2; any calibration local search for E3/local_limit; selected-tier no-trigger Boolean	M §6.9; prototype `v3_calib.R`
Escalation	At most once before final randomness; keep score leaders; preflight tier 1	M §6.1, §6.8–§6.9, §8.6
Final	Fresh iid π_0 draws; frozen grid and N_e; L burn-in + 4 measured rounds; one final local search	M §6.8; C R4
Local-search scope	Once after pilot, once after each set of four diagnostic islands, once after all final islands	C R4
Local-search budget	$\leq \ell$ strict-improvement moves + one scan; feasible flips/swaps; no particle alteration	M §6.10
HPM ties	Exact floating-point F equality only; smaller size, then lexicographic original indices (smaller index first, proper prefix first)	M §6.10; C R9, R12; lead ruling 2026-10-22 (RULINGS_2), findings 6/12
Local improvement ties	Strict F improvement only; lexicographic among best improvements, proper prefix first	M §6.10; R14 in ASSUMPTIONS §10; lead ruling 2026-10-22 (RULINGS_2), finding 6
Local certificate	Returned SMC HPM must itself be a scanned centre with no improvement; exact-F tie changing HPM leaves false; exact enumeration may set true directly	M §6.10; lead ruling 2026-10-22 (RULINGS_2), findings 8–9
Near ties	F$\geq$best-10^{-9} (inclusive); top 40p+1 store; truncated iff full and minimum stored F$\geq$best-10^{-9}	M §6.10; lead ruling 2026-10-22 (RULINGS_2), finding 7
Score cache	Capacity 500p; table scores excluded; eviction never discards HPM/leaders	M §8.6; C R2–R3
Primary PIP	Mean o_j over particles×4 measured rounds/ island, then equal mean over 4 islands	M §6.11; C R8; DP2 D2
Secondary PIP	Removed from reported output	DP2 D2
MPM	Primary PIP>1/2, reported even if size>K	M §6.11

Component	Value or rule	Source
Evidence pooling	Arithmetic mean of 4 linear-scale $\hat{Z}$_r, calculated by log-sum-exp	M §6.12
Evidence lower bound	$0.05\hat{Z}$, one-sided 95%; full scale and relative to m(∅)	M §6.12
Final ESS warning	Any final-stage empirical ESS<0.5N_e	M §2, §6.9
Final checks	E1–E3 and empirical ESS recorded as flags only	M §6.9
Check Boolean, final	Pass iff no E1/E2/E3/low_empirical_ESS; final E3 strict pre/post-local F gain	M §6.9; prototype `v3_run.R`
Warning retention	E1/E3/local_limit unchanged, rarely active at production budgets	DP2, "Not adopted now"
Stream IDs	0/1–4 tier 0; 5/6–9 tier 1; 10–13 final	M §6.1; C §5
Seeding	Documented reproducible independent derivation from (seed,stream[,particle])	C §5
Serial PRNG convention	x=(seed«32)+stream; four SplitMix64 outputs seed xoshiro256**; uniform=(next»11)2^{-53}	C §5; prototype `cpp/v3_util.h` (implementation convention)
Uniform list choice	Original-index sorted list, position floor(um)+1 for u∈[0,1)	C §4.1
Bernoulli/MH	Success iff u<probability	C §4.1
Round permutation	Fisher–Yates on canonical labels	C §4.1
Resampling variates	N uniforms in stream order; u=0→2^{-1074} without another draw; first W_k≥u_eff, W_N=1; prototype cumulative arithmetic (§I.4)	C §4.1; lead ruling 2026-10-22 (RULINGS_2), G2/finding 13
Draw consumption	Initial complete π_0 draws by particle; per stage N resample draws, then particles 1…N: m−1 order draws then G/U/N/I draws exactly as in §I.4; terminal rounds same order without resampling	Prototype `v3_run.R`/C++; lead ruling 2026-10-22 (RULINGS_2), G4
Stage diagnostics	Distinct resampled/moved particles; Gibbs change and U/N/I acceptance proportions with execution/self-loop/no-score counts (§I.6)	C §6; prototype `v3_run.R`; lead ruling 2026-10-22 (RULINGS_2), finding 11
Stable sums	Log-sum-exp throughout	M §6.13
Q recomputation	Non-negative formula when Q<$10^{-8}y^Ty$; no clipping	M §6.13
Score fallback	Cholesky failure→rebuild G_S from X_S→fresh refactor→augmented QR→numerical_failure; never artificial $-\infty$	M §6.13; lead ruling 2026-10-22 (RULINGS_2), finding 5
Preflight	Byte formula in §I.1 with prototype convolution/exact-store coefficients and no extra term; repeat before tier 1; SMC memory only, exact also E×measured s(k*)	M §8.6; prototype `v3_fit.R`; lead ruling 2026-10-22 (RULINGS_2), G1; lead ruling 2026-10-22 (RULINGS_3), N5
Exact-branch score timing	Time 200 scores at k=*min(K,⌈p/2⌉) on first min(p,max(64,k))* standardised columns; operational measurement, not a statistical setting	Prototype `v3_fit.R`, `v3_time_scores`; lead ruling 2026-10-22 (RULINGS_3), N4

Component	Value or rule	Source
Resource allocation	Actual job request; defaults mem_gb=8 GiB and hours=24	C §7; lead ruling 2026-10-22 (RULINGS_2), G1
Runtime guard	After every bridge stage of each pilot/diagnostic/ final island, each grid construction and each local search: stop if elapsed+just-completed-operation duration>hours · 3600; resource_limit, retain partial results; operational guard, not a statistical setting	lead ruling 2026-10-22 (RULINGS_2), G1; lead ruling 2026-10-22 (RULINGS_3), N6
Certification	Local criterion as above (exact enumeration needs no extra scan); global only exact and F(S)−η(S)>max other [F(S)+η(S)]; label bound heuristic (not a validated interval bound)	M §2, §6.10; C R9 as amended by lead ruling 2026-10-22, DP2_DECISIONS addendum
Numerical error heuristic	$\eta(S)=64\varepsilon[\nu(y^Ty/Q)(1+k\kappa_H)+k(\kappa_H+\kappa_A)+\lvert\log C_n\rvert+\nu\lvert\log(2b_0+1)\rvert]$; double $\varepsilon=2^{-52}$; factor-diagonal κ and full definitions in §I.2	lead ruling 2026-10-22, DP2_DECISIONS addendum; prototype `cpp/v3_score.h`
Cut-group report	`cut_group`: an uncapped $\lvert r_jl\rvert\geq\sqrt{(2\log p/n)}$ graph component spans ≥2 blocks; report component count, sizes, block spans and original members; informational, B unchanged; exact branch NA	lead ruling 2026-10-22, DP2_DECISIONS addendum (D4); lead ruling 2026-10-22 (RULINGS_2), finding 10
Excluded switches	No force_smc, B_override, pair moves, observer/ watch sets, exact-reference machinery or trace replay	Task source/exclusion rules; C R10–R11, §8, §11; R17–R18

I.9 Lead rulings and limits of deployment claims

Resolved — D4 cut-group report. The lead ruling 2026-10-22, DP2_DECISIONS addendum supplies the predicate and output fields in §I.3. It explicitly permits the uncapped threshold graph when the move graph differs; that is the reading applied here. The reporting predicate uses ≥ and no cap/forced partner. It changes no move, B, status or budget.

Resolved — R9/R16 numerical certificate. The same lead addendum adopts the prototype's heuristic η and comparison rule in §I.2, replacing the earlier requirement for a validated interval bound. Output carries the required "heuristic (not a validated interval bound)" label. The exact branch alone can set `global_certified`; this is a numerical safety rule with the stated limitation, not a new statistical tuning parameter. Neither former source item remains OPEN.

Source-location note. The supplied CORE_SPEC contains R1–R13; R14–R18 are recovered in context-only `ASSUMPTIONS.md` §10. R14 agrees with M's strict local-search rule. R15 concerns profiling logs; R17 is the excluded B override; R18 concerns excluded observer aggregates. The source hierarchy does not promote those context-only items into method changes. The explicit prototype conventions above resolve PRNG and check aggregation details. The later lead ruling 2026-10-22 (RULINGS_2) adopts RC-2 endpoint snapping and the zero-uniform guard, prescribes inverse-CDF guide-table lookup preserving each ancestor, and fixes the complete draw order. Endpoint snapping avoids an extra rounding-only stage; strict progress and interval-inequality validation remain binding. The same ruling supplies the known-dimension memory estimate, 24-hour default

and operational runtime guard, and resolves findings 5–14 in place. These rulings supersede the earlier exclusions of the snap/zero guard and the earlier resource wording. The later lead ruling 2026-10-22 (RULINGS_3), N1–N6, specifies the prototype's RC-2 exactly (forced pilot steps only; every grid point about to be appended), removes the redundant pilot snap-progress instruction, requires repeated backward guide-table adjustment, supplies the prototype timing procedure and memory coefficients without an extra term, and fixes the time-guard checkpoints. The remaining prototype differences are recorded in CHANGES.md.

Passing diagnostics is not a claim that the global mode was found or that PIP/evidence error is small. The restricted candidate grid controls one-step weight dispersion on V only; it cannot control an unscored mode, full-space ESS, occupancy or mixing. All resource budgets and escalation rules remain finite declared budgets. This section states the method to deploy, not new theorem guarantees.

Part II. Simulation design

This part is one account of the study design: regimes, data-generating model, truth, covariance scenarios, signal-to-noise, study size, seeds, pairing, conventions, and how the data are generated, sealed and verified. The checks of the truth placement and nominal SNR are computed by `tools/scenario_check.py`; its output is `tools/scenario_check.out`.

II.1 Regimes and scope

The six covariance designs are the project's S1–S6. Sources:

- `01_Methodology_and_Theory/Methodology/SMC_Composite_Likelihood_Methodology.tex`, Sec. "Simulation scenarios". This defines them at n=100, p=30.
- `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14/scripts/design.R`. This generalises them to any p using blocks of five, and was used at n=200, p=500.

The study keeps those six covariances. It sets n=200 everywhere and arranges the study in two regimes:

Regime	n	p	Role
L (low-dimensional)	200	30	n > p. **Used only to assess the accuracy of the SMC-BVS estimates of the evidence Z and of the PIPs** against **exact enumeration of all 2^30 models**, which gives the true posterior as a reference (D1). No benchmark method is run here.
H (high-dimensional)	200	500, 1000	p > n. **Bayesian variable selection (support recovery) and the benchmark comparison with F1–F5 and A1–A3 are run only here** (D1). This is the main selection and scalability test, and no exact reference exists.

The benchmark panel is fixed (Part III, §III.1) and runs in Regime H only. The SMC-BVS arm is out of scope for the benchmark runs except for the pairing interface (§IV.1.4); its methodology is Part I. Regime L serves the SMC arm alone (SMC-BVS against exact enumeration).

II.2 Common data-generating model

The model is the same for every p:

1. Raw rows $\tilde{x}_i \sim N_p(0, \Sigma_s)$, i = 1..200.
2. Centre each column, then divide it by its sample SD (denominator n−1) to get X.
3. $y = c \cdot X\beta + \varepsilon$, with $\varepsilon \sim N_{200}(0, I)$. So $\sigma = 1$, y is **not** centred, and every method fits **without an intercept** (where the software allows it).

The generator code is identical to `generate_case()` in `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14/scripts/design.R`: RNG Mersenne-Twister / Inversion / Rejection; draw order X first via `matrix(rnorm(n*p),n) %*% chol(Σ)`, then ε. The full specification is in §II.9.

The signal scale c gives paired weak-signal versions of each dataset, as in the Oct-14 weak-signal and benchmark campaigns. Each version uses the same X and the same ε, with `y_c = c·(Xβ) + ε`. The five scales are c = 1, .75, .5, .35, .25 (`scale_key` ∈ {c100, c075, c050, c035, c025}).

II.3 Truth by p

Truth (fixed across replicates and scenarios; its size depends on p). Revised 2026-10-19 at Felipe's request:

- **5 signals at p = 30.**
- **$\lfloor\sqrt{n}\rfloor = \lfloor\sqrt{200}\rfloor$ = 14 signals at p = 500 and p = 1000.** "√n" is read as the number of signals, rounded down. Rounding up would give 15 (open decision O1, Appendix A).

The coefficients follow the pattern (1.4, −1.0, 1.8, −1.2, 0.8, −1.6), cycled:

- p = 30 uses the first five values: (1.4, −1.0, 1.8, −1.2, 0.8).
- p = 500 and 1000 use the pattern twice, then 1.4, −1.0: (1.4, −1.0, 1.8, −1.2, 0.8, −1.6, 1.4, −1.0, 1.8, −1.2, 0.8, −1.6, 1.4, −1.0).

Placement:

- Each signal sits in its own population block of five.
- Its position within the block cycles (3, 2, 4, 1, 5, 3, …).
- The active blocks are evenly spaced, $1 + d \cdot (k-1)$ with d = 1, 7, 13 for p = 30, 500, 1000. Because $d \equiv 1 \pmod 6$, signal k always meets the same S6 correlation, so each coefficient keeps its ρ partner at every p.

p	s_0	Active indices	Population blocks	S6 ρ at each signal
30	5	3, 7, 14, 16, 25	1–5 (block 6 has no signal)	.4 .5 .6 .7 .8
500	14	3, 37, 74, 106, 145, 178, 213, 247, 284, 316, 355, 388, 423, 457	1, 8, 15, …, 92 (of 100)	.4 … .9, .4 … .9, .4 .5
1000	14	3, 67, 134, 196, 265, 328, 393, 457, 524, 586, 655, 718, 783, 847	1, 14, 27, …, 170 (of 200)	.4 … .9, .4 … .9, .4 .5

Every design except S2 is invariant to permuting the blocks, and within a block every design except S2 is exchangeable. So between p = 500 and p = 1000 only the number of null predictors changes. Between p = 30 and p > n, the support size (5 vs 14) also changes. In S2 at p ≥ 500 the signals are at least 31 indices apart, so they interact negligibly; at p = 30 they are 2–9 apart. Placement checked in `tools/scenario_check.out`.

II.4 The six covariance scenarios

Notation:

- H_jk = 1 if j and k are in the same block $\{5(b-1)+1, \ldots, 5b\}$.
- J is the all-ones matrix.
- There are p/5 blocks: 6, 100 or 200.

	Design	Σ_s	Challenge
S1	Independent	I_p	Baseline with no correlation
S2	AR(1)	0.5^|j−k|	Local substitutes with no block boundary

	Design	Σ_s	Challenge
S3	Moderate blocks	0.7H + 0.3I	Each signal has four correlated inactive proxies
S4	Strong blocks	0.85H + 0.15I	The same, with harder within-block ambiguity
S5	Blocks + global	0.3J + 0.4H + 0.3I	Within-block correlation 0.7 and between-block correlation 0.3. Hardest for the composite start.
S6	Heterogeneous blocks	Block b has ρ_b = 0.4 + 0.1 · ((b−1) mod 6); zero between blocks	Correlation strength varies across blocks

At p=30, S1–S6 reproduce the methodology's covariance matrices exactly. All Σ_s are positive definite; the smallest eigenvalue is 0.10 (S6).

II.5 Nominal signal-to-noise ratio

Nominal SNR = $c^2 \cdot \beta' \Sigma_s \beta$ ($\sigma = 1$). Computed by `scenario_check.py`:

p	Scenario	c = 1	0.75	0.50	0.35	0.25
30	S1, S3, S4, S6	8.28	4.66	2.07	1.01	0.52
30	S2	7.00	3.94	1.75	0.86	0.44
30	S5	6.77	3.81	1.69	0.83	0.42
500, 1000	S1–S4, S6	24.64	13.86	6.16	3.02	1.54
500, 1000	S5	17.44	9.81	4.36	2.14	1.09

Per-signal effect sizes are the same at every p. Total SNR is higher at p ≥ 500 only because there are 14 signals instead of 5.

- **S2 at p = 30 is lower:** the nearby opposite-sign signals partly cancel.
- **S5 is lower:** between-block correlation 0.3 combined with mixed signs.

As in the methodology, coefficients and noise are fixed and SNR is not equalised.

II.6 Size of the study, scales and pairing

Regime	p	s_0	Scenarios	Scales c	Replicates	Datasets (X, ε)	Cases (dataset × scale)	Benchmark cases per method	Benchmark primary fits (cases × 8 methods)	Regime-L runs (D1)
L	30	5	S1–S6	1, .75, .5, .35, .25	100	600	3,000	— (no benchmarks, D1)	—	3,000 SMC-BVS runs and 3,000 exact enumerations (one per case)
H	500	14	S1–S6	same	100	600 (same X and ε as Oct-14; new y)	3,000	3,000	24,000	—
H	1000	14	S1–S6	same	100	600	3,000	3,000	24,000	—
Total						**1,800** independent datasets (1,200 in H)	**9,000** cases (6,000 in H)	**6,000** benchmark cases per method (2 p values × 6 scenarios × 5 scales × 100)	**48,000 primary fits**	3,000 + 3,000

All 1,800 datasets are still generated and sealed (§II.9–§II.13). The benchmark panel uses the 1,200 Regime-H datasets; the SMC arm uses all 1,800 (Regime L for $\hat{\mathbf{Z}}$ and PIP accuracy, Regime H for selection). Whether Regime L needs all five scales and 100 replicates for the accuracy assessment is open (O12); the design is kept unchanged until then.

Pairing rules.

- For each (p, s, r) one X and one ε are drawn. The five scales share them: $y_c = c \cdot (X\beta) + \varepsilon$.
- Every method receives the identical (X, y_c) and the identical 10 CV folds. The folds are the same across the five scales. (At p = 30 no method uses folds after D1; see §II.7.)
- Scales are paired, not independent replicates. Aggregates must not treat them as independent.

The choice of five scales and 100 replicates is an open decision (O2, O3, Appendix A, which also records the reason for the weaker scales).

II.7 Seeds

seed = base + 10000 · s + r, for s = 1..6 and r = 1..100.

p	Data base	Development/pilot base (excluded from counts)	CV-fold base
30	1,310,000 (new)	2,310,000	3,310,000
500	1,410,000 (existing Oct-14)	2,410,000	3,410,000
1000	1,510,000 (new)	2,510,000	3,510,000

The new ranges (1,320,001–1,370,100 for p = 30 and 1,520,001–1,570,100 for p = 1000; the earlier scenario draft misquoted the latter as starting at 1,510,001) were checked: no earlier campaign in the project uses them as seeds. The fold seed for each case is fold base + 10000 · s + r, shared by every CV-tuned method and identical across scales. The pilot/dev data seeds use the development bases above and are excluded from all counts.

p = 500 keeps the Oct-14 seeds. In the generator, β does not consume random numbers. The same seeds therefore regenerate exactly the Oct-14 X and noise, and only y changes. The new y comes from the 14-signal truth, so earlier p = 500 *results* cannot be reused. Only the X and noise draws are paired with them.

Proposed new bases (open decision O11; collision check pending):

- Dev-data fold seeds: 4,310,000 / 4,410,000 / 4,510,000. The p = 500 base equals Oct-14's `pilot_cv_seed_base`.
- Method seed: data seed + 4,000,000. That gives 5,320,001–5,370,100, 5,420,001–5,470,100 and 5,520,001–5,570,100. It is passed as `seed` to every wrapper and recorded as `method_seed`. It drives the A3 MCMC and the A3 2-means starts, and the credible-set purity subsampling of A1 (which does not affect A1's PIPs); A2 and the deterministic frequentist fits ignore it (F5 passes the fixed internal `abess` seed 1).
- The method seed is the same for all five scales of a dataset. For A3 this gives exact common random numbers: bayesreg's draws per iteration consume the uniform stream independently of y when every sampling parameter is finite and positive (derivation from the R trunk RNG source in §III.3.8 e, step 3; identity of the R 4.5.2 routines assumed), and the runner checks empirically that `.Random.seed` after the MCMC is identical across the five scales.

After D1 (no benchmarks at p = 30): the p = 30 CV-fold seeds (base 3,310,000), the p = 30 dev fold base (4,310,000) and the p = 30 method-seed range (5,320,001–5,370,100) serve no method. They are kept in the tables so that the generator and the seed table stay uniform across p (the p = 30 folds are cheap to store); whether to stop generating them is part of O12.

II.8 Conventions for every method

Fixed, not open:

- n = 200 and σ = 1.
- Stored X is centred and scaled (sd, n−1); stored y is not centred. The benchmark intercept conventions are unchanged. The SMC-BVS v3 arm applies its own centring and sample-SD scaling to both X and y and fits no intercept (§I.1–§I.2; M §§4–5).
- PIP threshold 0.5.
- SuSiE uses `intercept=FALSE, standardize=FALSE` (project decision of 2026-10-07).
- No method receives the true support, the true size (5 or 14), Σ_s, or the population blocks (block labels). Any cap on the model size or number of effects must allow well over 14.
- **Regime H (p = 500, 1000): selection outcomes** for every benchmark method and for the SMC arm (D1).
  - Primary outcome: exact support recovery, out of 100 per scenario and scale.
  - Secondary outcomes:
    - false positives (FP) and false negatives (FN);

  - error type: extra-only, missed-only or mixed;
  - selection frequency of each active variable;
  - runtime.

- **Regime L (p = 30): accuracy outcomes for the SMC arm only** (D1), against the **exact posterior** from enumeration of all 2^{30} models:
  - log Z error (log $\hat{Z}$ against the exact log Z, on the same standardised working-model scale; §I.6);
  - PIP error (primary equal-island Rao–Blackwellised PIPs against the exact PIPs; DP2 D2).
  - Selection outcomes (support recovery) are not computed in Regime L. HPM and median-model agreement with the exact posterior, listed as Regime-L comparisons before D1, are not Regime-L outcomes under D1. The earlier wording "for SMC-BVS and any same-posterior sampler" no longer applies: no same-posterior sampler is in the study (§III.4.1).
  - How the errors are summarised (for example absolute log Z error, maximum or mean absolute PIP error) remains a study-integration item; the v3 fit outputs do not choose that summary.
- Failed or unfinished fits stay in the denominator and are reported separately from wrong selections (denominator rules: §IV.1.3).

II.9 Generator specification

The generator extends `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14/scripts/design.R` and keeps its draw order and RNG exactly:

```
generate_dataset(p, scenario, replicate, kind = c("prod","dev"))
  stopifnot(p %in% c(30L,500L,1000L), scenario %in% 1:6, replicate %in% 1:100)
  base <- list(prod=c(`30`=1310000L,`500`=1410000L,`1000`=1510000L),
               dev =c(`30`=2310000L,`500`=2410000L,`1000`=2510000L))[[kind]][[as.character(p)]]
  seed <- base + 10000L*scenario + replicate
  RNGkind("Mersenne-Twister","Inversion","Rejection"); set.seed(seed)
  Sigma <- scenario_covariance(scenario, p)          # unchanged from design.R
  raw_X <- matrix(rnorm(n*p), nrow=n) %*% chol(Sigma) # X first
  X <- scale(raw_X, center=TRUE, scale=TRUE)          # sd with n-1
  eps <- rnorm(n, sd=1)                               # then noise
  tr <- truth(p)                                       # no RNG use
  beta <- numeric(p); beta[tr$active] <- tr$beta
  signal <- drop(X %*% beta)
  Y <- sapply(c(c100=1,c075=.75,c050=.50,c035=.35,c025=.25), function(c) c*signal + eps)
```

The function `truth(p)` is a line-by-line R port of `scenario_check.py::truth`:

- S0 = {30: 5, 500: 14, 1000: 14}.
- STEP = {30: 1, 500: 7, 1000: 13}.
- POS = (3, 2, 4, 1, 5, 3), cycled.
- PATTERN = (1.4, −1.0, 1.8, −1.2, 0.8, −1.6), cycled.
- blocks = 1 + STEP · k, and active = 5 · (block − 1) + POS[k mod 6].

Required output, matching `scenario_check.out`:

- p = 30: active = 3, 7, 14, 16, 25.

- p = 500: active = 3, 37, 74, …, 457.
- p = 1000: active = 3, 67, 134, …, 847.

Fold assignment follows the Oct-14 `run_case.R` rule exactly:

```
fold_seed <- c(`30`=3310000L,`500`=3410000L,`1000`=3510000L)[[p]] + 10000L*s + r
RNGkind("Mersenne-Twister","Inversion","Rejection"); set.seed(fold_seed)
foldid <- sample(rep(1:10, length.out=200))
```

For p = 500 this rule reproduces the Oct-14 folds.

II.10 Files and separation of truth

There are three sealed directories per p. **Fitting jobs open only `inputs/`.**

Directory	Content per dataset `p{pppp}_S{s}_r{rrr}`	Read by
`inputs/p{pppp}/`	`X` (n×p, with `scaled:*` attributes stripped), `Y` (n×5, columns c100…c025), `foldid`, `dataset_id`, `data_seed`, `fold_seed`	fitting tasks, SMC arm
`truth/p{pppp}/`	`active`, `beta`, `signal` = Xβ, `eps`, scenario id	generator verification and scoring only
`manifests/`	`inputs_manifest.csv` and `truth_manifest.csv`: file SHA-256 plus float64 little-endian SHA-256 of X, each Y column, `signal`, `eps` and `foldid` (the `weak_response_sha256` convention of Oct-14)	everyone

- After sealing, files are `chmod a-w`.
- During fitting waves, `truth/` is set to mode 000. Scoring restores it. If a fitting job tries to read truth, it fails loudly.
- Dev (pilot) datasets go to a separate `dev/` tree with the same layout.

II.11 Size on disk

- X is $200 \cdot p \cdot 8$ bytes: 1.6 MB at p = 1000, 0.8 MB at p = 500, 48 KB at p = 30.
- The 1,800 datasets plus dev data come to about 1.5 GB uncompressed. Use `saveRDS(compress=FALSE)` so reads are fast and file hashes are stable.

II.12 Storage location on NOTS

The recommended root is `STUDY=/projects/ml75/fb31/BVS/bench20261019`.

- This is the persistent group filesystem used by the Oct-7 campaign (`/projects/ml75/fb31/BVS/{archives,runs,validation}` exist).
- `df` shows 7.1 TB free there. The group quota was not visible (**UNVERIFIED**).

Alternatives considered:

- `/home/fb31/projects/BVS` has only about 1.3 GB of headroom (8,890 MB used of a 10,240 MB quota). It must hold only small code and links.

- `/scratch/fb31` has 205 TB free but a **14-day purge** (Oct-14 README). It is acceptable as working space only if every output is archived before the purge. It must never hold the only copy.

Layout under `$STUDY`:

- `inputs/`, `truth/`, `manifests/`, `dev/`
- `runs/<YYYYMMDDTHHMMZ>_bench20261019_<piece>/{code/,manifest.json}`
- `results/<run_id>/p{pppp}/<case_id>/<method>.{rds,csv,sha256,done}`
- `status/`, `logs/`

Final local archive: `C:/Users/felip/Documents/NOTS_Archives/<date>_bench20261019/`, following the Oct-14 convention. The project folder receives aggregated tables only.

Oct-14 p = 500 inputs are on scratch. `…/Experiments_October_14_WeakSignals/paired_inputs` → `/scratch/fb31/BVS_Paired_October_14/production` (602 entries, sealed manifest). The bit-identity check in §II.13 needs them, so it must run before they are purged (they were created on 2026-10-14 and 2026-10-15).

II.13 Verification checks (gate G2)

1. **Truth placement.** For each p, `truth(p)` must reproduce `scenario_check.out` exactly: indices, blocks, within-block positions, S6 ρ at each signal, and β. The smallest eigenvalue of each Σ_s must match to 1e-4 (S6: 0.1000; S4: 0.1500).
2. **Standardisation.** For every dataset, max|colMeans(X)| < 1e-10 and max|colSums(X^2)/(n−1) − 1| < 1e-10. This is the Oct-14 `design_tolerance`.
3. **Response construction.** max|Y[, c] − (c · signal + ε)| ≤ 1e-12 · max(1, max|Y|) for every column.
4. **Bit-identical regeneration.**
   - (a) Regenerate 3 datasets per p (S1 r1, S4 r50, S6 r100) on a different node and compare by `identical()` and float64 hash.
   - (b) **p = 500:** for all 600 datasets, check `identical(X_new, X_oct14)`, and check that the ε hash equals the sealed Oct-14 `noise_sha256`. β consumes no random numbers, so only y changes (§II.7).
   - If (b) fails, the cause is almost certainly a BLAS or `chol` difference. Stop and report; do not silently regenerate.
5. **Manifest closure.** 1,800 production and 90 dev dataset rows. All hashes recompute. Seeds are unique and equal base + 10000 · s + r.
6. **Sanity of Σ (descriptive only).** Average the within-block sample correlation over the 100 replicates of each scenario. It should be within ±0.02 of the population value (0.7 in S3, 0.85 in S4, 0.7 within and 0.3 between in S5, ρ_b in S6).

Part III. Benchmarks

The benchmark panel is eight methods, fixed by Felipe. Section §III.1 lists it, §III.2 gives the evidence for each choice, §III.3 gives the exact run specification (the part to implement), and §III.4 lists what was considered and not selected, and what decision D1 dropped (§III.4.5). The panel runs in Regime H only (D1). SMC-BVS itself is not in the panel (Part I, §I.7 and §IV.1.4 give the pairing interface).

III.1 The final panel

#	Method	Regime	Software (version)	Primary selection rule (no oracle input)
F1	Lasso	H only	`glmnet` 5.1	CV λ_min on the common folds. λ_1se, EBIC(γ=1) and EBIC(γ=0.5) are reported from the same fit.
F2	Elastic net	H only	`glmnet` 5.1	Joint CV over α ∈ {.1, .25, .5, .75, .9} and λ
F3	Adaptive lasso	H only	`glmnet` 5.1 (ridge pilot in base R)	Weights from a CV ridge fit, then CV λ_min
F4	MCP (SCAD only as a labelled sensitivity)	H only	`ncvreg` 3.16.0	EBIC(γ=1)
F5	Best subset (ABESS)	H only	`abess` 0.4.11	ABESS + external EBIC(γ=1). Exact best subset is infeasible at p = 1000.
A1	SuSiE	H only	`susieR` 0.14.2	PIP > 0.5, with L = 50
A2	Spike-and-slab lasso (SSL)	H only	`SSLASSO` 1.2.3.9001 (patched Oct-7 build of 1.2.3)	Adaptive θ ~ Beta(1, p), unknown variance; the support at the end of the λ0 ladder
A3	Horseshoe	H only	`bayesreg` 1.3	A variable is selected when its equal-tailed 95% credible interval excludes 0

Every method runs in Regime H only (p = 500 and 1000), by D1; there are no benchmark fits at p = 30. The Regime-L settings of the earlier plan (F5 exact `leaps` + BIC, F4 BIC, SuSiE L = 10, the EBIC cap of 30 and the related sensitivities) are recorded in §III.4.5. Nothing else is in the panel. SMC-BVS v3 is the separate arm specified in Part I; its B = min(⌊√n⌋, ⌊√p⌋, 16) rule gives B = 14 at both Regime-H sizes, and only its primary RB PIP is reported (M §3; DP2 D2). Sensitivity tiers (S0 free, S1 budgeted, S2 optional) and every extra rule are defined in §III.3.0 and under each method in §III.3.

III.2 Why each method was chosen

Evidence base

Full evidence lives in two files, each written by an independent Opus agent from primary sources:

- `evidence/FREQUENTIST.md`: 51 citations verified (22 method references, 13 usage papers and 16 software checks), 2 unverified (the journal venue of the T-Rex paper; the uniLasso failure claim).
- `evidence/BAYESIAN.md`: 28 of 31 references verified, and 16 of 17 software pages.

The quotes and notes are in `evidence/notes_frequentist/` and `evidence/notes_bayesian/`.

Lead checks. The frequentist quote check was re-run against the saved full texts: 23 of 23 match. The following CRAN and PyPI facts were re-fetched:

- `hdi` archived 2026-07-10;
- `stabs` 0.7-1;
- `knockpy` 1.3.5;
- `horseshoe` archived 2025-10-10;
- `mombf` archived 2026-09-30;
- `modelSelection` 1.0.7;
- `susieR` 0.14.2.

The BB(1,p) prior tail was also recomputed by the lead (the numbers are in §IV.5).

What changes at p > n (the reason some frequentist methods cannot be in a p > n panel; after D1 the panel runs at p > n only):

- Fixed-X knockoffs and BH on full-model OLS p-values need n > p. The R `knockoff::create.fixed` stops with "Input X must have dimensions n > p". They could only run in Regime L, and are not in the final panel (§III.4).
- Exact best subset is infeasible at p = 500 to 1000. COMBSS dropped MIO at p = 1000 "due to time computational constraints". ABESS fills the best-subset slot.

What changed since the p = 30 list (Bayesian side). `horseshoe` was removed from CRAN on 2025-10-10, and `mombf` on 2026-09-30; their maintained replacements are `bayesreg` and `modelSelection`. The SMC now selects by HPM (highest-posterior model) under a BB(1,p) model prior, not by MPM (median-probability model) under Bernoulli(0.2). SuSiE, SSL and the horseshoe remain the most-run Bayesian comparators at n = 200, p = 1000.

Rules common to every method are in §II.8 (no oracle input, same data, no intercept where the software allows). Where a published comparison used an oracle setting, it is not copied: Kollipara et al. gave SuSiE the true L, and gave SSVS and the regularized horseshoe the true expected model size. Every setting is fixed before any run, and every deviation is logged.

F1 Lasso

- **References.** Tibshirani 1996, JRSS-B 58(1) 267-288, doi:10.1111/j.2517-6161.1996.tb02080.x. Algorithm: Friedman, Hastie & Tibshirani 2010, JSS 33(1), doi:10.18637/jss.v033.i01.
- **Software.** `glmnet` 5.1, CRAN 2026-09-24.

- **Use at p > n.** Hastie, Tibshirani & Tibshirani 2020, Stat. Sci. 35(4), doi:10.1214/19-STS733, §3.1: "high-5: n = 50, p = 1000, s = 5". Zhu et al. 2020 PNAS (ABESS), "High-Dimensional Case": lasso via glmnet with SIC and CV, n = 500, p up to 2,500.
- **Why.** The baseline convex $\ell 1$ method, and already in the Oct-14 p = 500 panel (Lasso CV-min/1se; Lasso/MCP/ABESS with EBIC, gamma 0.5 and 1), which gives continuity.
- **Caveat on the evidence.** CV-min is prediction-tuned and over-selects (Hazimeh & Mazumder 2020 §5.3.1: "tuning parameter selection based on prediction error makes the Lasso select models with many nonzero coefficients"). Irrepresentability is likely violated in S3/S4. Hence all three arms (λ_min, λ_1se, EBIC) are reported.

F2 Elastic net

- **Reference.** Zou & Hastie 2005, JRSS-B 67(2) 301-320, doi:10.1111/j.1467-9868.2005.00503.x.
- **Software.** `glmnet` 5.1 (2026-09-24).
- **Use at p > n.** Hazimeh & Mazumder 2020, Oper. Res. 68(5), doi:10.1287/opre.2019.1919, §5.1/§5.3: "Elastic Net ... We use the implementation of glmnet", p = 1000 and n swept from 100 to 1000. Kim, Wang, Carbonetto & Stephens 2024, JMLR 25(185), §5.1 Table 1 and §5.2: "n = 500 and p = 1,000".
- **Why.** The correlation-aware convex method: the grouping effect pulls in same-block proxies in S3–S6. It is the slowest convex slot (Kim et al. §5.4.4: EN "was considerably slower ... because we tuned both of the Elastic Net parameters by CV").

F3 Adaptive lasso

- **Reference.** Zou 2006, JASA 101(476) 1418-1429, doi:10.1198/016214506000000735.
- **Software.** `glmnet` 5.1 (`penalty.factor`).
- **Use at p > n.** Freijeiro-González, Febrero-Bande & González-Manteiga 2022, Int. Stat. Rev. 90(1), doi:10.1111/insr.12469, §4.1: a neutral comparison with p = 100 and n = 25, 50 (p > n), "Adaptive LASSO (AdapL): glmnet"; AdapL was the only method that "selects the complete set S without including any noise ... for a large enough value of n". Nouraie, Zhu & Muller 2025 (arXiv:2511.02306, preprint), §3, n = 100, p = 1000.
- **Why.** The oracle-property weighted penalty. Ridge, not OLS, supplies the initial estimate, so the same rule works when p > n.

F4 MCP (SCAD as a sensitivity)

- **References.** MCP: Zhang 2010, Ann. Statist. 38(2) 894-942, doi:10.1214/09-AOS729. SCAD: Fan & Li 2001, JASA 96(456) 1348-1360, doi:10.1198/016214501753382273. Algorithm: Breheny & Huang 2011, Ann. Appl. Stat. 5(1) 232-253, doi:10.1214/10-AOAS388.
- **Software.** `ncvreg` 3.16.0, CRAN 2025-11-06.
- **Use at p > n.** Zhu et al. 2020 PNAS, "High-Dimensional Case": "LASSO (4), SCAD (6), and MCP (7) ... implemented in R packages glmnet and ncvreg", with SIC and also CV, n = 500, p ∈ {500, 1500, 2500}. Huang, Muller & Tarr (arXiv:2505.22012v2, 2026), §4: SCAD via ncvreg at 10-fold-CV λ, with (n, p) = (20, 1000) and (100, 500). Kim et al. 2024 JMLR, Table 1.

- **Why.** The folded-concave slot. EBIC(γ=1) gives continuity with the Oct-14 p = 500 panel. (The earlier Regime-L rule, BIC following the project methodology, was dropped by D1; §III.4.5.)
- **Caveat on the evidence.** Zhu et al. report that "the MCP and SCAD underestimate the model size" under SIC.

F5 Best subset (ABESS)

- **References.** ABESS: Zhu, Wen, Zhu, Zhang & Wang 2020, PNAS 117(52) 33117-33123, doi:10.1073/pnas.2014241117. EBIC: Chen & Chen 2008, Biometrika 95(3) 759-771, doi:10.1093/biomet/asn034.
- **Software.** `abess` 0.4.11 (CRAN 2025-12-08).
- **Use at p > n.** Hazimeh, Mazumder & Nonet 2023, JMLR 24 (L0Learn), §4 Table 1: "we compare with glmnet, ncvreg, picasso, and abess", n = 1000, p $\in$ {10^3, 10^4, 10^5}.
- **Why.** The $\ell 0$ paradigm. Exact search is infeasible at p $\geq$ 500 (COMBSS, Moka et al. 2024, Stat. Comput. 34(2) §7.1, doi:10.1007/s11222-024-10387-8, dropped MIO at n = 100, p = 1000), so ABESS fills the slot.
- **Caveat on the evidence.** ABESS has no global certificate in finite samples.
- The exact branch-and-bound arm in Regime L (`leaps` + BIC; Furnival & Wilson 1974; Zhu et al. 2020 "Low-Dimensional Case") was dropped by D1. Its references, evidence and run specification are kept in §III.4.5.

A1 SuSiE

- **Reference.** Wang, Sarkar, Carbonetto & Stephens 2020, JRSSB 82(5):1273–1300, doi:10.1111/rssb.12388.
- **Software.** `susieR` 0.14.2, CRAN 2025-06-05.
- **Use at p > n.** Kollipara et al. 2026, Stat Med, n = 200, p = 1000: "SuSiE was implemented using susie from susieR" (§4.2). Kim et al. 2024 JMLR, n = 500, p = 1000, Table 1: "SuSiE susieR variational inference" (§5.1).
- **Why.** Variational sum-of-effects. Wang et al. report robustness to overstating L (§3.3). Feasibility: 0.4–5 s per fit at n = 200, p = 1000 with small L (Kollipara et al.).
- **Caveat on the evidence.** Kollipara set L to the true size (oracle), which is not copied. (The earlier reason for L = 50, that L = 10 "would make recovery impossible", is not correct for a PIP rule; L = 50 is kept as a capacity fixed from n, §III.3.6 d.) Kollipara also reports that SuSiE "failed completely" in some correlated and heavy-tail cells (§4.4). With L = 50 the cost per IBSS sweep grows linearly in L (about $12 \cdot L \cdot n \cdot p$ flops plus L one-dimensional optimisations per sweep; §III.3.6 j); the number of sweeps is unmeasured.

A2 Spike-and-slab lasso

- **Reference.** Ročková & George 2018, JASA 113(521):431–444, doi:10.1080/01621459.2016.1260469.
- **Software.** `SSLASSO` 1.2.3, CRAN 2025-11-04, source defaults read at github.com/cran/SSLASSO. The study uses the patched build 1.2.3.9001 (reason in §III.3.7).

- **Use at p > n.** Kollipara 2026, n = 200, p = 1000: "Spike-and-slab LASSO (SL) ... implemented using SSLASSO" (§4.2). Ray & Szabó 2022, JASA 117(539):1270–1281, doi:10.1080/01621459.2020.1847121, (n, p) = (100, 400), (100, 1000), (200, 800): SSLASSO with "a = 1, b = p, and penalty="adaptive"" (§5.2, Table 2). Das 2026 (arXiv:2604.04964), (p, n) = (1000, 200) (§5, Table 1).
- **Why.** Its Beta(1, p) prior on θ mirrors the SMC's BB(1,p) prior, so it is the nearest penalised-mode comparator. The slab differs (Laplace, not g-type) and the output is a point estimate, so it has no PIPs and no evidence. Feasibility: 1–10 s per fit.

A3 Horseshoe

- **References.** Carvalho, Polson & Scott 2010, Biometrika 97(2):465–480, doi:10.1093/biomet/asq017. Implementation: Makalic & Schmidt 2016, arXiv:1611.06649 (id as cited on the CRAN page; the arXiv page itself was not opened).
- **Software.** `bayesreg` 1.3, CRAN 2024-09-30. The `horseshoe` package (last 0.2.0, 2019-07-18) was archived 2025-10-10.
- **Use at p > n.** Porwal & Rodríguez 2024 (arXiv:2411.00471v3), n = 500, p = 750: "Computation under the Horseshoe prior relies on version 1.2 of the R package bayesreg" (§6.2). Kollipara 2026, n = 200, p = 1000 (horseshoe package, §4.2). Das 2026 (bayesreg MATLAB, §5). The selection rule is Porwal & Raftery 2022, PNAS 119(16), doi:10.1073/pnas.2120737119: "we calculated a 95% credible interval and checked whether 0 was included". The 5000 + 1000 iteration budget follows Das 2026 and Kollipara 2026.
- **Why.** Global-local continuous shrinkage by Gibbs MCMC.
- **Caveat on the evidence.** Runtime risk at p = 1000: Kollipara reports HS 20–300 s with the `horseshoe` package but HS+ 250–17,000 s with Bayesreg. The manual does not state which sampler bayesreg uses when p > n; the source does: Bhattacharya et al. when $p/n \geq 2$, so at both p = 500 and p = 1000 at n = 200 (§III.3.8 a) [V-src].

Why A1–A3

A1–A3 are the three Bayesian methods run at exactly n = 200, p = 1000 by both Kollipara 2026 and Das 2026. They span three of the four Bayesian paradigms that recent p > n papers actually run: variational sum-of-effects (A1), spike-and-slab penalised mode (A2) and continuous shrinkage (A3). The fourth, model-space BMA (BAS and nonlocal priors), is not in the panel (§III.4). None of A1–A3 gives an evidence value comparable to the SMC's Z, because each has a different model. A1 gives PIPs.

III.3 Run specification for each panel method

This is the specification to implement. It applies the settings of §III.1 and the conventions of §II.8. Tags ([V-doc], [V-NOTS], [V-run], UNVERIFIED, EST) are explained in the reading guide.

III.3.0 Common rules for all eight methods

Fitting interface. The interface follows the Oct-7 contract (`02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks/CONTRACT.md`), extended:

```
fit_<method>(X, y, foldid, seed, settings)
# returns list(rules = data.frame(rule_id, role, support, k, extra...),
#              pip = <p-vector or NULL>, beta = <p-vector>,
#              converged = TRUE/FALSE, or NA where the method has no convergence
#                          criterion (A3 MCMC),
#              diagnostics = list(...), warnings = chr)
```

- The function body is sourced into an environment whose parent is `asNamespace("stats")`, as in Oct-14 `run_case.R`. This hides the runner's local variables, but the parent chain still reaches the global environment (checked on NOTS, R 4.5.2: stats → imports:stats → base → R_GlobalEnv). The guarantee that truth is unreachable therefore comes from the runner never loading truth (fitting jobs open only `inputs/`, and `truth/` is mode 000 during fitting waves, §II.10), not from the environment.
- The caller records the elapsed time of the whole fit, including tuning.
- **Failures.** A failed fit is never replaced by the null model. A failed rule has `status = "failed"`, `support = NA` and `k = NA`. If the fit returned a support (a fit-level gate failed, such as non-convergence or a consistency check, or a rule's selected point failed its check), that support is kept for audit in the column `support_audit`; the scoring step sets `error_type = failed` and counts the row as non-recovery. When the fit errors, `support_audit` is NA too. A rule-level failure (for example A3's 2-means step) fails only that rule. Failure takes precedence over a warning flag on the same row.
- **Status.** `status` reflects only R conditions and failure: `ok`; `warning` (at least one warning or message was captured during the fit; all are stored verbatim in `warnings` and `messages`); `failed`. Wrappers capture both warnings and messages (`common.R::with_captured_conditions`), because susieR reports some conditions through `message()` (§III.3.6 e). A captured condition never changes the selection and never sets `diag_flag`. `timeout` and `not_run` are written only by the reconciliation step (below).
- **Flags.** `diag_flag` (logical) is method-wide: it is TRUE on every rule row of a fit when any predeclared numerical-quality check of that fit failed anywhere. It defines the passing-only counts of §IV.1.3 and never changes `status` or a selection. The flag-setting checks are, and are only: F1–F3 `kkt_path`; F4 `kkt_path`, `path_truncated`; F5 none; A1 `L_saturated`, `elbo_drop`, `sigma2_floor`; A2 `tail_unstable`, `fixed_point` (`capped_intermediate` is informational since the fourth revision); A3 `ess_tau`, `ess_ci95`, `crn_mismatch` (the last set by the scoring step, §III.3.8 e). The codes of the checks that fired are written to `diag_codes`.
- **Codes.** `diag_codes` is a `;`-separated list drawn from a closed vocabulary: the flag codes above plus the informational codes `lambda_first`, `lambda_last`, `alpha_edge`, `kappa_edge`, `floored_selected`, `ebic_at_largest`, `capped_intermediate` (A2). Informational codes never change a selection, a flag or a count.
- **Convergence.** `converged` records the solver's own criterion only: F1–F3 TRUE when `jerr == 0` in every glmnet fit (so always TRUE when not failed); F4 TRUE when the path is not truncated and the last returned point used fewer than `max.iter` iterations; F5 NA (abess returns no iteration count, §III.3.5 h); A1 `fit$converged`; A2 TRUE when the final ladder point is not capped; A3 NA (MCMC). KKT, ELBO, ESS and audit checks go to `diag_flag`, never to `converged`. A failed fit has `converged = FALSE` unless the method's value is NA.

- **Runtime.** `runtime_s` is the elapsed time of the whole wrapper call, including tuning and the method's own diagnostics (KKT products, ESS); T5 reports it as such. A3 additionally records the diagnostics time `diag_s`.

Method registry and rule frame. `config.R` holds a table `METHODS` with one row per method id (F1–F5, A1–A3 and the S1 ids `F4_SCAD`, `SuSiE-L100`, `SSL-RS`): `method_id`, `fn` (wrapper function name), `variant` (`settings$variant` passed to it: `"primary"`, `"scad"`, `"L100"`, `"RS"`), `method_label`, `package`, `pkg_version`, `rule_ids` and `roles` (lists, in the order of each method's item f), `tier` (primary or S1). The runner loops over `METHODS` rows; `make_rule_rows()` and `fail_rows()` read the registry, so a failed fit always produces exactly the registered rule rows. Each wrapper returns `rules` as a data frame with these columns: `rule_id`, `role`, `status`, `support`, `support_audit`, `k`, `converged`, `diag_flag`, `diag_codes`, plus the method's extra columns of item g (NA where not applicable); the runner adds the provenance, design, seed, hash, method and compute columns of §IV.1.2.

One capture layer. The wrapper captures the package call itself (`with_captured_conditions`) and turns its own gate failures into registered failed rows. The runner wraps each wrapper call in an outer `tryCatch` whose only job is to turn an uncaught error (for example a version assertion or a `stopifnot` outside the wrapper's capture) into `fail_rows()` output with the error message in `error`. For a failed fit, `beta` is an NA vector of length p and `pip` is NULL.

Files and missing outputs. Each (case, method_id) writes the four files of §IV.1.1 (`<method>.rds`, `.csv`, `.sha256`, `.done`, under `results/<run_id>/p{pppp}/<case_id>/`), one CSV per fit holding that fit's rule rows. There are no in-R time limits (`setTimeLimit` cannot interrupt C code). After each wave, the G8 reconciliation script writes, for every registered (case, method_id) with no `.done`, rule rows with `status = timeout` if the task's SLURM state was TIMEOUT and `not_run` otherwise. S2 methods are not registered and emit no rows.

Settings in code. In §III.3.1–§III.3.8 a bare block name in code (for example `GRID$nlambda`, `GLMNET`, `F4$eps`, `A1$L`) stands for the same field of `settings` (`settings$GRID$nlambda`, …); wrappers read every constant through `settings`, never from the global environment.

- **RNG.** The runner calls `RNGkind("Mersenne-Twister", "Inversion", "Rejection"); set.seed(method_seed)` immediately before every method call, and every wrapper that uses random numbers sets its own seed at entry. No method may rely on RNG state left by another: `abess.default` calls `set.seed(seed)` on entry and `set.seed(NULL)` on exit (abess 0.4.11 `R/abess.R` l.338, l.585) [V-src], which re-seeds the session RNG from the clock after every F5 fit.
- **Library.** The runner asserts that every package is loaded from the sealed study library (`find.package(pkg)` lies under `$STUDY/lib/R452/`), and each wrapper asserts its exact package version (`common.R::assert_pkg_version`).

Configuration and code layout. All constants live in `code/R/config.R`, frozen before the pilot. It defines one named list per block: `COMMON`, `GRID`, `GLMNET`, `F1`–`F5`, `A1`, `A2`, `A2RS` (the S1 ladder), `A3`; a field is written `BLOCK$field` (field names keep the case used in §III.3.1–§III.3.8). Study-wide scalars (`METHOD_SEED_OFFSET` = 4,000,000, `PIP_THRESHOLD` = 0.5, `TEST_SEED` = 20261019) are top-level constants. The `settings` argument of every wrapper is this whole configuration list plus `settings$variant` (for example `"primary"`, `"L100"`, `"RS"`). Code tree under the study workspace:

- `code/R/config.R` (constants), `code/R/common.R` (input assertions, `assert_pkg_version`, OLS refit `ols_rss`, EBIC/BIC scorer, support keys, `make_rule_rows`, `fail_rows`, `with_captured_conditions`, IO, timing);

- `code/R/methods/glmnet_helpers.R` (shared by F1–F3) and one file per method: `fit_F1_lasso.R`, `fit_F2_enet.R`, `fit_F3_alasso.R`, `fit_F4_mcp.R` (MCP and SCAD), `fit_F5_abess.R`, `fit_A1_susie.R`, `fit_A2_ssl.R`, `fit_A3_horseshoe.R`;
- `code/R/run_case.R` (runner), `code/tests/test_common.R` and `code/tests/test_<method>.R` (G3 unit tests), `code/slurm/` (job, submit and monitor scripts).

Input assertions. Every wrapper checks the design as Oct-14 `benchmark_fit` does:

- X is finite and centred, with sd 1 to within 1e-10;
- length(foldid) = n, with labels 1..10.

EBIC/BIC. The formula is exactly the Oct-14 one [V-run]:

$\mathrm{EBIC}_\gamma(S) = n \cdot \log(\mathrm{RSS_S}/n) + |S| \cdot \log n + 2\gamma \cdot \log C(p, |S|)$

- RSS_S comes from a zero-intercept OLS refit on the original y (`lm.fit`, tol 1e-10).
- k = |S| counts slopes only.
- The null model is eligible.
- A rank-deficient support gets EBIC = ∞.
- Ties are broken by criterion, then smaller size, then the support key (sorted indices joined by `;`) in C-locale byte order, implemented as `order(crit, k, key, method = "radix")`, so the order does not depend on the node's locale.
- A rule whose chosen size equals the largest size among its valid candidates gets the code `ebic_at_largest`; each EBIC row also records `k_valid_max` and `k_path_max`, so a cap can be told from a path end. The code is informational and never changes the choice.
- $\gamma = 0$ is BIC.
- The candidate cap is $|S| \leq 100$ (Regime H; Oct-14). (The Regime-L cap of 30 was dropped by D1; §III.4.5.)
- Supports are deduplicated before scoring.

Size caps never use the truth. All caps are for Regime H, the only regime with benchmark fits (D1).

Method	Regime H cap
EBIC candidates	100
ABESS sizes	0..100
SuSiE L	50
CV rules (λ_min, λ_1se), SSL, horseshoe	no cap

Threads. `OMP_NUM_THREADS = OPENBLAS_NUM_THREADS = MKL_NUM_THREADS = 1`. Use `n.cores=1` / `num.threads=1` where the package has such an argument.

Sensitivity tiers.

- **S0 (free):** an extra rule read from the primary fit. It reports the same runtime, and runtimes are never summed across rules.
- **S1 (budgeted):** a separate fit, run in production on every case of the stated regime.
- **S2 (optional):** not budgeted and not run unless approved.

Versions to pin. Install everything into one sealed study library; see §IV.3, gate G1.

Package	Version	Status on NOTS (2026-10-19)
glmnet	5.1	installed in `$HOME/R/library` (built for R 4.5.2)
ncvreg	3.16.0	installed in `$HOME/R/library`; **not** in the Oct-7 `library_portable` (`ncvreg NA`)
leaps	3.2	installed; no longer used by the panel (the exact F5 arm in Regime L was dropped by D1, §III.4.5)
abess	0.4.11	installed
susieR	0.14.2 (tarball SHA-256 `ba02322e…`)	installed in `$HOME/R/library` (built R 4.5.2, 2026-09-29); imports Matrix 1.7.5, matrixStats 1.5.0, mixsqp 0.3.54, reshape 0.8.10, crayon 1.5.3, ggplot2 4.0.2 installed; Rfast optional and not needed
SSLASSO	**1.2.3.9001** (patched build from Oct-7 `02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks/SSL/vendor/SSLASSO_1.2.3.9001.tar.gz`, SHA-256 `f82bf8db…`; upstream 1.2.3 SHA-256 `67fb4a88…`)	present in `/home/fb31/projects/BVS/library_portable` (built R 4.5.2, 2026-10-05; `SSL_gaussian` registered with 15 parameters); not in `$HOME/R/library`
bayesreg	1.3 (SHA-256 `21e19a4b…`; pure R)	**not installed**
pgdraw	1.1 (SHA-256 `05b951b2…`; C++, needs Rcpp $\geq$ 0.12.16; Rcpp 1.1.2 installed)	**not installed** (bayesreg imports it)
doParallel	1.0.17 (SHA-256 `b96a25ad…`; needs foreach, iterators)	**not installed**; foreach 1.5.2 and iterators 1.0.14 installed
coda	0.19.4.1	installed (for ESS)
digest	0.6.37	installed (system library; RNG-state hash of A3)

The runtime is GCC/14.3.0 + R/4.5.2 with OpenBLAS 0.3.30 via FlexiBLAS 3.4.5, the same stack as Oct-14.

III.3.1 F1 Lasso

Scope of §III.3.1–§III.3.5. These five sections are the deployment specification for the frequentist panel. A new thread that reads only this master should be able to write `code/R/methods/fit_F1_lasso.R` … `fit_F5_abess.R`, the shared glmnet engine, their tests, and the frequentist part of `code/R/config.R` and `code/R/common.R` from them. Facts read from the package sources carry the tag **[V-src]**: checked on 2026-10-19 against the CRAN source tarball (SHA-256 below) at the file and line given. The line references and the hashes are collected in `evidence/FREQUENTIST.md`, section "Deployment study 2026-10-19". The other tags are as in the reading guide. The specification was reviewed by GPT-6 Astra; the findings and their resolution are in `deployment_20261019/FREQ/ASTRA_RESOLUTION.md`.

Tarball	SHA-256	Note
glmnet_5.1.tar.gz	a6c23c906806f89a451de30b202e5bf088577f4da260d0b7f8887a7df3f22778	identical to the Oct-7 vendor copy
ncvreg_ 3.16.0.tar.gz	354ec5c7166ae0b166878b4609fafe4542e2be4f8df936671243d5a8be092145	CRAN 2025-11-06
abess_0.4.11.tar. gz	44bf69f7504bbd48630af714e8f400f1138753e307fee1dc14db8fab7fe7a31c	identical to the Oct-7 vendor copy

Conventions shared by F1–F5 (they implement §III.3.0; the helpers named here are listed in item k of each method).

- n = 200, p ∈ {500, 1000}. X is centred and scaled (sd with n − 1). y is the uncentred `Y[, scale_key]` column. The model has no intercept.
- **λ fraction grid** (F1–F4): `frac = exp(seq(0, log(1e-4), length.out = 200))`. A method's grid is `1.01 · entry · frac`, where `entry` is the smallest penalty that gives the null model for that method on the full data. The factor 1.01 makes the first point the null model with a strict inequality.
- **Supports** are the indices with an exactly nonzero coefficient (`SUPPORT_ZERO_TOL = 0`). The support key is the sorted indices joined by `;`; the null support is the empty string.
- **OLS refit** (`ols_rss`): the null support returns RSS = Σy^2 and rank 0; otherwise `z <- lm.fit(X[, S, drop = FALSE], y, tol = 1e-10)`, RSS = `sum(z$residuals^2)`, rank = `z$rank`. `drop = FALSE` is required: without it a one-variable support becomes a vector and `lm.fit` fails (Oct-14 `methods.R` l.4–7).
- **EBIC scorer** (common, §III.3.0): `EBIC_γ(S) = n·log(RSS_S/n) + k·log n + 2γ·log C(p, k)`, k = |S|, RSS_S from `ols_rss` on the original y. Candidates are the null support plus the distinct supports with k ≤ `KMAX_EBIC` = 100 = ⌊n/2⌋. A candidate is valid only if `rank == k`, RSS is finite and RSS > 0; an invalid candidate scores `Inf`. The minimiser is taken with `order(crit, k, key, method = "radix")`: criterion, then smaller size, then the support key in C-locale byte order, so the order does not depend on the node's locale. γ = 1 is `ebic1`, γ = 0.5 is `ebic05`, γ = 0 is `bic`.
- **Largest-candidate code.** An EBIC rule gets the code `ebic_at_largest` when it selects the largest valid candidate size. This describes the choice; it is not evidence about larger supports. Each EBIC row also records `k_valid_max` (the largest valid candidate size) and `k_path_max` (the largest support size anywhere on the path, cap ignored), so that a reader can tell a cap from a path end. At p = 500 under the old 6-signal truth, MCP with EBIC(0.5) chose a median size of 97 and ABESS with EBIC(0.5) had p95 size 100 (Oct-14 `summary/validated_rows.csv`, recomputed for this plan) [V-run].
- **Status, flags, failure.** `status` is `ok`, `warning` or `failed` (§IV.1.2). `warning` means that R signalled at least one warning during the fit; the warnings are captured with `withCallingHandlers` and stored, and they do not change the selection. `diag_flag` is method-wide: it is TRUE on every rule row of a fit when any numerical-quality check of that fit failed anywhere (for example a KKT violation at any grid point of any of its path fits). It defines the "passing-only" counts of §IV.1.3, so a violation that one rule selected cannot disappear from another rule's passing-only eligibility. A rule whose own points fail is, in addition, `failed`. `diag_codes` lists informational codes (boundary, largest candidate, floored weights) separated by `;`. A failed rule has `support`

= NA and k = NA; any support it would have chosen is kept in support_audit (§III.3.0). It is never replaced by the null model.

- **Seeds.** F1–F4 draw no random numbers: the folds are supplied and the solvers are deterministic. Each wrapper still calls RNGkind("Mersenne-Twister", "Inversion", "Rejection"); set.seed(seed) at entry. The runner passes the method seed of §II.7 (data seed + 4,000,000) and writes it to method_seed; F1–F4 use no random numbers after it. F5 passes the fixed constant seed = 1 to abess() (Oct-14).
- **Threads.** One thread (§III.3.0). glmnet and ncvreg have no thread argument; abess gets num.threads = 1.

a. What it estimates and why this configuration. The lasso minimises RSS/(2n) + $\lambda\|\beta\|_1$ on the supplied X, with no intercept and no standardisation. λ is tuned by 10-fold prediction CV on the common folds (primary: λ_min). λ_1se and EBIC on the path are free sensitivities, because CV-min is known to over-select (§III.2). Note for the comparison: the primary rules of F1–F3 are prediction-tuned (CV-min) while those of F4 and F5 are EBIC(1), so T1 confounds the penalty with the selection criterion; T1 therefore carries the ebic1 rule of F1–F3 as a pre-declared companion column (§IV.1.3). The objective, grid and rules are the Oct-14 p = 500 ones [V-run]. The CV is now computed by the shared glmnet engine (e) instead of cv.glmnet, so that every fold fit can be checked; cv.glmnet becomes a cross-check (i).

b. Software.

- Package glmnet 5.1 (CRAN 2026-09-24). On NOTS it is installed in /home/fb31/R/library (R 4.5.2, built 2026-09-29) [V-NOTS]. Production uses the sealed study library $STUDY/lib/R452/ built at gate G1 from the tarball above.
- Version assertion in the wrapper: assert_pkg_version("glmnet", "5.1"), i.e. stopifnot(identical(as.character(utils::packageVersion("glmnet")), "5.1")). The runner separately asserts that find.package("glmnet") lies under the sealed library (§III.3.0).
- Function used: glmnet::glmnet (11 calls per case). The formals on NOTS match the source [V-NOTS 2026-10-19: formals(glmnet::glmnet)].

c. Inputs and preprocessing passed to the solver.

- x = X or a training-fold subset X[train, , drop = FALSE]; y the matching uncentred response; intercept = FALSE, standardize = FALSE.
- With standardize = FALSE glmnet sets xs = 1 and does not rescale X [V-src standardize.hpp l.62]. With intercept = FALSE it does not centre X or y (ym = 0), and the returned intercept is 0 [V-src standardize.hpp l.40–47; gaussian.hpp l.453–459].
- glmnet always rescales y internally: with weights 1/n it sets ys = sqrt(mean(y²)), divides y and the user λ by ys, and multiplies λ and the coefficients back by ys [V-src standardize.hpp l.35–47; gaussian.hpp l.437, l.448–451]. On the original scale the native solver therefore minimises RSS/(2n) + $\lambda \cdot a\|\beta\|_1 + \lambda(1-a)\|\beta\|^2/(2 \cdot ys)$, where a is glmnet's alpha (derivation in §III.3.2 c). At a = 1 the $\ell 2$ term vanishes and the objective is exactly RSS/(2n) + $\lambda\|\beta\|_1$. F1 needs no compensation; the shared engine's map (§III.3.2 c) reduces to the identity (m = 1).
- Penalty factors are the default (all 1). glmnet rescales factors to sum to p, which leaves them unchanged [V-src elnet_driver/base.hpp l.10–14].
- Algorithm controls are passed per call through control = list(...), never as top-level arguments and never through a global glmnet.control() call. In 5.1 the top-level thresh, maxit,

`dfmax` and `pmax` are deprecated, and passing them emits a `.Deprecated` warning once per R session [V-src `glmnet.R` l.164–170, l.201–203, l.407–417; `glmnet.control.R` l.238–250]. In a task with many fits only the first would then carry status `warning`. Per-call overrides are restored on exit [V-src `glmnet.control.R` l.256–259; `glmnet.R` l.419].

d. Parameter table.

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
`family`	`"gaussian"`	model	fixed	—	(literal)
`alpha`	1	lasso	fixed	—	`F1$alpha`
`intercept`, `standardize`	`FALSE`, `FALSE`	project (§II.8)	fixed	—	`COMMON$intercept`, `COMMON$standardize`
`type.gaussian`	`"naive"`	package default for p ≥ 500 [V-NOTS formals]; set explicitly	fixed	—	`GLMNET$type_gaussian`
λ grid length	200	Oct-14	fixed	—	`GRID$nlambda`
λ grid ratio	1e-4	Oct-14	fixed (pilot action, i)	—	`GRID$min_ratio`
λ grid top multiplier	1.01	Oct-14	fixed	—	`GRID$max_mult`
λ entry value	zmax = max_j abs(X_j'y)/n (full data)	lasso KKT at β = 0	data	via X, y	(computed)
`control$thresh`	1e-16	Oct-7 EN and AL; project decision for F1 (Oct-14 used 1e-9)	fixed (pilot action)	—	`GLMNET$thresh`
`control$maxit`	**1e7** passes, total over the path (1e6 until the 2026-10-19 dev runs, which reached it)	Oct-14; pilot action §IV.2.2	fixed	—	`GLMNET$maxit`
`control$fdev`, `control$devmax`	0, 1	Oct-14	fixed (inert, e)	—	`GLMNET$fdev`, `GLMNET$devmax`
`control$dfmax`, `control$pmax`	package defaults p + 1 and p	[V-src `glmnet.control.R` l.270–273]	fixed	p	(not set)
Folds	common folds, 10 × 20	§II.9	input	n	`COMMON$nfolds` = 10
CV loss	squared error; fold-size-weighted mean; grouped SE	`cv.glmnet` definitions [V-src]	fixed	—	(engine)
Returned-λ tolerance	1e-12 relative	Oct-7	fixed	—	`GLMNET$lambda_rel_tol`
KKT tolerance	1e-5, relative to max(1, max_j abs(X_j'y) / n_fit)	Oct-7 EN gate	fixed	—	`GLMNET$kkt_rel_tol`
EBIC cap	100 = ⌊n/2⌋	Oct-14; equals the SMC k_max (§I)	fixed	n	`COMMON$kmax_ebic`
EBIC γ	`c(ebic1 = 1, ebic05 = 0.5, bic = 0)` (F1–F3 use the first two; F4, F5 all three)	Oct-14	fixed	—	`COMMON$ebic_gamma`
OLS rank tolerance	1e-10	Oct-14	fixed	—	`COMMON$ols_tol`
Design tolerance	1e-10	Oct-14	fixed	—	`COMMON$design_tol`

e. Algorithm.

1. `assert_inputs(X, y, foldid, settings$COMMON)` (common.R): §III.3.0 input assertions, plus fold labels 1..10 each with at least 3 rows and no `scaled:*` attributes on X.
2. `assert_pkg_version("glmnet", "5.1")`; `set.seed(seed)`.
3. `zmax <- max(abs(crossprod(X, y)))/n`; stop if it is not finite or ≤ 0. `lam <- GRID$max_mult * zmax * frac`.
4. `cv <- glmnet_cv_engine(X, y, foldid, alpha = 1, lam = lam, pf = rep(1, p), G = GLMNET)` (glmnet_helpers.R, code below). It runs the full-data fit and the 10 training-fold fits on the same grid. Every one of the 11 fits passes `check_glmnet_path()` or the fit fails.
5. With a user grid glmnet never stops the path early: the stop test returns "continue" whenever `flmin >= 1` [V-src `elnet_path/base.hpp` l.307]. `fdev = 0, devmax = 1` are therefore inert and are kept only as a guard.
6. Choose `i_min` and `i_1se` with `cv_choose(cv$cvm, cv$cvsd)` (glmnet's own rule, f).
7. Supports, keys and k for the 200 full-data columns; EBIC candidates once (`ebic_candidates()`); rules (f); return the list of §III.3.0.

```
# glmnet_helpers.R (shared by F1, F2, F3)
glmnet_target_path <- function(Xd, yd, alpha, lam, pf, G) {
  s <- sqrt(mean(yd^2)); m <- alpha + s * (1 - alpha)      # m == 1 exactly when alpha == 1
  fit <- glmnet::glmnet(Xd, yd, family = "gaussian", alpha = alpha / m, lambda = lam * m,
                        penalty.factor = pf, intercept = FALSE, standardize = FALSE,
                        type.gaussian = G$type_gaussian,
                        control = list(thresh = G$thresh, maxit = G$maxit,
                                       fdev = G$fdev, devmax = G$devmax))
  check_glmnet_path(fit, lam * m, G)   # stop(): jerr != 0; length(fit$lambda) != length(lam);
                                       # max(abs(fit$lambda / (lam * m) - 1)) > G$lambda_rel_tol;
                                       # any(fit$a0 != 0); any non-finite beta
  B <- as.matrix(fit$beta)
  scale <- max(1, max(abs(crossprod(Xd, yd))) / nrow(Xd))
  list(B = B, s = s, m = m, a = alpha / m, jerr = fit$jerr, npasses = fit$npasses,
       kkt = kkt_glmnet(Xd, yd, B, lam, alpha, pf, scale))      # length-200 relative KKT
}
glmnet_cv_engine <- function(X, y, foldid, alpha, lam, pf, G) {
  n <- nrow(X); K <- max(foldid); L <- length(lam)
  full <- glmnet_target_path(X, y, alpha, lam, pf, G)
  oof <- matrix(NA_real_, n, L); kkt_fold <- matrix(NA_real_, K, L); fold_diag <- vector("list", K)
  for (k in seq_len(K)) {
    te <- foldid == k
    fk <- glmnet_target_path(X[!te, , drop = FALSE], y[!te], alpha, lam, pf, G)
    oof[te, ] <- X[te, , drop = FALSE] %*% fk$B
    kkt_fold[k, ] <- fk$kkt; fold_diag[[k]] <- fk[c("s", "m", "a", "jerr", "npasses")]
  }
  if (any(!is.finite(oof))) stop("non-finite out-of-fold prediction")
  nk  <- tabulate(foldid, K)
  mse <- t(vapply(seq_len(K), function(k)
           colMeans((y[foldid == k] - oof[foldid == k, , drop = FALSE])^2), numeric(L)))
  cvm  <- drop(nk %*% mse) / n
  cvsd <- sqrt(drop(nk %*% sweep(mse, 2, cvm)^2) / n / (K - 1))
  if (any(!is.finite(mse)) || any(!is.finite(cvm)) || any(!is.finite(cvsd)) || any(cvsd < 0) ||
      length(cvm) != L || any(!is.finite(full$kkt)) || any(!is.finite(kkt_fold)))
    stop("non-finite CV summary or KKT diagnostic")
```

```
  list(full = full, oof = oof, mse = mse, cvm = cvm, cvsd = cvsd,
       kkt_fold = kkt_fold, fold_diag = fold_diag)
}
cv_choose <- function(cvm, cvsd) {            # glmnet's getOptcv rule on a decreasing grid
  i_min <- min(which(cvm <= min(cvm)))        # largest lambda among exact minima
  i_1se <- min(which(cvm <= cvm[i_min] + cvsd[i_min]))
  c(min = i_min, one_se = i_1se)
}
```

- `kkt_glmnet` (relative KKT per λ): g = X_D'(y_D − X_D B)/n_D; for active j, abs(g_j − λ(1 − α)pf_j β_j − λα · pf_j · sign(β_j)); for inactive j, max(abs(g_j) − λα · pf_j, 0); the column maximum divided by `scale`. glmnet applies the penalty factor to both parts of the penalty [V-src `elnet_point/internal/base.hpp` l.61, l.66].
- The CV formulas reproduce `cv.glmnet` with `type.measure = "mse", grouped = TRUE`: `cvm` is the fold-size-weighted mean of the fold MSEs and `cvsd = sqrt(Σ_k n_k (mse_k - cvm)²/Σ_k n_k/(K - 1))` [V-src `cvstats.R` l.6–7, `cvcompute.R`]. `cv_choose` reproduces `getOptcv.glmnet` [V-src l.5–12].
- **Why not `cv.glmnet`.** `cv.glmnet` does not return its fold fits, and a fold fit whose path is truncated (negative `jerr`, maxit reached) still yields finite predictions on the full grid, because `predict()` clamps λ outside the returned path to its end point [V-src `cv.glmnet.raw.R` l.48–58; `buildPredmat.default.R` l.18–19; `lambda.interp.R` l.21–22]. Only an R warning would show it. The engine checks every fold instead (Astra finding 1).
- **Dimensions.** The production grid has 200 points, asserted once in the wrapper (`length(lam) == GRID$nlambda`); the engine and `check_glmnet_path()` check against `length(lam)`, so the canaries in i can use shorter grids.
- **What is shared across the five scales.** Only X and `foldid`. The grid depends on X'y and is recomputed for each scale; every fit is recomputed.

f. Selection rules.

`rule_id`	Role	Definition
`cv_min`	primary	Full-data support at `i_min`: the largest λ among exact ties of the minimum `cvm`. No size cap.
`cv_1se`	S0	Full-data support at `i_1se`: the largest λ with `cvm` ≤ `cvm[i_min] + cvsd[i_min]`. No size cap.
`ebic1`	S0	EBIC($\gamma = 1$) minimiser over the distinct full-data path supports with k ≤ 100 plus the null model.
`ebic05`	S0	The same with $\gamma = 0.5$.

- `cv_min` fails if the KKT check fails at `i_min` in the full-data fit or in any of the 10 fold fits (the fold fits at that index determine its CV value; Astra finding 2). `cv_1se` fails if the KKT check fails at `i_1se` in the full-data fit or any fold fit, **or at `i_min` in any fold fit**, because its threshold `cvm[i_min] + cvsd[i_min]` is computed from the fold fits at `i_min` (Astra round 2, finding 1).
- An EBIC rule fails if no full-data path occurrence of the chosen support passes KKT (the Oct-14 MCP rule, applied to every path method). The null support is always valid.

- Boundary codes: `lambda_first` (index 1, the null model) and `lambda_last` (index 200). A null `cv_min` support is a valid selection.
- Each CV row also carries the OLS RSS and EBIC(1) of its support as diagnostics (Oct-14); these never select.

g. Outputs.

- Rule rows (§IV.1.2): `method_id = "F1"`, `method_label = "Lasso"`, `package = "glmnet"`, `package_version = "5.1"`, four rows as in f. Extra columns: `lambda`, `cv_mean`, `cv_se`, `rss`, `ebic1_value`, `crit_value`, `index`, `k_valid_max`, `k_path_max`, `diag_codes`.
- `beta`: the full-data penalised coefficients at `i_min` (numeric p).
- RDS `diagnostics`: `path` (200 rows: λ, key, k, cvm, cvsd, full-data KKT, maximum fold KKT); `fold_mse` (10 × 200); `fits` (11 rows: data set, `jerr`, `npasses`, max KKT); `candidates` (key, k, rank, rss, valid, ebic05, ebic1); `heldout` (n × 2: out-of-fold predictions at `i_min` and `i_1se`); `foldid`; `zmax`; `index`.
- Size (EST): 50–90 KB per case uncompressed, mostly support keys; the RDS is written with `compress = "xz"` (Oct-14).

h. Convergence, warnings and failures.

- Fit failed: an R error in any of the 11 fits, including a `check_glmnet_path()` stop (`jerr` $\neq$ `0`, which glmnet reports as a warning plus a truncated path for negative codes and as an error for positive codes [V-src `elnet.R` l.50–52; `exceptions.hpp` l.14–21]; a returned path shorter than or different from the requested grid; nonzero intercept; non-finite coefficients); non-finite out-of-fold predictions, fold MSEs, `cvm`, `cvsd` or KKT values.
- Rule failed: as in f.
- Flag (`diag_flag = TRUE` on all four rows, code `kkt_path`): a KKT violation anywhere in the 11 fits. An unselected point can still influence `i_min` or the 1-SE threshold; the flag puts the case in the all-saved but not the passing-only count (§IV.1.3). The pilot reports the flag rate next to the results.
- Warning (`status = "warning"`): any captured R warning or message (§III.3.0).

i. Verification.

- **Orthogonal canary** (G3 item 1; n = 200, p = 30, X'X = (n − 1)I, X centred). Closed form at every grid λ: β_j = S(z_j, λ)/s² with z_j = X_j'y/n, s² = (n − 1)/n and S the soft threshold. Agreement within 1e-8 · max(1, zmax).
- **Cross-check against `cv.glmnet`**: on one dev case per p, `cv.glmnet(X, y, alpha = 1, lambda = lam, foldid = foldid, intercept = FALSE, standardize = FALSE, type.gaussian = "naive", type.measure = "mse", grouped = TRUE, keep = TRUE, control = <same>)` must give the same `cvm`, `cvsd` (1e-10 relative), `fit.preval` (1e-10) and indices (`fit$index`, exact) as the engine.
- **KKT at p > n**: all 11 fits of one dev case per p, every grid point ≤ 1e-5.
- **Grid check**: the first grid point is the null model on the full data.
- **No-intercept canary** (G3 item 3, §IV.3): with the same grid, the full-data path for y + 3 equals the path for y within 1e-8, because 1'X = 0. CV curves are not expected to agree, because training folds are not centred.

- **Control hygiene**: `glmnet.control()` is unchanged after a fit; a test fails if a deprecation warning is captured.
- **Determinism**: two runs give identical supports and bit-identical `cvm`.
- **Pilot (G6) measures**: runtime and MaxRSS; `npasses`; the maximum KKT in each fit; the rates of `lambda_last`, null `cv_min`, `kkt_path` and `ebic_at_largest`; the size distribution of each rule. Pre-declared actions (diagnostics only, before G7; §IV.2.2): `lambda_last` for `cv_min` in more than 5% of pilot cases → `GRID$min_ratio` = 1e-5 for F1–F4; any non-zero `jerr` → `GLMNET$maxit` = 1e7, or `GLMNET$thresh` = 1e-12 for F1–F3 together if the runtime forbids.

j. Compute (EST).

- Basis: Oct-14 NOTS single-thread median for `cv.glmnet` with the same 11 fits at p = 500 and `thresh = 1e-9`: 0.78 s per case (p95 1.39 s), EBIC refits included [V-run, recomputed from `validated_rows.csv`]. The tighter threshold and the 11 KKT products ($2 \cdot n \cdot p \cdot 200$ flops each) add an unmeasured amount; the cost per pass is about proportional to p.
- EST: 1–2 s at p = 500, 2–4 s at p = 1000. Memory: two p × 200 matrices alive at a time (1.6 MB each at p = 1000) plus the n × 200 prediction matrix; the Oct-14 three-method pilot peaked at 257 MiB MaxRSS [V-run `reviews/pilot_resources_2200771.json`].
- F1 has no SLURM request of its own; it runs inside the dataset task (§IV.2.1).

k. Code to write.

- `code/R/config.R`: the lists `COMMON`, `GRID`, `GLMNET`, `F1` (names as in d; `GLMNET <- list(type_gaussian = "naive", thresh = 1e-16, maxit = 1e7, fdev = 0, devmax = 1, lambda_rel_tol = 1e-12, kkt_rel_tol = 1e-5, pkg_version = "5.1")`).
- `code/R/common.R` (shared by all methods): `assert_inputs(X, y, foldid, common)`, `assert_pkg_version(pkg, version)`, `lambda_fraction(grid)`, `support_key(idx)`, `parse_key(key)`, `ols_rss(X, y, idx, tol)`, `ebic_candidates(X, y, supports, kmax, gammas, tol)`, `ebic_pick(cand, column)`, `occurrence_ok(key, path_keys, path_ok)`, `make_rule_rows(...)`, `with_captured_conditions(expr)` (value, warnings, messages, error), `fail_rows(...)`, `elapsed()`, IO and timing helpers.
- `code/R/methods/glmnet_helpers.R` (new; F1–F3): `glmnet_target_path()`, `glmnet_cv_engine()`, `cv_choose()`, `kkt_glmnet()`, `check_glmnet_path()`.
- `code/R/methods/fit_F1_lasso.R`: `fit_F1_lasso(X, y, foldid, seed, settings)` with `settings` the full configuration list of §III.3.0 (`settings$GLMNET`, `settings$F1`, …). It uses `assert_inputs`, `assert_pkg_version`, `lambda_fraction`, `support_key`, `ebic_candidates`, `ebic_pick`, `occurrence_ok`, `make_rule_rows` (common.R) and the engine; the runner wraps the call in `with_captured_conditions`.
- Tests: `code/tests/test_F1.R` (item i) and `code/tests/test_common.R` (EBIC scorer fixtures from Oct-14 `validate.R`: null RSS = Σy^2, a hand-computed two-variable score, rank deficiency → Inf, singleton support, tie order).

l. Open items.

- Runtime with `thresh = 1e-16` at p = 1000 (EST; G6). If the pilot p95 of F1–F3 together exceeds three times the EST, the lead re-plans before G7 on diagnostics only.
- Whether all 11 fits pass KKT at `thresh = 1e-16` at p > n is UNVERIFIED until G3/G4; the pilot actions in i cover a failure.

III.3.2 F2 Elastic net

a. What it estimates and why this configuration. The elastic net minimises RSS/(2n) + $\lambda[\alpha\|\beta\|_1 + (1-\alpha)\|\beta\|^2/2]$ on the supplied X, with no intercept. (α, λ) are tuned jointly by 10-fold CV on the common folds over $\alpha \in \{.1, .25, .5, .75, .9\}$ (O8); $\alpha = 1$ is F1, and $\alpha = 0$ (ridge) is not sparse. The objective is the glmnet one with an exact correction for glmnet's response scaling (c).

b. Software. As F1: `glmnet` 5.1, same location and version assertion; `glmnet::glmnet` called through the shared engine. `cv.glmnet` cannot be used: the correction needs a different native (α, λ) for every training fold.

c. Inputs, preprocessing and the response-scaling correction.

- `x`, `y`, `intercept = FALSE`, `standardize = FALSE`, `type.gaussian = "naive"`, `penalty.factor` all 1, controls in `control = list(...)`, exactly as F1 c.
- **Derivation** [V-src]. Let s = sqrt(mean(y^2)) of the data being fitted (the full data or one training fold). glmnet replaces y by y/s and the user λ by l/s [`standardize.hpp` l.43–47; `gaussian.hpp` l.437], minimises $(1/2n)\|y/s - X\beta'\|^2 + (l/s)[a\|\beta'\|_1 + (1-a)\|\beta'\|^2/2]$ by coordinate descent [`elnet_point/internal/base.hpp` l.50–68], and returns $\beta = s \cdot \beta'$ [`gaussian.hpp` l.451]. Substituting $\beta' = \beta/s$ and multiplying by s^2 gives the original-scale objective $RSS/(2n) + l \cdot a\|\beta\|_1 + l(1-a)\|\beta\|^2/(2s)$.
- To obtain the target objective, solve $l \cdot a = \lambda\alpha$ and $l(1-a)/s = \lambda(1-\alpha)$. The unique solution is $m = \alpha + s(1-\alpha)$, $a = \alpha/m$, $l = \lambda \cdot m$. Both identities hold exactly. This is the Oct-7 map (`ElasticNet/METHOD.md`), re-derived here from the 5.1 source; Astra found it supported by the source.
- s is recomputed from each training fold's own y, so no held-out response enters a fold fit. Without the map, `cv.glmnet` would weight the $\ell 2$ term by 1/s, which differs between folds and between scales.

d. Parameter table.

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
α grid (target)	.1, .25, .5, .75, .9	§III.1, O8	tuned by CV	—	`F2$alphas`
λ grid (target), per α	`1.01·zmax/` `α·frac`, 200 points	Oct-7 range; 1.01 as F1	tuned by CV	via X, y	`GRID$*`
native `alpha`	a = α/m	derived (c)	per fit	via that fit's y	(computed)
native `lambda`	l = $\lambda \cdot$ m	derived (c)	per fit	via that fit's y	(computed)
`control`	thresh 1e-16, maxit 1e6, fdev 0, devmax 1	Oct-7	fixed	—	`GLMNET$*`
`type.gaussian`	`"naive"`	as F1	fixed	—	`GLMNET$type_gaussian`
Fits per case	5 α × (1 full + 10 folds) = 55	design	fixed	—	(derived)

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	config.R name
CV loss and SE	as F1 (engine)	Oct-7; cv.glmnet definitions	fixed	—	(engine)
Tie rule	largest α, then largest λ	Oct-7	fixed	—	F2$ties
KKT tolerance	1e-5 relative to max(1, max_j abs(X_j'y) / n_fit)	Oct-7 gate; scale unified with F1	fixed	—	GLMNET$kkt_rel_tol
Returned-λ tolerance	1e-12 relative	Oct-7	fixed	—	GLMNET$lambda_rel_tol
EBIC cap, γ	100; 1	§III.3.0	fixed	n	COMMON$*

e. Algorithm.

1. Assertions and seed as F1. `zmax <- max(abs(crossprod(X, y)))/n`.
2. For each α in `F2$alphas`: `lam <- GRID$max_mult * zmax / alpha * frac` (the target grid, fixed by the full data and shared by the 10 folds); `cv[[a]] <- glmnet_cv_engine(X, y, foldid, alpha, lam, rep(1, p), GLMNET)`. Keep the full-data coefficient matrix only for the chosen α (step 4) and the keys and k for all five.
3. Stack the five α tables into one 1,000-row table (α, λ index, λ, cvm, cvsd, k on the full path).
4. Choose (α, *i*) by the tie rule (f); i_1se within α*.
5. Rules (f), KKT status (h), return.

- The map gives, for each fit, `s`, `m` and the native `a` and `l`; they are stored per fit (g).
- **Shared across scales:** X and `foldid` only; all 275 fits per dataset are recomputed per scale.

f. Selection rules.

rule_id	Role	Definition
cv_min	primary	Minimum cvm over all 1,000 (α, λ). Exact ties: largest α, then largest λ. Support of the full-data fit at that (α, λ). No size cap.
cv_1se	S0	At the chosen α only: the largest λ with cvm ≤ cvm_min + cvsd_min. α is not re-tuned (Oct-7).
ebic1	S0	EBIC(γ = 1) over the distinct supports of the chosen α's full-data path, k ≤ 100, plus the null model.
ebic05	S0	The same with γ = 0.5 (as F1).

- `cv_min` fails if the KKT check fails at (α, *i*) in the full-data fit or in any of the 10 fold fits at α* (Astra finding 2). `cv_1se` fails on a violation at its own (α, *i_1se) in the full or fold fits, or at (*α, i*) in any fold fit, because its threshold comes from (*α, i*) (Astra round 2, finding 1). The EBIC rule fails if no full-data path occurrence of its support passes KKT.
- Codes: `lambda_first`, `lambda_last`, `alpha_edge` (the chosen α is .1 or .9), `ebic_at_largest`.

g. Outputs.

- Rule rows: `method_id = "F2"`, `method_label = "Elastic net"`, four rows. Extra columns: `alpha`, `lambda`, `cv_mean`, `cv_se`, `rss`, `crit_value`, `k_valid_max`, `k_path_max`, `diag_codes`.
- `beta`: full-data coefficients at the `cv_min` (α, λ).
- RDS `diagnostics`: `cv_table` (1,000 rows: α, λ index, target λ, cvm, cvsd, k); `paths` (for each α the 200 full-data keys); `fits` (55 rows: α, data set, s, m, native a, `jerr`, `npasses`, max KKT); `kkt` (55 $\times$ 200 relative KKT, about 88 KB); `fold_mse` (5 $\times$ 10 $\times$ 200); `candidates` for the chosen α; `heldout` (n $\times$ 2); `foldid`.
- Size (EST): 150–300 KB per case uncompressed.

h. Convergence, warnings and failures.

- Fit failed: an R error or a `check_glmnet_path()` stop in any of the 55 fits; non-finite out-of-fold predictions, fold MSEs, `cvm`, `cvsd` or KKT values.
- Rule failed: as in f.
- Flag (`kkt_path`, `diag_flag = TRUE` on all three rows): any KKT violation in any of the 55 fits. `converged` follows §III.3.0 (glmnet `jerr`), not the KKT check. Oct-7 treated any violation as non-convergence of the whole fit. Here a violation at an unselected point is flagged rather than failed (project decision): such a point can change the winner or the 1-SE threshold, so the case drops out of the passing-only count, but failing every case for a violation far from the optimum would turn solver noise into non-recovery. Astra recommended failing or retrying these points in round 1 and accepted the flag as a disclosed study policy in round 2, provided the flag rate is reported with the results; see `ASTRA_RESOLUTION.md`. The flagged fits certify the declared finite-tolerance procedure, not that its CV winner equals the exactly solved winner.

i. Verification.

- **Orthogonal canary**: $\beta_j = S(z_j, \lambda\alpha)/(s_X^2 + \lambda(1-\alpha))$ with $s_X^2 = (n-1)/n$, at every (α, λ), within 1e-8.
- **Map identities**: $l \cdot a = \lambda\alpha$ and $l(1-a)/s = \lambda(1-\alpha)$ to 1e-14 for every fit.
- **Generic-backend cross-check** (G3 item 4) at p = 500, one dev case, $\alpha \in \{.1, .9\}$, 20 λ: `glmnet(..., family = gaussian(), alpha = α, lambda = λ, intercept = FALSE, standardize = FALSE)` minimises the target objective without response scaling (`glmnetFlex.R` computes λ_max from the gradient with no `ys`, l.1040–1093; Oct-7 test) [V-run p = 30]. Both solutions must pass KKT $\leq$ 1e-6 and agree within 1e-6 $\cdot$ max(1, max abs(β)).
- **Engine at $\alpha = 1$** equals F1 (same code path, m = 1); covered by F1's `cv.glmnet` cross-check.
- **Pilot (G6)**: runtime (the main F-side cost), `npasses` per fit, the `kkt_path` rate, the distribution of α^*, the `alpha_edge` and `lambda_last` rates. The F1 pilot actions apply.

j. Compute (EST).

- Basis: 55 path fits against F1's 11 (5×); Oct-7 EN at n = 100, p = 30 took 2.3 s [V-run]; the 55 KKT products cost about 55 $\times$ 2 $\cdot$ n $\cdot$ p $\cdot$ 200 flops (4.4e9 at p = 1000, under 1 s with OpenBLAS).
- EST: 8–15 s at p = 500, 16–30 s at p = 1000. Memory: at most two p $\times$ 200 matrices alive plus the n $\times$ 200 out-of-fold matrix and the five path key vectors.

k. Code to write. `code/R/methods/fit_F2_enet.R`: `fit_F2_enet(X, y, foldid, seed, settings)` and `enet_choose(cv_table)` (tie rule). It uses `assert_inputs`, `assert_pkg_version`, `lambda_fraction`, `support_key`, `ebic_candidates`, `ebic_pick`, `occurrence_ok`, `make_rule_rows` (common.R) and `glmnet_cv_engine`, `kkt_glmnet` (glmnet_helpers.R). `config.R`: `F2 <- list(alphas = c(.1, .25, .5, .75, .9), ties = "alpha_desc_lambda_desc")`. Tests: `code/tests/test_F2.R`.

l. Open items.

- Runtime at p = 1000 (EST; G6).
- The generic-backend test at p > n is still to run (G3); the scaling itself is now read from the source [V-src].

III.3.3 F3 Adaptive lasso

a. What it estimates and why this configuration. The adaptive lasso minimises RSS/(2n) + λ Σ_j v_j abs(β_j) with data-driven weights from a ridge pilot (Zou 2006, $\gamma = 1$). Ridge, not OLS, supplies the pilot because OLS does not exist at p > n. O7 default: weights from one full-data ridge pilot whose κ is tuned by CV on the common folds; λ is then tuned by CV on the same folds with the weights held fixed. The nested, leakage-free version is S2.

b. Software. `glmnet` 5.1 as F1 (the shared engine with `penalty.factor`). The ridge pilot uses base R `svd()`. No other package.

c. Inputs and preprocessing.

- **Ridge pilot.** b(κ) minimises $\|y - Xb\|^2/(2n) + \kappa\|b\|^2/2$, i.e. (X'X + nκI)b = X'y. With the thin SVD X = U D V', b(κ) = V diag(d/(d² + nκ)) U'y, computed from `svd(X)` (min(n, p) = 200 singular values; the centred X has one near-zero value, whose term d/(d² + nκ) is negligible). This is exact and avoids glmnet's response scaling: glmnet with α = 0 would use the $\ell 2$ weight κ/s (§III.3.2 c with a = 0).
- **Ridge CV.** For each training fold, the SVD of `X[train, , drop = FALSE]` gives b_k(κ) for all 61 κ at once; held-out predictions `X[test, , drop = FALSE] %*% b_k(κ)`. `cvm` and `cvsd` as in the F1 engine. κ_min = argmin `cvm`; exact ties go to the larger κ.
- **Weights.** b = b(κ_min) on the full data. w_j = 1/max(abs(b_j), ε) with ε = max(1e-8, 1e-6 · max_j abs(b_j)) (Oct-7 floor), $\gamma = 1$.
- **glmnet normalisation of the penalty factors.** glmnet multiplies the supplied factors by p/Σ ("internally rescaled to sum to nvars") [V-src `elnet_driver/base.hpp` l.10–14; `glmnet.R` doc l.188]. The wrapper passes v = p · w/Σw, which glmnet leaves unchanged, so the objective is exactly RSS/(2n) + λΣ v_j abs(β_j). At α = 1 the response scaling has no effect (F1 c).
- **Weighted λ grid.** The null-model entry value is max_j abs(X_j'y)/(n · v_j). The grid is `1.01 · that · frac` (200 points).
- The fold fits use the same full-data v (O7 default; the leakage is accepted and documented, §III.2).

d. Parameter table.

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
κ grid	`10^seq(3, -6, length.out = 61)`	Oct-7	tuned by CV	—	`F3$kappa`

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	config.R name
κ tie rule	larger κ	Oct-7	fixed	—	F3$kappa_ties
Ridge equation check	relative residual ≤ 1e-7	Oct-7	fixed	—	F3$ridge_tol
γ (weight power)	1	Zou 2006; project	fixed	—	F3$gamma
Weight floor	ε = max(1e-8, 1e-6 · max abs(b))	Oct-7	data	—	F3$floor_abs, F3$floor_rel
penalty.factor	v = p · w/Σw	derived (c)	data	p	(computed)
λ grid	1.01·max_j abs(X_j'y)/(n v_j)·frac, 200 points	as F1	tuned by CV	via X, y, v	GRID$*
glmnet settings	as F1 (alpha 1, naive, control list)	as F1	fixed	—	GLMNET$*
KKT tolerance	1e-5 relative to max(1, zmax_fit)	as F1	fixed	—	GLMNET$kkt_rel_tol
EBIC cap, γ	100; 1	§III.3.0	fixed	n	COMMON$*

e. Algorithm.

1. Assertions and seed as F1.
2. `ridge <- ridge_cv(X, y, foldid, F3$kappa)`: 10 training-fold SVDs and one full-data SVD. Check the normal equations (X'X/n + κI)b = X'y/n for every κ of the full-data fit, using `crossprod(X, X %*% b)/n` (no p × p matrix), relative to 1 + max abs(X'y/n).
3. Weights w, floor ε, v = p · w/Σw; record `n_floored = sum(abs(b) < ε)`.
4. Grid and engine:

```
z   <- abs(drop(crossprod(X, y))) / n
lam <- GRID$max_mult * max(z / v) * exp(seq(0, log(GRID$min_ratio), length.out = GRID$nlambda))
cv  <- glmnet_cv_engine(X, y, foldid, alpha = 1, lam = lam, pf = v, G = GLMNET)
idx <- cv_choose(cv$cvm, cv$cvsd)
```

5. Rules (f); return.

- **Shared across scales:** X and `foldid`. The SVDs depend only on X and the folds and could be cached per dataset; they are recomputed per case instead (EST under 0.5 s at p = 1000), so that each case is self-contained.

f. Selection rules.

rule_id	Role	Definition
cv_min	primary	Full-data support at idx["min"] (largest λ among exact ties). No size cap.
cv_1se	S0	Full-data support at idx["one_se"]. No size cap.
ebic1	S0	EBIC(γ = 1) over the distinct weighted-path supports, k ≤ 100, plus the null model.

rule_id	Role	Definition
ebic05	S0	The same with $\gamma = 0.5$ (as F1).
cv_min_nested	S2 (not run)	Leakage-free nested weights (Oct-7 AdaptiveLasso/fit.R); O7.

- Failure rules as F1 (full-data and fold KKT at the selected index; a KKT-passing occurrence for EBIC). Codes: lambda_first, lambda_last, kappa_edge (κ_min is the first or last grid value), floored_selected (a selected variable has a floored weight), ebic_at_largest.

g. Outputs.

- Rule rows: method_id = "F3", method_label = "Adaptive lasso", four rows. Extra columns as F1 plus kappa.
- beta: penalised coefficients at cv_min.
- RDS diagnostics: ridge (61 rows: κ, cvm, cvsd; κ_min; boundary; maximum normal-equation residual); b, w, v (p-vectors); eps_floor, n_floored; path, fold_mse, fits, candidates, heldout as F1.
- Size (EST): 70–110 KB per case uncompressed.

h. Convergence, warnings and failures. As F1. In addition, a failed ridge normal-equation check, or a non-finite or non-positive v, fails the fit. kappa_edge and floored_selected are informational codes.

i. Verification.

- **Ridge closed form**: against solve(crossprod(X)/n + κ diag(p), crossprod(X, y)/n) for κ ∈ {1e-2, 1, 1e2} at p = 500 (well conditioned), within 1e-8; against the glmnet route with the F2 map at $\alpha = 0$ (m = s, a = 0, l = κ · s), within 1e-6.
- **Penalty-factor invariance**: the path with penalty.factor = 7·v equals the path with v (internal rescaling), within 1e-10.
- **Orthogonal canary**: $\beta_j = S(z_j, \lambda\, v_j)/s_X^2$.
- **Pilot (G6)**: runtime; the κ_min distribution and kappa_edge rate; n_floored and the floored_selected rate (the earlier concern about weight ties at p = 1000); KKT. Pre-declared action: kappa_edge in more than 5% of pilot cases → extend the κ grid by three decades at that end before G7.

j. Compute (EST). 11 thin SVDs of an at most 200 × 1000 matrix ($O(n^2p)$ each) plus one engine call as F1. EST: 2–3 s at p = 500, 3–5 s at p = 1000. Memory as F1 plus V (p × 200, 1.6 MB).

k. Code to write. code/R/methods/fit_F3_alasso.R: fit_F3_alasso(X, y, foldid, seed, settings), internal ridge_path_svd(X, y, kappa), ridge_cv(X, y, foldid, kappa), alasso_weights(b, F3). It uses the same common.R helpers as F1 and glmnet_cv_engine, cv_choose. config.R: F3 <- list(kappa = 10^seq(3, -6, length.out = 61), kappa_ties = "larger", ridge_tol = 1e-7, gamma = 1, floor_abs = 1e-8, floor_rel = 1e-6). Tests: code/tests/test_F3.R.

l. Open items. None beyond the runtime and the pilot actions. Whether the κ grid range suits p = 1000 is unverified against data; the boundary code measures it.

III.3.4 F4 MCP (SCAD as a sensitivity)

a. What it estimates and why this configuration. MCP minimises RSS/(2n) + Σ_j P_MCP(β_j; λ, γ = 3), with P_MCP(t) = λ abs(t) − t²/(2γ) for abs(t) ≤ γλ and γλ²/2 beyond, on the original X with no intercept. ncvreg computes a pathwise local solution by coordinate descent (Breheny & Huang 2011). λ is chosen by EBIC(γ = 1) on OLS refits of the path supports (Oct-14 continuity). SCAD (a = 3.7) is a budgeted S1 sensitivity.

b. Software.

- `ncvreg` 3.16.0 (CRAN 2025-11-06), installed on NOTS in `/home/fb31/R/library` [V-NOTS]; production from the sealed library (G1). It is **not** in the Oct-7 `library_portable` (`ncvreg NA` in `nots_checks_20261019.txt`).
- Version assertion `assert_pkg_version("ncvreg", "3.16.0")`. Function `ncvreg::ncvreg`; formals on NOTS as in `nots_checks_20261019.txt` [V-NOTS]; source as below [V-src].

c. Inputs, ncvreg's mandatory standardisation, and the compensation.

- ncvreg always centres y (`yy <- y - mean(y)`) and standardises X with column means and RMS scales s_j = sqrt(Σ_i (x_ij − x̄_j)²/n) [V-src `ncvreg.R` l.204, l.209; `standardize.c` l.27, l.35]. There is no option to turn this off.
- Our X has mean 0 and sd 1 (denominator n − 1), so s_j = s = sqrt((n − 1)/n) for every column (to 1e-10), and XX = X/s.
- Centring y: because 1'X = 0, ‖y − Xβ‖² = ‖y − ȳ1 − Xβ‖² + nȳ², so the objective changes by a constant only. The returned intercept is discarded.
- Scale: with θ = sβ, the internal problem is (1/2n)‖yc − XXθ‖² + Σ P(θ_j; λ', γ'). For MCP, P(sβ; λ/s, γs²) = P(β; λ, γ) on both pieces: (λ/s) · s · abs(β) − s²β²/(2γs²) = λ abs(β) − β²/(2γ) for abs(β) ≤ γλ, and γs²(λ/s)²/2 = γλ²/2 beyond. So passing `lambda = λ/s` and `gamma = 3s²` (= 2.985) reproduces the original-scale MCP objective exactly, up to a constant (Oct-14 `METHODS_REVIEW.md`; identity re-derived here; Astra found it supported by the source).
- The coordinate update uses the unit-RMS columns (`v = 1` in `MCP()`) [V-src `ncvreg_init.c` l.86–93; `gaussian.c` l.133–137]. MCP needs γ' > 1, which 2.985 satisfies.
- **SCAD has no exact compensation.** SCAD's slope and first knot are both λ. Passing λ/s keeps the slope at λ on the β scale but moves the knots to λ/s² and aλ/s². The S1 fit is therefore reported as "SCAD (a = 3.7) on the RMS-standardised design X/s"; s² = 0.995, a 0.5% shift.
- Path controls: with a user grid ncvreg fits every λ from the first (`user = 1`, `lstart = 0`), stopping only if the active count exceeds `dfmax` or the **total** iteration count over the path reaches `max.iter`; the remaining λ get `iter = NA` and are dropped from the returned path [V-src `gaussian.c` l.81–86, l.101–104; `ncvreg.R` l.254–261].
- Convergence per λ: the largest coefficient change on the internal scale is below `eps·sdy`, sdy = sqrt(mean(yc²)) [V-src `gaussian.c` l.87, l.150].

d. Parameter table.

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
`penalty`	`"MCP"` (S1: `"SCAD"`)	design	fixed	—	(per wrapper)
MCP shape γ (original scale)	3	ncvreg default; Oct-14	fixed	—	`F4$mcp_gamma`

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	config.R name
gamma passed (MCP)	$3s^2 = 2.985$	derived (c)	fixed	n	(computed)
gamma passed (SCAD, S1)	3.7	Fan & Li 2001; ncvreg default	fixed	—	F4$scad_gamma
lambda passed	grid/s; grid = F1 grid	derived (c); Oct-14	EBIC-selected	via X, y	GRID$*
alpha	1 (no $\ell 2$ part)	Oct-14	fixed	—	F4$alpha
eps	1e-8	Oct-14	fixed	—	F4$eps
max.iter	1e6, total over the path	Oct-14	fixed	—	F4$maxit
dfmax	p	Oct-14	fixed	p	(computed ncol(X))
penalty.factor	all 1	default	fixed	p	(default)
convex	TRUE (records convex.min)	default	fixed	—	F4$convex
warn, returnX	TRUE, FALSE	Oct-14	fixed	—	F4$warn, F4$returnX
KKT tolerance	1e-4 absolute	Oct-14 [V-run: 3,000 cases, 0 failures]	fixed	—	F4$kkt_abs_tol
Residual-reconstruction tolerance	$1e\text{-}10 \cdot \Sigma yc^2 + 1e\text{-}8 \cdot \max(\text{loss_pkg}, \text{rss_rec})$	project (new check)	fixed	—	F4$loss_abs_tol, F4$loss_rel_tol
EBIC cap, γ	100; 1, 0.5, 0	§III.3.0	fixed	n	COMMON$*

e. Algorithm (MCP; SCAD identical except penalty, γ and the KKT form).

1. Assertions and seed as F1. `s <- sqrt((n - 1)/n)`; `lam` = the F1 grid (`1.01·zmax·frac`).
2. Call:

```
fit <- ncvreg::ncvreg(X, y, family = "gaussian", penalty = "MCP", gamma = F4$mcp_gamma * s^2,
                      alpha = 1, lambda = lam / s, eps = F4$eps, max.iter = F4$maxit,
                      convex = TRUE, dfmax = ncol(X), warn = TRUE, returnX = FALSE)
b   <- as.matrix(fit$beta[-1, , drop = FALSE])   # original-scale slopes; row 1 is the intercept
L   <- length(fit$lambda)                         # returned points (at most 200)
lam_orig <- fit$lambda * s
```

3. Checks: `L` $\geq$ `1`; `fit$lambda` equals `(lam/s)[1:L]` within 1e-12 relative (as `GLMNET$lambda_rel_tol`); finite `b`. If L < length(lam), add the code `path_truncated` (total iteration cap) and set `diag_flag`.
4. **Residual reconstruction** (new): for every returned point l, `loss_pkg <- fit$loss[l]` (the package's residual sum of squares on the centred response [V-src `ncvreg_init.c` l.63–66; `gaussian.c` l.207]) and `rss_rec <- sum((y - mean(y) - X %*% b[, l])^2)` must satisfy $\text{abs}(\text{loss_pkg} - \text{rss_rec}) \leq 1e\text{-}10 \cdot \Sigma yc^2 + 1e\text{-}8 \cdot \max(\text{loss_pkg}, \text{rss_rec})$. A failure fails the fit. This checks that the returned coefficients are on the original scale and reproduce the package's residuals; it does not check the penalty compensation, which has its own canary (i).

The original/centred relation RSS(y) = RSS(yc) + n$\bar{y}^2$ is tested once in G3, not per fit (Astra finding 6).

5. **KKT on the original scale** per point, g = X'(y − X b_l)/n: active abs(g_j − sign(β_j) · max(λ − abs(β_j)/3, 0)); inactive max(abs(g_j) − λ, 0). A point passes if the maximum ≤ 1e-4 and, for the last returned point, the total iterations are below `max.iter`.
6. EBIC candidates from the L path supports; rules (f).
7. **SCAD (S1)**: `penalty = "SCAD"`, `gamma = 3.7`, `lambda = lam/s`. KKT on the internal scale with θ = s · β, λ' = λ/s, g' = g/s: active abs(g'_j − sign(θ_j) · d(abs(θ_j))) with SCAD derivative d(t) = λ' for t ≤ λ', (aλ' − t)/(a − 1) for λ' < t ≤ aλ', 0 beyond; inactive max(abs(g'_j) − λ', 0); tolerance 1e-4.

- **Shared across scales:** X only (no folds). Every fit is recomputed.

f. Selection rules.

`rule_id`	Role	Definition
`ebic1`	primary	EBIC($\gamma = 1$) over the distinct path supports, k ≤ 100, plus the null model.
`ebic05`	S0	$\gamma = 0.5$.
`bic`	S0	$\gamma = 0$.
SCAD `ebic1`, `ebic05`, `bic`	S1	The same three rules on the SCAD path, under `method_id = "F4_SCAD"`, all with role S1.
`cv_min`	S2 (not run)	`cv.ncvreg(X, y, fold = foldid, <same arguments>)`, `lambda.min`. Inside folds ncvreg re-standardises each training X by its own column RMS [V-src `cv-ncvreg.R` l.139; `ncvreg.R` l.204], so the λ/s compensation is approximate there.

- A rule fails if its non-null support has no path occurrence that passes KKT and the iteration check (Oct-14). The null support is always valid.
- Codes: `ebic_at_largest`, `path_truncated`, `kkt_path`.

g. Outputs.

- Rule rows: `method_id = "F4"` (`"F4_SCAD"` for S1), `method_label = "MCP"` (`"SCAD on X/s"`), `package = "ncvreg"`, `package_version = "3.16.0"`. Extra columns: `lambda` (original scale), `rss`, `crit_value`, `path_index`, `k_valid_max`, `k_path_max`, `diag_codes`.
- `beta`: coefficients at the first KKT-passing path occurrence of the `ebic1` support (zero vector for the null support).
- RDS `diagnostics`: `path` (L rows: original λ, key, k, `iter`, `kkt_max`, `passes`, residual-reconstruction error); `candidates`; `convex_min` (path index); `discarded_intercept` (L values); requested and returned λ counts; `internal_scale = s`; `internal_gamma`.
- Size (EST): 30–60 KB per case.

h. Convergence, warnings and failures.

- Fit failed: an R error; non-finite coefficients; altered λ; a residual-reconstruction failure.
- Rule failed: as in f.

- Flag (`diag_flag = TRUE` on all rows of the fit): `path_truncated`, or a KKT failure at any returned point (`kkt_path`).
- Warning: ncvreg's "Maximum number of iterations reached" [V-src `ncvreg.R` l.261] or any other warning.
- MCP is a nonconvex local method; `convex.min` is recorded, not acted on.

i. Verification.

- **Compensation identity** (Oct-14 `validate.R` test, re-run): `ncvreg:::std(X)` scale equals s for every column within 1e-12, and the original and internal objectives differ by $\bar{y}^2/2$ at three β vectors.
- **Orthogonal MCP canary**: X'X = (n − 1)I, z_j = X_j'y/n, $s^2 = (n-1)/n$. Per coordinate: $\beta_j = 0$ if $|z_j| \le \lambda$; $\mathrm{sign}(z_j)(|z_j| - \lambda)/(s^2 - 1/\gamma)$ if $\lambda < |z_j| \le \gamma\lambda s^2$; z_j/s^2 beyond. Compare with `fit$beta[-1, ]` at every path point, within 1e-6.
- **Orthogonal SCAD canary** (documents the non-equivalence): with $u_j = z_j/s$ (the internal correlation), $\lambda' = \lambda/s$ and a = 3.7, ncvreg's threshold [V-src `ncvreg_init.c` l.95–102] gives $\theta_j = 0$ for $|u| \le \lambda'$; $\mathrm{sign}(u)(|u| - \lambda')$ for $\lambda' < |u| \le 2\lambda'$; $\mathrm{sign}(u)((a-1)|u| - a\lambda')/(a-2)$ for $2\lambda' < |u| \le a\lambda'$; u beyond. Compare with `s * fit$beta[-1, ]` in every interval, within 1e-6 (Astra finding 4).
- **KKT at p > n**: all returned points of one dev case per p.
- **Pilot (G6)**: runtime, total iterations, the KKT-failure and `path_truncated` rates, the size of each rule, the `ebic_at_largest` rate (expected to be high for `ebic05`, Oct-14).

j. Compute (EST). Basis: Oct-14 p = 500 median 1.5 s per case (p95 2.4 s), EBIC refits and KKT included [V-run]. EST: 2 s at p = 500, 4 s at p = 1000; the same for SCAD.

k. Code to write. `code/R/methods/fit_F4_mcp.R`: `fit_F4_mcp(X, y, foldid, seed, settings)` and `fit_F4_scad(X, y, foldid, seed, settings)`, both calling `ncv_fit(X, y, settings, penalty)`; helpers `kkt_mcp(X, y, b, lam, gamma)` and `kkt_scad_internal(X, y, b, lam, a, s)`; from common.R `assert_inputs`, `assert_pkg_version`, `lambda_fraction`, `support_key`, `ols_rss`, `ebic_candidates`, `ebic_pick`, `occurrence_ok`, `make_rule_rows`. `config.R`: `F4 <- list(mcp_gamma = 3, scad_gamma = 3.7, alpha = 1, eps = 1e-8, maxit = 1e6, convex = TRUE, warn = TRUE, returnX = FALSE, kkt_abs_tol = 1e-4, loss_abs_tol = 1e-10, loss_rel_tol = 1e-8, pkg_version = "3.16.0")`. `foldid` is accepted and unused (interface). Tests: `code/tests/test_F4.R`.

l. Open items. None numerical. If G3 finds a mismatch in the residual reconstruction, the check is not dropped: the lead first confirms that the installed code is the audited source (G1 hashes), re-derives the coefficient reconstruction and reviews the tolerance, and only then records any change (Astra round 2, finding 6).

III.3.5 F5 Best subset

F5 is ABESS + EBIC($\gamma = 1$) in Regime H. The exact `leaps` + BIC arm of the earlier plan is not run (D1); its specification is kept in §III.4.5.

a. What it estimates and why this configuration. For each size k = 0..100, ABESS (adaptive best-subset selection by splicing, Zhu et al. 2020) returns a support of size k produced by ranked splicing: at each step it tries to exchange the $j \le$ `c.max` = 2 active variables with the smallest backward sacrifice for the j inactive variables with the largest forward sacrifice (with `splicing.type = 2`, j = 2 and then j = 1 [V-src `Algorithm.h` l.546–552]), and it stops when none of these proposed exchanges lowers the training loss by more than $\tau_k = 0.01 \cdot k \cdot \log(p) \cdot \log(\log n)/n$ [V-

src `Algorithm.h` l.174–181, l.504–507, l.520–543]. Other exchanges are not tried, so the result is not certified even against all two-variable swaps (Astra round 2, finding 2). The external EBIC(γ = 1) on zero-intercept OLS refits then chooses k. Exact best subset is infeasible at p $\geq$ 500. ABESS has no global certificate in finite samples.

b. Software.

- `abess` 0.4.11 (CRAN 2025-12-08), installed on NOTS in `/home/fb31/R/library` and in the Oct-7 `library_portable` [V-NOTS]; production from the sealed library.
- Version assertion `assert_pkg_version("abess", "0.4.11")`. Function `abess::abess` (dispatches to `abess.default`); formals on NOTS printed 2026-10-19 [V-NOTS].
- **RNG side effect** [V-src `abess.R` l.338, l.585; V-NOTS deparse]: `abess.default` calls `set.seed(seed)` on entry and `set.seed(NULL)` on exit. After every ABESS fit the session RNG is re-seeded from the clock. No F method needs the RNG after F5, but any method run later in the same R process must set its own seed (§III.3.0).

c. Inputs and preprocessing.

- `x = X`; `y = yc = y - mean(y)`. Because 1'X = 0, every fixed-support RSS changes by the same $n\bar{y}^2$, so supports and slopes are unchanged; this also avoids the nonzero null-model intercept seen in Oct-14 (`METHODS_REVIEW.md`) [V-run]. The splicing compares losses of supports of equal size only, so the common shift cannot change any exchange decision. External EBIC uses the original y.
- `normalize = 0`: no centring or scaling inside ABESS (the gaussian default 1 would normalise) [V-src `initialization.R` l.1089–1103].
- `fit.intercept = FALSE`.
- **`screening.num = NULL` is required.** With `NULL` the R layer sets `screening = FALSE` [V-src `initialization.R` l.505–508]. With `screening.num = p` it sets `screening = TRUE` [l.533] while the C++ call receives `screening_size = -1` (no screening) [`abess.R` l.452]; the R post-processing then maps coefficients through an empty screening set and returns all-zero paths [`abess.R` l.565–570]. This is the Oct-14 0.4.11 bug (diagnostic jobs 2200634 and 2200737) [V-run], now located in the source.
- `support.size = 0:100` is required. The package default would be 0..round(n/(log(log n) · log p)) = 0..19 at p = 500 and 0..17 at p = 1000 [V-src `initialization.R` l.593–594; arithmetic for this plan], barely above the 14 true signals. The largest size must be below n (100 < 200) [l.607].
- `important.search = p`: the splicing considers all p variables (`U_size = N`) [V-src `Algorithm.h` l.231–234]. The default would be min(p, 128) [`initialization.R` l.802].

d. Parameter table.

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
`family`	`"gaussian"`	model	fixed	—	(literal)
`tune.path`	`"sequence"`	Oct-14	fixed	—	`F5$tune_path`
`support.size`	0:100	Oct-14; cap $\lfloor n/2 \rfloor$	fixed	n	`F5$sizes` (= `0:COMMON$kmax_ebic`)

Argument or constant	Value	Source	Fixed or tuned	Depends on n, p	`config.R` name
`tune.type`	`"bic"` (internal; never used to select)	Oct-14	fixed	—	`F5$tune_type`
`normalize`	0	Oct-14	fixed	—	`F5$normalize`
`fit.intercept`	`FALSE`	Oct-14	fixed	—	`F5$fit_intercept`
`lambda`	0 (no $\ell 2$ term)	default	fixed	—	`F5$lambda`
`screening.num`	`NULL`	Oct-14 bug fix	fixed	—	`F5$screening_num`
`important.search`	p	Oct-14	fixed	p	(computed `ncol(X)`)
`c.max`	2	default; Oct-14	fixed	—	`F5$c_max`
`max.splicing.iter`	100 (default 20); **inert here** (h)	Oct-14	fixed	—	`F5$max_splicing_iter`
`splicing.type`	2	default	fixed	—	`F5$splicing_type`
`warm.start`	`TRUE`	default	fixed	—	`F5$warm_start`
`early.stop`	`FALSE` (its code is commented out in 0.4.11 [V-src `path.h` l.112–126])	Oct-14	fixed	—	`F5$early_stop`
`num.threads`	1	§III.3.0	fixed	—	`F5$threads`
`seed`	1	Oct-14	fixed	—	`F5$seed`
RSS consistency tolerance	1e-7 relative	Oct-14	fixed	—	`F5$rss_rel_tol`
Stationarity tolerance	1e-6 · max(1, zmax)	Oct-14	fixed	—	`F5$stat_rel_tol`
Intercept tolerance	1e-12	Oct-14	fixed	—	`F5$intercept_tol`
EBIC γ	1, 0.5, 0	§III.3.0	fixed	—	`COMMON$ebic_gamma`

e. Algorithm.

1. Assertions and seed as F1. `yc <- y - mean(y)`.
2. Call (Oct-14 `bench_abess`) [V-run; arguments V-NOTS]:

```
fit <- abess::abess(X, yc, family = "gaussian", tune.path = "sequence", tune.type = "bic",
                    normalize = 0, fit.intercept = FALSE, support.size = F5$sizes, lambda = 0,
                    screening.num = NULL, important.search = ncol(X), warm.start = TRUE,
                    early.stop = FALSE, c.max = 2, max.splicing.iter = 100,
                    splicing.type = 2, num.threads = 1, seed = 1)
b <- as.matrix(fit$beta)                          # p x length(F5$sizes) = p x 101 in production
```

3. Mandatory consistency checks (Oct-14; any failure fails the fit), with S = `F5$sizes` (production `0:100`, asserted once in the wrapper; canaries may pass e.g. `0:29`): `dim(b) == c(p,`

length(S)); fit$support.size == S; the support of column k has exactly k elements; finite b; all abs(fit$intercept) $\leq$ 1e-12; for every k, the reconstructed RSS sum((y - X b_k)²) equals the OLS RSS of that support and equals 2n·fit$dev[k] + n·$\bar{y}^2$ within 1e-7 relative; active-set stationarity max abs(X_S'(y − X b_k))/n $\leq$ 1e-6 · max(1, zmax).

4. EBIC candidates: the 101 supports (sizes 0..100, all within the cap); rules (f).

- **Shared across scales:** X only. Every fit is recomputed.

f. Selection rules.

rule_id	Role	Definition
ebic1	primary	EBIC($\gamma = 1$) over the 101 ABESS supports.
ebic05	S0	$\gamma = 0.5$.
bic	S0	$\gamma = 0$.

- The package's own best.size and tune.value are recorded, never used.
- Code: ebic_at_largest (here: chosen k = 100).

g. Outputs.

- Rule rows: method_id = "F5", method_label = "Best subset (ABESS)", package = "abess", package_version = "0.4.11", three rows. Extra columns: rss, crit_value, k_valid_max, diag_codes.
- beta: ABESS coefficients at the ebic1 size (equal to the OLS slopes on that support).
- RDS diagnostics: path (101 rows: size, key, package dev, reconstructed RSS, OLS RSS, centred RSS, stationarity, intercept); coefficients (list of 101 nonzero vectors); candidates; best_size_internal; response_center = $\bar{y}$.
- Size (EST): 40–80 KB per case.

h. Convergence, warnings and failures.

- Fit failed: an R error or any consistency check in e step 3.
- **The iteration cap is inert, and no splicing count is returned.** With important.search = p the splicing set is all variables (U_size = N), and the outer loop bounded by max.splicing.iter exits after its first pass [V-src Algorithm.h l.357, l.463–467]. The exchanges themselves run in an inner while (true) loop with no cap [l.407–435]. That loop terminates, because an exchange is accepted only if it lowers the training loss by more than τ_k ≥ 0 and there are finitely many supports [l.537] (derived here). abessGLM_API returns coefficients, losses and criteria, not iteration counts [V-src workflow.h l.258–262], so non-convergence cannot be observed; wall time is the only guard (Astra finding 5). max.splicing.iter = 100 is kept for continuity with Oct-14 and has no effect.
- Warnings as F1.

i. Verification.

- **Orthogonal canary**: with X'X = (n − 1)I (p = 30, sizes 0..29) the best subset of each size is the k largest abs(X_j'y); ABESS must return exactly these supports.
- **Known-bug canary**: with screening.num = ncol(X) the installed 0.4.11 must return an all-zero coefficient path. If it does not, the installed code is not the audited code: stop.

- **Consistency checks** of e step 3 on one dev case per p, plus the Oct-14 deliberate-corruption tests (`validate.R`).
- **Determinism**: two calls on the same data give identical supports and coefficients. The RNG side effect itself is documented, not tested; the runner's reseeding before every method is tested in G3 (§III.3.0; Astra finding 9).
- **Pilot (G6)**: runtime per case (the only guard against slow splicing), the size of each rule, the `ebic_at_largest` rate.

j. Compute (EST). Basis: Oct-14 p = 500 median 0.11 s per case (p95 0.18 s), refits included [V-run]. EST: 0.2 s at p = 500, 0.5 s at p = 1000.

k. Code to write. `code/R/methods/fit_F5_abess.R`: `fit_F5_abess(X, y, foldid, seed, settings)`, internal `abess_consistency(fit, X, y, yc, F5)`; from common.R `assert_inputs`, `assert_pkg_version`, `support_key`, `ols_rss`, `ebic_candidates`, `ebic_pick`, `make_rule_rows`. `config.R`: `F5 <- list(tune_path = "sequence", tune_type = "bic", sizes = 0:100, normalize = 0, fit_intercept = FALSE, lambda = 0, screening_num = NULL, c_max = 2L, max_splicing_iter = 100L, splicing_type = 2L, warm_start = TRUE, early_stop = FALSE, threads = 1L, seed = 1L, rss_rel_tol = 1e-7, stat_rel_tol = 1e-6, intercept_tol = 1e-12, pkg_version = "0.4.11")`. `list()` keeps the named `NULL` element; the wrapper asserts `"screening_num" %in% names(F5) && is.null(F5$screening_num)` and passes `screening.num = F5$screening_num`. `foldid` is accepted and unused. Tests: `code/tests/test_F5.R`.

l. Open items. Splicing non-convergence is not observable (h); the pilot runtime is the only check. No other item.

III.3.6 A1 SuSiE

How to read §III.3.6–§III.3.8. Each method has items (a) to (l) in the same order. Code is written for the layout `code/R/config.R`, `code/R/common.R`, `code/R/methods/fit_<method>.R`, `code/R/run_case.R`, `code/slurm/` (§III.3.0). Source references use a new tag:

- **[V-src]**: read in the package source tarball downloaded from CRAN on 2026-10-19 (SHA-256 in `evidence/BAYESIAN.md`, section "Deployment study 2026-10-19"), or in the Oct-7 vendor copy of SSLASSO, with file and line. "l." is a line of that file. "Derived here" marks algebra done for this plan from a [V-src] fact. The specification was reviewed by GPT-6 Astra; "Astra finding n" and "Astra review n" refer to `deployment_20261019/BAYES/ASTRA_RESOLUTION.md`.

Every constant written `A1$…`, `A2$…`, `A2RS$…`, `A3$…` or in capitals below lives in `config.R` (§III.3.0) and is frozen before the pilot. No constant uses the truth.

(a) Model, algorithm and the quantity read.

```
y = X b + e,          e ~ N_n(0, σ² I_n)          no intercept; y not centred
b = Σ_{l=1..L} b_l,   b_l = γ_l β_l
γ_l ~ Multinomial(1, π),   π_j = 1/p              uniform prior weights
β_l | V_l ~ N(0, V_l)                             V_l and σ² estimated (empirical Bayes)
```

- Inference is IBSS, coordinate ascent on the evidence lower bound (ELBO) over L single-effect factors q_l [V-src `update_each_effect.R` l. 9–43].
- One sweep updates l = 1..L in order. For effect l, with d_j = $\|x_j\|^2$ and the residual r_l = y − Σ_{l′≠l} X E[b_l′]:
  - betahat_j = x_j^T r_l / d_j and $s_j^2 = \sigma^2/d_j$;

  - V_l is re-estimated by Brent's method on log V over [−30, 15], started at log max(betahat_j² − s_j², 1). The old V_l is kept if the new value is worse, and V_l is set to exactly 0 if the single-effect log-likelihood at 0 is at least as large (`check_null_threshold = 0`) [V-src `single_effect_regression.R` l. 135–175];
  - log BF_j = log N(betahat_j; 0, V_l + s_j²) − log N(betahat_j; 0, s_j²); $\alpha_{lj} \propto \pi_j$ BF_j; $\mu_{lj} = (1/V_l + d_j/\sigma^2)^{-1}\, x_j^T\, r_l / \sigma^2$ [l. 75–126].
- After the sweep the ELBO is computed. Then $\sigma^2 = E\|y - Xb\|^2/n$, floored at `var(y)/1e4` [V-src `susie.R` l. 431–444; `estimate_residual_variance.R` l. 39–42].
- **Stopping rule.** The loop stops when ELBO(i+1) − ELBO(i) < `tol`. A decrease therefore also stops it with `converged = TRUE`. If `max_iter` sweeps pass, the package warns "IBSS algorithm did not converge" and sets `converged = FALSE` [V-src `susie.R` l. 420–455].
- **Quantity read.** $PIP_j = 1 - \prod_{l: V_l > prior_tol} (1 - \alpha_{lj})$, with `prior_tol = 1e-9`. Effects with V_l ≤ 1e-9 are dropped [V-src `susie_utils.R` l. 410–441, called at `susie.R` l. 478].
  - A **weak active effect** (small positive V_l, near-uniform α_l) still enters. It raises PIP_j by $\alpha_{lj}(1 - PIP_j)$, about $(1 - PIP_j)/p$ (derived here). Diffuse background of this kind is small: even 50 uniform effects at p = 500 give $1 - (499/500)^{50} \approx 0.095$.
  - Weak effects that concentrate on a few correlated proxies can, algebraically, lift a proxy above 0.5. Whether this happens in S3–S6 is a pilot question (item i).
  - Since $\Sigma_j PIP_j \leq$ the number of active effects, fewer than 2L variables can pass PIP > 0.5 (derived here).
- **Credible sets.** For each effect with V_l > 1e-9 (hard-coded, not `prior_tol`; `susie_utils.R` l. 279) the 95% set is the smallest set of top-α variables with $\Sigma\alpha \geq 0.95$. Duplicate sets are removed, and only sets with minimum |corr| ≥ 0.5 are kept. Purity uses a random subsample of 100 members when a set has more than `n_purity = 100` members (l. 495), so it consumes the RNG [V-src l. 262–345, 479–513].
- These are variational PIPs under SuSiE's own prior. They are not posterior probabilities under the SMC model.

(b) Software.

- Package `susieR` 0.14.2 (CRAN 2025-06-05). Tarball SHA-256 `ba02322eb1f7…`, the same hash pinned by Oct-7 `SuSiE/METHOD.md` [V-src].
- Imports (DESCRIPTION): methods, graphics, grDevices, stats, Matrix, matrixStats, mixsqp, reshape, crayon, ggplot2. On NOTS all are installed: susieR 0.14.2 built for R 4.5.2 on 2026-09-29, Matrix 1.7.5, matrixStats 1.5.0, mixsqp 0.3.54, reshape 0.8.10, crayon 1.5.3, ggplot2 4.0.2 [V-NOTS 2026-10-19]. `Rfast` (optional, purity only, same values) is not installed and is not needed.
- The installed code matches the CRAN source: on NOTS the deparsed `susie()` body contains the stopping test `(elbo[i + 1] - elbo[i]) < tol`, and `susie_get_pip` and the hard-coded `res$V > 1e-09` in `susie_get_cs` print as in the tarball [V-NOTS 2026-10-19].
- **Install route (G1).** Install from the saved tarball into the sealed study library `$STUDY/lib/R452/` together with its imports, as for the other packages (§IV.3).
- **Version assertion** (in `fit_A1_susie`, via `common.R::assert_pkg_version`): `packageVersion("susieR") == A1$PKG_VERSION = "0.14.2"`, else the rule fails with "package version".

(c) Inputs and preprocessing actually passed.

- `X`: the stored matrix, centred, sd 1, so d_j = n − 1 = 199; storage mode double; no column names needed.
- `y`: the stored response column `Y[, scale_key]`, **not centred**.
- `intercept = FALSE`: y is not centred. `standardize = FALSE`: X is not rescaled. Internally `compute_colstats(X, center = FALSE, scale = FALSE)` gives column centres 0, scales 1 and d_j = ‖x_j‖² [V-src `susie.R` l. 369–383]. So the likelihood is the no-intercept one of §II.8.
- `var(y)` (the usual centred sample variance) enters three places only: the initial V_l = 0.2 · var(y), the initial σ^2 = var(y), and the floor var(y)/1e4 [V-src `initialize.R` l. 52–90; `susie.R` l. 323]. These are scales, not an intercept.
- **Effect of the uncentred y (derived here).** X is centred, so $x_j^T y = x_j^T (y - \bar{y}1)$: every betahat_j, α_lj and μ_lj is the same as with centred y at the same σ^2. Only σ^2 changes: E‖y − Xb‖²/n = (E‖y_c − Xb‖² + n$\bar{y}^2$)/n, an excess of $\bar{y}^2$. Here $\bar{y}$ = mean(ε) (X centred), so $\bar{y}^2$ is about 1/n = 0.005.
- **Post-fit checks** (Oct-7 assertions): `fit$intercept == 0` and `all(fit$X_column_scale_factors == 1)`. Otherwise the rule fails with "contract".

(d) Parameters. Every argument of `susie()` is passed explicitly, so no result depends on an unstated default. Formals checked on NOTS (`evidence/nots_checks_20261019.txt`) and in the source [V-src `susie.R` l. 301–325].

Argument or constant	Value	Source	Fixed or estimated	Depends on n, p	`config.R` name
`L`	50	project decision: a capacity fixed from n (n/4), which meets the §II.8 rule that caps allow well over 14 (master)	fixed cap	n/4 at n = 200	`A1$L`
`L` in the S1 fit	100	project decision: n/2 (`evidence/BAYESIAN.md` §4(b))	fixed	n/2	`A1$L_S1`
`scaled_prior_variance`	0.2, so V_l starts at 0.2 · var(y)	package default	V_l estimated	—	`A1$SCALED_PRIOR_VAR`
`residual_variance`	`NULL`, so σ^2 starts at var(y)	package default	σ^2 estimated	—	`A1$RESID_VAR_INIT`
`prior_weights`	`NULL`, so π_j = 1/p	package default	fixed	p	`A1$PRIOR_WEIGHTS`
`null_weight`	0 (treated as `NULL`; `susie.R` l. 336–337)	package default	fixed	—	`A1$NULL_WEIGHT`
`standardize`	`FALSE`	project decision 2026-10-07 (§II.8)	fixed	—	`A1$STANDARDIZE`
`intercept`	`FALSE`	project decision 2026-10-07 (§II.8)	fixed	—	`A1$INTERCEPT`
`estimate_residual_variance`	`TRUE`	package default	—	—	`A1$EST_RESID_VAR`
`estimate_prior_variance`	`TRUE`	package default	—	—	`A1$EST_PRIOR_VAR`

Argument or constant	Value	Source	Fixed or estimated	Depends on n, p	config.R name
estimate_prior_method	"optim" (Brent on log V in [−30, 15])	package default	—	—	A1$EST_PRIOR_METHOD
check_null_threshold	0	package default	—	—	A1$CHECK_NULL_THRESHOLD
prior_tol	1e-9	package default	—	—	A1$PRIOR_TOL
residual_variance_lowerbound	var(y)/1e4, passed explicitly	package default formula (susie.R l. 323)	data-derived	—	A1$RESID_LB_DIVISOR = 1e4
residual_variance_upperbound	Inf	package default	—	—	A1$RESID_UB
s_init	NULL (default initialisation: $\alpha = 1/p$, $\mu = 0$)	package default	—	—	—
coverage	0.95	package default	—	—	A1$COVERAGE
min_abs_corr	0.5	package default	—	—	A1$MIN_ABS_CORR
n_purity	100	package default	—	—	A1$N_PURITY
compute_univariate_zscore	FALSE	package default	—	—	A1$UNIV_Z
na.rm	FALSE	package default	—	—	—
max_iter	1000	project (evidence/BAYESIAN.md A1; default 100)	fixed	—	A1$MAX_ITER
tol	1e-3 (absolute ELBO increment)	package default	fixed	—	A1$TOL
verbose, track_fit	FALSE, FALSE	package default	—	—	A1$TRACK_FIT
refine	FALSE	package default (O6)	—	—	A1$REFINE
PIP threshold	0.5, strict (PIP > 0.5)	§II.8	fixed	—	PIP_THRESHOLD (shared)
ELBO-decrease tolerance of the elbo_drop flag (the fit fails beyond A1$TOL)	1e-6	Oct-7 gate; equals the package's own warning tolerance in susie_get_objective (susie_utils.R l. 93)	fixed	—	A1$ELBO_DROP_TOL
BFDR level and roundoff allowance	q = 0.1; 64 machine epsilons	Oct-7 SuSiE/fit.R	fixed	—	A1$BFDR_Q, A1$BFDR_EPS_MULT
Saved α entries	CS members when $\lvert CS\rvert \leq 100$, else the 20 largest	project decision (storage)	fixed	—	A1$ALPHA_TOP = 20
Seed	method seed = data seed + 4,000,000; same for all five scales	§II.7 proposal (O11)	fixed	—	METHOD_SEED_OFFSET

Argument or constant	Value	Source	Fixed or estimated	Depends on n, p	config.R name
S2 susie_auto	L_init = 1, L_max = 512, init_tol = 1, max_iter = 100, tol = 1e-2	package defaults (susie_zzz_auto.R)	—	—	A1$AUTO_*

- **Why L = 50.** It is a prespecified capacity from n, not from the truth. The master's reason, that the default L = 10 "would make recovery impossible", is not correct as stated: PIPs are marginal, and up to 2L − 1 variables can pass PIP > 0.5 (Astra finding 7). L = 10 would still be a binding capacity in practice for 14 effects, and L = 50 removes that concern while keeping the cost linear in L.
- The seed affects only the purity subsample of credible sets larger than 100 members. Such sets are near-diffuse and almost never pass purity 0.5, so PIPs and the primary rule do not depend on it (unit test 6 below checks this).
- Departures from Oct-7 (n = 100, p = 30): L 10 → 50; `tol` 1e-4 → 1e-3 (package default); `refine` TRUE → FALSE (O6). The settled-increment check of Oct-7 (final |ΔELBO| < 1e-4) is dropped because the package stops only when the increment is below `tol`.

(e) Algorithm, step by step. Each of the five scales of a dataset is fitted independently from the default initialisation. Nothing is shared across scales except X, the seed and the settings; there is no warm start.

```
fit_A1_susie <- function(X, y, foldid, seed, settings) {
  assert_inputs(X, y, foldid)                                    # common.R
  assert_pkg_version("susieR", settings$A1$PKG_VERSION)          # common.R
  L <- if (identical(settings$variant, "L100")) settings$A1$L_S1 else settings$A1$L
  set.seed(seed)                                                 # purity subsampling only
  args <- list(X = X, y = y, L = L,
    scaled_prior_variance = settings$A1$SCALED_PRIOR_VAR, residual_variance = NULL,
    prior_weights = NULL, null_weight = 0, standardize = FALSE, intercept = FALSE,
    estimate_residual_variance = TRUE, estimate_prior_variance = TRUE,
    estimate_prior_method = "optim", check_null_threshold = 0,
    prior_tol = settings$A1$PRIOR_TOL,
    residual_variance_upperbound = Inf,
    residual_variance_lowerbound = stats::var(y) / settings$A1$RESID_LB_DIVISOR,
    s_init = NULL, coverage = settings$A1$COVERAGE, min_abs_corr = settings$A1$MIN_ABS_CORR,
    compute_univariate_zscore = FALSE, na.rm = FALSE,
    max_iter = settings$A1$MAX_ITER, tol = settings$A1$TOL,
    verbose = FALSE, track_fit = FALSE, refine = FALSE, n_purity = settings$A1$N_PURITY)
  res <- with_captured_conditions(do.call(susieR::susie, args))  # common.R: warnings AND messages
  if (!is.null(res$error)) return(fail_rows(...))                # gate 1, support = NA
  fit <- res$value
  ...                                                            # steps 4-8 below
}
```

1. Assert inputs and the package version (common helpers).
2. Set the seed. The runner has already set `RNGkind("Mersenne-Twister", "Inversion", "Rejection")` (§III.3.0).

3. Call `susieR::susie` with every argument explicit (above). Capture warnings **and messages**: susieR reports some conditions through `warning_message()`, which calls `message()`, not `warning()` [V-src `susie_utils.R` l. 986–995].
4. Recompute the PIPs with `susieR::susie_get_pip(fit, prune_by_cs = FALSE, prior_tol = settings$A1$PRIOR_TOL)` and assert `identical(unname(fit$pip), pip)`; `fit$pip` carries names when X has column names (`susie.R` l. 481–487) and `susie_get_pip` returns an unnamed vector (`susie_utils.R` l. 440) (Astra review 7). Compute β = `susieR::susie_get_posterior_mean(fit, prior_tol = settings$A1$PRIOR_TOL)` [V-src `susie_utils.R` l. 106–121].
   - **Effect summaries (4a):** `n_active` = #{l : V_l > 1e-9}; `n_cs` = the number of reported credible sets (`length(fit$sets$cs)`); `pip_unreported_max` = max_j of $1 - \prod (1 - \alpha_{lj})$ over the active effects whose index is not in `fit$sets$cs_index` (0 if there are none). The package's `fit$sets$coverage` is saved as returned; it is filtered by `include_idx` but not by `is_pure` (`susie_utils.R` l. 325–338), so it is provenance only and is not used (Astra review 8).
5. Run the gates of item (h).
6. Build the rules of item (f) from `pip` and `fit$sets`.
7. Build the compact saved objects of item (g).
8. Return the §III.3.0 list. The S1 fit is the same function with `settings$variant = "L100"` and `method_id = "SuSiE-L100"`.

(f) Selection rules. `support` is written as sorted, semicolon-separated indices; the empty support is the empty string with k = 0.

- **Primary, `rule_id = "pip05"`:** {j : PIP_j > 0.5}. Strict inequality; no tie rule is needed. There is no cap beyond L: since Σ_j PIP_j $\leq$ the number of effects with V_l > 1e-9, fewer than 2L = 100 variables can pass (derived here).
- **S0, `rule_id = "cslead"`:** for each credible set in `fit$sets$cs` (95% coverage, purity $\geq$ 0.5, deduplicated), the member with the largest PIP; ties go to the smallest index (`which.max` on the increasing index vector). The support is the sorted union of the leads. No credible set gives the empty support.
- **S0, `rule_id = "bfdr10"`:** order the variables by decreasing PIP, ties by increasing index; take the longest prefix whose mean (1 − PIP) is $\leq 0.1 + 64 \cdot$ `.Machine$double.eps`. The empty support if no prefix qualifies (Oct-7 `.susie_bfdr_support`).
- **S1, `method_id = "SuSiE-L100"`:** the same fit with L = 100; it emits only `pip05` (role S1).
- **S2 (not run unless approved):**
  - `method_id = "SuSiE-auto"`: `susieR::susie_auto(X, y, L_init = 1, L_max = 512, standardize = FALSE, intercept = FALSE)` with the package defaults `init_tol = 1, max_iter = 100, tol = 1e-2` [V-NOTS formals; V-src `susie_zzz_auto.R`]. The function doubles L until some V_l = 0. Rule `pip05` from `fit$pip`. Two cautions before it is enabled: its tolerances and iteration budget (100 sweeps per stage, `tol = 1e-2`) differ from the primary fit and must be fixed explicitly; and the loop tests `L <= L_max` before doubling, so it can try L = 1,024 > L_max (l. 99–105).
  - `method_id = "SuSiE-refine"`: the primary call with `refine = TRUE` (O6). Its cost can be large at L = 50: each refinement round runs two full fits per credible set [V-src `susie.R` l. 508–575].

(g) Outputs.

- **Rule rows** (§IV.1.2): three per primary fit (`pip05` primary; `cslead`, `bfdr10` S0) and one per S1 fit. `method_id = "A1"` or `"SuSiE-L100"`, `package = "susieR"`, `package_version = "0.14.2"`, `method_seed` = the seed passed. All rules of one fit share `runtime_s`, `status`, `converged` and `diag_flag`.
- **Saved in the RDS** (list element `diagnostics`), with types and EST sizes at p = 1000:

Object	Type	EST size
`pip`, `beta` (posterior mean)	double, length p each	16 KB
`V` (prior variances), `lbf` (per effect)	double, length L	$<$ 1 KB
`sigma2`, `niter`, `converged`, `intercept`	scalars	—
`elbo` (trace, length `niter`)	double	$\leq$ 8 KB
`sets`: `cs` (list), `purity` (data frame), `cs_index`, `coverage` as returned by susieR (coverage provenance only)	list	$\leq$ 10 KB
`n_active` (#V_l $>$ 1e-9), `n_cs` (#reported sets), `L_saturated` (n_active = L)	integers, logical	—
`pip_unreported_max` (step 4a)	double	—
captured warnings and messages; settings echo	character, list	$<$ 5 KB

Total EST 30–60 KB per fit. The full α (L $\times$ p) is not saved; the PIP vector suffices for the primary rule and its G5 recomputation, and `sets` suffices for the S0 rules.

(h) Convergence, warnings and failures.

- **Failed rule** (`status = "failed"`; every rule of the fit fails; not retried, §IV.2.4). `support` and `k` are NA; the support the fit returned, if any, is kept in `support_audit` (§III.3.0); the scoring step counts it as non-recovery (`error_type = failed`). The null model is never substituted.
  1. `susie()` raises an error (then support = NA).
  2. `fit$converged` is not `TRUE` (`max_iter` reached).
  3. Any ELBO value non-finite, or any decrease larger than `A1$TOL` (1e-3), i.e. `diff(fit$elbo) < -A1$TOL`. The package itself counts a decrease as convergence, so this gate is needed; smaller decreases are rounding-level events of Brent's inexact V_l step and only flag (below).
  4. PIPs not finite or outside [0, 1]; β not finite; σ^2 not finite; any V_l negative or non-finite.
  5. Contract check of item (c) fails, or the version assertion fails.
- **Flagged fit** (`diag_flag = TRUE` on every rule row; `status` is untouched; included in the all-saved count and excluded from the passing-only count, both with denominator 100, §IV.1.3):
  - `L_saturated`: every one of the L effects has V_l $>$ 1e-9, so the cap may bind;
  - `elbo_drop`: some increment satisfies $-$`A1$TOL` $\leq$ `diff(fit$elbo)` $<$ $-$`A1$ELBO_DROP_TOL` ($-$1e-6);
  - `sigma2_floor`: σ^2 equals its lower bound var(y)/1e4 (the package clamps there, so the floor cannot be crossed). Captured warnings and messages set `status = "warning"` only (§III.3.0).
- **Recorded only:** `niter`, the final ELBO increment, `n_active`, `n_cs`, `pip_unreported_max`.

- `converged` in the rule row is `fit$converged` (§III.3.0).

(i) Verification (G3 unit tests and G6 pilot). Tests run on NOTS in the G3 job; every fixture is synthetic and seeded, and none uses the study's truth.

Fixtures are frozen in the test file: fixture seed `TEST_SEED = 20261019L` for every simulated test design, and the stated sizes. A test with a stochastic fixture (4 and 5 below) is run once in G3, its outcome recorded, and only then used as a gate in later reruns (Astra review 10).

1. **Single-effect closed form** (port of Oct-7 `SuSiE/tests.R`): L = 1, fixed V and σ^2, no estimation, uncentred columns of unequal norms. α, μ and the posterior mean must equal the closed forms to 1e-10. The final ELBO must equal the exact log marginal likelihood **plus log(1 + p · √ε_mach)** to 1e-8: susieR adds `√.Machine$double.eps` to every prior weight (`single_effect_regression.R` l. 93, 116–117), which leaves α unchanged under uniform weights but shifts the log evidence by about 1.8e-6 at p = 120 (Astra review 1). Run it twice: n = 12, p = 3 (as Oct-7) and n = 50, p = 120 (p > n).
2. **PIP recomputation:** $1 - \prod$ over the effects with V_l > 1e-9 of (1 − α_lj), coded independently of susieR, equals `unname(fit$pip)` to 1e-12.
3. **No-intercept canary:** with fixed V and σ^2 (L = 1), the fits on y and y + 3 give identical α and μ to 1e-12 (X centred). With σ^2 estimated, `fit$intercept == 0` for both. (The size of the change in σ^2 is not tested: refits change the variational distribution as well; Astra review 4.)
4. **Strong-signal canary at p > n:** n = 200, p = 500, X with independent N(0, 1) columns (seed `TEST_SEED`), centred and scaled; β_j = 1.5 at indices {7, 123, 250, 401, 499}, σ = 1. Expected: `pip05` support equal to the five indices, `converged` TRUE, `L_saturated` FALSE.
5. **Null canary** (β = 0, same X): expected `pip05` empty. Any selection is reported for review; it is not a gate, because the G3 bound "at most 0.3 · p" cannot bind (fewer than 2L = 100 < 150 can be selected).
6. **Determinism and seed scope:** two runs with the same seed are `identical()`; a run with another seed gives `identical()` PIPs (the seed only affects large-set purity).
7. **Gate tests:** a fit with `max_iter = 1` must produce `status = "failed"` rows with the fit's support kept in `support_audit`; a synthetic fit object with an ELBO drop of 1e-5 must fail gate 3 (Oct-7 test).
8. **Rule tests:** the Oct-7 BFDR vectors (`c(0.99, 0.95, 0.75, 0.1)` and `c(.95, .85, .2)`); a CS-lead tie on a hand-made `sets` list.
9. **Orthogonal canary** of §IV.3 (p = 30): `pip05` recovers the five signals.

- **Pilot (G6) must measure**, at p = 500 and 1000, for L = 50 and L = 100: elapsed time and MaxRSS; the distribution of `niter` and the count of `niter` ≥ 500; the non-convergence and ELBO-drop counts; `n_active`, `n_cs` and the saturation rate; `pip_unreported_max` (the hypothesis that several weak effects spread over one correlated block lift a proxy above 0.5). The pilot reads diagnostics only, never recovery.

(j) Compute (EST unless stated).

- **Basis.** One sweep runs L Brent searches, each a few dozen evaluations of an O(p) log-likelihood. It also does about $6 \cdot L \cdot n \cdot p$ multiply-adds ($12 \cdot L \cdot n \cdot p$ flops): four $n \cdot p$ products per effect (removing and adding X E[b_l], the SER's $X^T r$, and the X E[b_l] of the expected log-likelihood) plus two L × p by p × n products for the ELBO and the σ^2 update [V-src `update_each_effect.R`

l. 21–40; `single_effect_regression.R` l. 80; `elbo.R` l. 17–33] (count corrected by Astra finding 6). At L = 50, n = 200, p = 1000 that is about 1.2×10^8 flops per sweep. Sweep cost is linear in L; which part dominates the wall time is measured in the pilot. Measured anchor: Oct-7, n = 100, p = 30, L = 10: 1.9 s median per fit (`refine = TRUE`). The number of sweeps at L = 50, p > n is unmeasured.

- **EST per fit:** 15 s at p = 500 and 30 s at p = 1000 (master figures, kept); S1 with L = 100 about twice that.
- **Memory:** α, μ, μ^2 are $3 \cdot L \cdot p$ doubles (1.2 MB at L = 50, p = 1000); EST peak below 300 MB per R process.
- **SLURM:** A1 runs inside the dataset task (§IV.2.1); it adds no separate request.

(k) Code to write.

- File `code/R/methods/fit_A1_susie.R`. It only defines functions; sourcing it runs nothing.
  - `fit_A1_susie(X, y, foldid, seed, settings)`: the interface of §III.3.0; `settings$variant` ∈ `{"primary", "L100", "auto", "refine"}` selects the fit.
  - `a1_args(X, y, L, settings)`: returns the explicit argument list of item (e).
  - `a1_gates(fit, conditions, settings)`: returns `list(failed, reasons, flags, converged)`.
  - `a1_rules(fit, pip, settings, variant)`: returns the rule-row fragments (rule_id, role, support, k).
  - `a1_bfdr_support(pip, q, eps_mult)` and `a1_cslead_support(sets, pip)`: integer index vectors.
  - `a1_compact(fit, settings)`: the saved objects of item (g).
- Shared helpers from `common.R`: `assert_inputs`, `assert_pkg_version`, `with_captured_conditions`, `support_key`, `make_rule_rows`, `fail_rows` (failure rows; any returned support goes to `support_audit`), `elapsed`.
  - `with_captured_conditions(expr)` evaluates `expr` with `withCallingHandlers` (muffling and recording every warning and message) inside `tryCatch` (recording an error), and returns `list(value, warnings, messages, error)`, with `value = NULL` and `error` a character string when the call failed, else `error = NULL`.
  - `fail_rows(rule_ids, roles, support, reason)` builds `status = "failed"` rows with `support = NA`; `support_audit` holds the returned support, or NA when the fit returned nothing.
- Tests: `code/tests/test_A1_susie.R`, defining `run_A1_tests()` (items i.1–i.9).

(l) Open items.

1. The runtime and the number of IBSS sweeps at L = 50 and 100 for p ≥ 500 (EST). Resolved by the G6 pilot.
2. Whether large L lifts proxies above 0.5 through weak diffuse effects (`pip_unreported_max`). Measured in G6; reported, never used to change L after seeing recovery.
3. O6 (`refine`) stays open; the S2 variant exists if approved.

III.3.7 A2 Spike-and-slab lasso

(a) Model, algorithm and the quantity read.

```
y | β, σ²  ~ N_n(X β, σ² I_n)                                  no intercept; y not centred
π(β | θ)   = Π_j [ θ ψ(β_j | λ1) + (1 - θ) ψ(β_j | λ0) ],     ψ(β | λ) = (λ/2) exp(-λ|β|)
θ ~ Beta(a, b),  a = 1,  b = p                                  ("adaptive" penalty)
π(σ²) ∝ 1/σ², independent of β                                  ("unknown" variance)
```

- The prior on σ^2 is not coded explicitly. The code's update $\sigma^2 = \mathrm{RSS}/(n+2)$ [V-src `SSL_functions.c` l. 102–113] is the conditional mode under $\pi(\sigma^2) \propto 1/\sigma^2$ with β independent of σ^2 (derived here). The Oct-7 audit read the same model from Moran, Ročková & George (2019) (Oct-7 `SSL/METHOD.md`); this plan did not re-read that paper.
- **Algorithm** (coordinate ascent for a posterior mode, along an increasing λ_0 ladder with warm starts) [V-src `SSL_gaussian.c` l. 51–490, `SSL_functions.c`]. Write $d_j = \|x_j\|^2$ (computed in C from the supplied X, l. 82–87), $z_j = x_j^{\mathsf{T}} r + d_j \beta_j$, and
  - $p_\theta(\beta) = \theta\lambda_1 e^{-\lambda_1|\beta|} / (\theta\lambda_1 e^{-\lambda_1|\beta|} + (1-\theta)\lambda_0 e^{-\lambda_0|\beta|})$, $\lambda_\theta(\beta) = \lambda_1 p_\theta(\beta) + \lambda_0 (1 - p_\theta(\beta))$ (l. 56–100);
  - threshold $\Delta_j = \sqrt{(2 d_j \sigma^2 \log(1/p_\theta(0)))} + \sigma^2\lambda_1$ *if* $g_\theta(0) > 0$, *otherwise* $\sigma^2\lambda_\theta(0)$, where $g_\theta(x) = (\lambda_\theta(x) - \lambda_1)^2 + (2 d_j/\sigma^2) \log p_\theta(x)$; $\Delta_j = \sigma^2\lambda_1$ if $\lambda_0 = \lambda_1$ (l. 143–166);
  - update $\beta_j \leftarrow 0$ if $|z_j| \le \Delta_j$, else $\mathrm{sign}(z_j)(|z_j| - \sigma^2\lambda^*_\theta(\hat{\beta}_j^{\mathrm{old}}))_+ / d_j$ (l. 116–141);
  - every `counter` = 10 coordinate updates: $\theta \leftarrow (q+a)/(a+b+p) = (q+1)/(2p+1)$, with q the current number of nonzero coefficients (l. 73–87), and, when σ^2 is being estimated at this point, $\sigma^2 \leftarrow \mathrm{RSS}/(n+2)$, reset to its initial value if below `min_sigma2` (`SSL_gaussian.c` l. 307–326);
  - a ladder point ends when every coefficient's relative change is $\le$ `eps` (0/0 counts as converged; `SSL_functions.c` l. 36–53) and the strong-set and full scans find no new nonzero coefficient; or when the sweep counter reaches `max_iter` (l. 263–475);
  - between points (l. 204–226): θ is recomputed from the previous point; σ^2 is re-estimated during the next point only if the previous point took fewer than 100 sweeps; a capped previous point resets σ^2 to its initial value.
- **Quantity read:** $\hat{\beta}$ at the last ladder point. The selected support is $\{j : \hat{\beta}_j \neq 0\}$ (exact zeros). This equals the wrapper's `$model`, "A single model chosen after the stabilization of the regularization path" [V-src `SSLASSO.R` l. 38, 205].
- It is a mode, not a posterior: there are no PIPs and no evidence. The conditional slab probabilities $p^*_\theta(\beta_j)$ are not marginal PIPs (Oct-7 contract).

(b) Software.

- Package `SSLASSO` **1.2.3.9001**, the Oct-7 patched build of CRAN 1.2.3 (2025-11-04). Upstream tarball SHA-256 `67fb4a881d0b…`; re-downloaded on 2026-10-19 with the same hash [V-src]. Install tarball `02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks/SSL/vendor/SSLASSO_1.2.3.9001.tar.gz`, SHA-256 `f82bf8db6bb4…` (`vendor/patched_manifest.json`).

- **The patch** changes one line of `src/SSL_gaussian.c`, `int thres = n;` to `int thres = 0;` (patched file l. 163), and the DESCRIPTION version (`vendor/SSLASSO-1.2.3.9001.patch`).
  - Upstream takes a Gram-matrix branch when p < min(n, max_iter); its `XTY` loop starts at `i = 1` and drops observation 1 (upstream l. 159–172).
  - With n = 200 and `max_iter = 500`, the threshold is min(n, max_iter) = 200. At p ≥ 500 both builds take the residual-update branch, so **the patch does not change any Regime-H fit** (derived here).
  - The patched build takes the residual branch at every p. The small-p unit tests below therefore exercise the same code path as production.
- Dependencies: stats, graphics (DESCRIPTION) and a C compiler. On NOTS the patched build is in `/home/fb31/projects/BVS/library_portable` (built for R 4.5.2 on 2026-10-05); it is not in `$HOME/R/library`. The registered routine is `SSL_gaussian` with 15 parameters [V-NOTS 2026-10-19].
- **Install route (G1):** `R CMD INSTALL -l $STUDY/lib/R452 SSLASSO_1.2.3.9001.tar.gz` from the vendor tarball, after checking its SHA-256.
- **Version assertions** in `fit_A2_ssl`: `packageVersion("SSLASSO") == "1.2.3.9001"` and `getDLLRegisteredRoutines("SSLASSO")$.Call$SSL_gaussian$numParameters == 15`.

(c) Inputs and preprocessing actually passed.

- **Native call, not the wrapper (O5 default).** The public `SSLASSO()` centres X and y (`XX <- scale(X, center = TRUE, scale = FALSE)`, `yy <- y - mean(y)`) [V-src `SSLASSO.R` l. 123, 127]. It does **not** scale X, although the manual says it standardises; the code's commented-out rescaling confirms that (l. 186–191). The wrapper therefore fits an implicit intercept. The native call passes the stored X and the uncentred y.
- **What centring changes (derived here; scope narrowed after Astra finding 5).** X is centred, so for a **common** β the statistics $x_j^T r$, and hence every z_j, are the same whether r is computed from y or from $y - \bar{y}1$, and RSS_uncentred(β) = RSS_centred(β) + $n\bar{y}^2$. A σ^2 update at a common β therefore differs by $n\bar{y}^2/(n+2)$, about $\bar{y}^2 \approx 1/n$. Once σ^2 differs, the thresholds Δ_j and the shrinkage $\sigma^2\lambda^*$ differ, the paths can separate, and the sweep counts that switch σ^2 estimation on and off can differ. So **no bound on the difference between the final fits follows**; the size of the O5 choice is measured in the pilot. The initial σ and `min_sigma2` use sd(y), which centring does not change. With $\bar{y} = 0$ exactly the two routes coincide, up to the rounding of the wrapper's re-centring of X. The native route with uncentred y is the coherent no-intercept fit of §II.8.
- **Initialisation reproduced from the wrapper** [V-src `SSLASSO.R` l. 154–166]: s_y = `sd(y)`; ncp = $s_y^2 \cdot$ `qchisq(1 - 0.9, 3)`$/3$ (written as in the code block; `qchisq(0.1, 3)` differs in the last bit and can change the β path at p = 1000); $\sigma_0 = \sqrt{(3 \cdot \text{ncp}/5)}$; `min_sigma2` = s_y^2/n; $\theta_0 = 0.5$; $\beta_0 = 0$.
- **Column norms.** The C code uses the actual d_j = 199 (l. 82–87). No rescaling to norm $\sqrt{n}$ is needed, contrary to the manual (Oct-7 audit, confirmed here).

(d) Parameters. Argument order of the native routine [V-src `SSL_gaussian.c` l. 51; `SSLASSO.R` l. 169–171]: X, y, beta.init, penalty, variance, lambda1, lambda0, theta, sigma, min_sigma2, a, b, eps, max_iter, counter.

Argument or constant	**Value**	**Source**	**Fixed or estimated**	**Depends on n, p**	**`config.R` name**
`beta.init`	`numeric(p)` (zeros)	wrapper default	initial	p	—
`penalty`	`"adaptive"` ($\theta \sim$ Beta(a, b), updated)	wrapper default	θ estimated	—	`A2$PENALTY`
`variance`	`"unknown"`	project decision (master; Ray & Szabó 2022) — wrapper default is `"fixed"`	σ^2 estimated	—	`A2$VARIANCE`
`lambda1`	$1 = \lambda 0[1]$	wrapper default	fixed	—	`A2$LAMBDA1`
`lambda0`	`seq(1, n, length.out = 100)` = 1, 3.01, …, 200	wrapper default	fixed ladder	upper end n	`A2$NLAMBDA = 100`, `A2$LAMBDA0_MAX = "n"`
`theta`	0.5	wrapper default	initial value	—	`A2$THETA_INIT`
`sigma`	$\sigma_0 = \sqrt{(3 \cdot \text{ncp}/5)}$, ncp = $\text{sd(y)}^2 \cdot$ qchisq(0.1, 3)/3	wrapper formula for `variance = "unknown"`	data-derived initial value	—	`A2$SIGMA_DF = 3`, `A2$SIGMA_QUANT = 0.9`
`min_sigma2`	$\text{sd(y)}^2/\text{n}$	wrapper formula	data-derived floor	n	—
`a`	1	wrapper default	fixed	—	`A2$A`
`b`	p	wrapper default	fixed	p	`A2$B = "p"`
`eps`	0.001 (relative change)	wrapper default	fixed	—	`A2$EPS`
`max_iter`	500 sweeps per ladder point (integer)	wrapper default	fixed	—	`A2$MAX_ITER`
`counter`	10 (integer)	wrapper default	fixed	—	`A2$COUNTER`
σ^2-estimation switch	re-estimate at a point only if the previous point took $<$ 100 sweeps	hard-coded (l. 212–224)	—	—	not configurable
S1 `lambda1`	0.01	Ray & Szabó 2022 §5.2 (`evidence/notes_bayesian/evidence_log.md` E3)	fixed	—	`A2RS$LAMBDA1`
S1 `lambda0`	`seq(1, p, length.out = 200)` ("an arithmetic series between 1 and p with 200 elements")	Ray & Szabó 2022 §5.2	fixed ladder	upper end p	`A2RS$NLAMBDA = 200`, `A2RS$LAMBDA0_MAX = "p"`
Tail-stability window	last 5 ladder points	Oct-7	diagnostic	—	`A2$STABLE_TAIL`
Fixed-point flag tolerance	$10 \cdot \text{eps} \cdot (1 + \max\lvert\beta\rvert)$	project diagnostic tolerance scaled with `eps` (see (h))	diagnostic	—	`A2$FP_TOL_MULT = 10`
Seed	method seed recorded; SSL uses no random numbers	—	—	—	`METHOD_SEED_OFFSET`

- The string values "p" and "n" in A2$B, A2$LAMBDA0_MAX and A2RS$LAMBDA0_MAX are resolved to the numbers by a2_ladder() and the native-call helper; the call receives only numbers.
- Departures from Oct-7 (n = 100, p = 30): Oct-7 froze λ1 = 0.1, the ladder c(1:100, 125, 150, 175, 200), eps = 1e-7 and max_iter = 5000. This plan uses the package defaults as the primary (master), so the Oct-7 audit tolerances are rescaled to eps (item h).

(e) Algorithm, step by step. SSL is deterministic. Each scale is fitted independently from β = 0; nothing is shared across scales.

```
a2_native_path <- function(X, y, lambda0, lambda1, settings, sigma0 = NULL) {   # sigma0: tests only
  n <- nrow(X); p <- ncol(X)
  stopifnot(all(diff(lambda0) > 0), lambda1 <= min(lambda0))   # the wrapper's own checks (l. 140-148)
  s_y  <- stats::sd(y)
  ncp  <- s_y^2 * stats::qchisq(1 - settings$A2$SIGMA_QUANT, settings$A2$SIGMA_DF) / settings$A2$SIGMA_DF
  sig0 <- if (is.null(sigma0)) sqrt(settings$A2$SIGMA_DF * ncp / (settings$A2$SIGMA_DF + 2)) else sigma0
  storage.mode(X) <- "double"
  res <- .Call("SSL_gaussian", X, as.double(y), as.double(numeric(p)),
               settings$A2$PENALTY, settings$A2$VARIANCE,
               as.double(lambda1), as.double(lambda0), as.double(settings$A2$THETA_INIT),
               as.double(sig0), as.double(s_y^2 / n), as.double(settings$A2$A), as.double(p),
               as.double(settings$A2$EPS), as.integer(settings$A2$MAX_ITER),
               as.integer(settings$A2$COUNTER), PACKAGE = "SSLASSO")
  list(beta = matrix(res[[1]], nrow = p, ncol = length(lambda0)),   # p x L, column-major
       iter = res[[3]], theta = res[[4]], sigma = res[[5]],
       sigma0 = sig0, min_sigma2 = s_y^2 / n)                        # res[[2]] (loss) is not used
}
```

1. Assert inputs, the package version and the routine registration; requireNamespace("SSLASSO") loads the DLL so that the .Call by name resolves (the Oct-7 route) [V-run Oct-7].
2. Build the ladder: primary λ0 = seq(1, n, length.out = 100), λ1 = 1; S1 (method_id = "SSL-RS") λ0 = seq(1, p, length.out = 200), λ1 = 0.01.
3. Call a2_native_path. The output list has five elements: β (p · L), loss, iter, theta, sigma [V-src SSL_gaussian.c l. 21–40].
4. **Do not use res[[2]] (loss).** At a point that stops at max_iter without a clean final scan, loss, theta and sigma are never written: theta and sigma stay NaN and loss is uninitialised memory [V-src l. 104–114, 467–474]. Recompute RSS in R at every point: colSums((y - X %*% beta)^2).
5. Per ladder point record: λ0, iterations, capped (iter = max_iter), θ, σ, k, R-side RSS, support key.
6. Final point L: β = beta[, L]; support $\{j : \beta_j \neq 0\}$.
7. Run the gates and the audit of item (h).
8. Build the rules and the saved objects; return the §III.3.0 list. β returned = β at the last point.

(f) Selection rules.

- **Primary, rule_id = "ssl_final":** $\{j : \beta_j \neq 0\}$ at the last λ0 of the default ladder. Exact zeros; no ties; no cap.
- **S0:** none.

- **S1, `method_id = "SSL-RS"`, `rule_id = "ssl_final"` (role S1):** the same rule on the Ray–Szabó ladder ($\lambda 1 = 0.01$, $\lambda 0$ from 1 to p in 200 steps). Reason: the default ladder stops at $\lambda 0 = n = 200 < p$.
- **S2 (not run unless approved), `method_id = "SSL-wrapper"`:** the public wrapper `SSLASSO::SSLASSO(X, y, penalty = "adaptive", variance = "unknown")` with every other argument at its default, rule `$model`. It is the O5 alternative (implicit intercept) and costs one more SSL fit.

(g) Outputs.

- **Rule rows:** one per fit; `method_id = "A2"` or `"SSL-RS"`; `package = "SSLASSO"`, `package_version = "1.2.3.9001"`.
- **Saved in the RDS:**

Object	Type	EST size
`path`: per ladder point $\lambda 0$, iterations, capped, θ, σ, k, RSS (R-side), support key	data frame, 100 rows (200 for SSL-RS)	10–40 KB (support keys may be long at the first points)
`beta_final`	double, length p	8 KB
`audit`: fixed-point error, support mismatch, threshold Δ, z at the final point; tail stability	list	25 KB
`capped_index`, `n_capped`, `sigma0`, `min_sigma2`, `theta_L`, `sigma_L`, `rss_L`	integers, doubles	—
settings echo; warnings	list, character	< 5 KB

The full β path ($p \times 100$ doubles, 0.8 MB at $p = 1000$) is not saved; the support keys and RSS per point are.

(h) Convergence, warnings and failures.

- **Failed rule** (`status = "failed"`; not retried):
  1. the native call errors, or the version or registration check fails;
  2. any β at the final point non-finite;
  3. the final point is capped (iterations = `max_iter`); this is the wrapper's own "Algorithm did not converge at the last lambda0 value" condition [V-src `SSLASSO.R` l. 182–184];
  4. θ or σ at the final point non-finite or outside $(0, 1)$ and $(0, \infty)$.
- **Flagged fit** (`diag_flag = TRUE`; `status` is untouched):
  - `capped_intermediate` (**informational code only** since the fourth revision; it does not set `diag_flag`): any capped intermediate point (`n_capped` > 0). Oct-7 found persistent 5- and 9-sweep cycles at intermediate penalties in S4; they do not invalidate the final point;
  - `tail_unstable`: the supports at the last `A2$STABLE_TAIL` $= 5$ points are not all identical;
  - `fixed_point`: the fixed-point audit fails one of its two checks (below).
- **Native stopping is not an endpoint certificate** (Astra finding 1). The native test checks coefficient changes only; θ and σ^2 are updated every 10 coordinate updates; the thresholds Δ_j are recomputed with the σ^2 in force **before** that tick's σ^2 update (`SSL_gaussian.c` l. 309–326),

and σ^2 estimation can be off at a point. An independent check that uses the exported θ_L and σ_L can therefore fail although the solver stopped normally. For that reason the audit below only flags; it never fails a rule.

- **What is not ported from Oct-7** `SSL/fit.R` (Astra finding 2): its whole-path finiteness check of θ and σ (capped intermediate points may legitimately keep `NaN`), its comparison with the native `loss` at every point (uninitialised at capped points), and its requirement that the last five points pass the endpoint checks as a condition of convergence. Intermediate θ, σ and loss that the C code did not write are stored as `NA`. Tail stability is a flag here, not a gate.
- **Fixed-point audit at the final point** (port of Oct-7 `.ssl_coordinate_audit`, with tolerances matched to `eps`). Recompute z_j, p_θ, λ_θ and Δ_j from β, θ_L and σ_L, apply one coordinate map and compare:
  - support of the mapped β with the returned support (any mismatch flags);
  - max_j $\|\text{map}(\beta)_j - \beta_j\| \le 10 \cdot \text{eps} \cdot (1 + \max\|\beta\|)$. Reason: the solver stops when each coordinate's relative change in one sweep is ≤ eps = 1e-3, so a residual map error of order $\text{eps} \cdot \|\beta_j\|$ is expected. A stricter check, such as the Oct-7 tolerance $1\text{e-}5 \cdot (1 + \max\|\beta\|)$ set for eps = 1e-7, can fail despite native convergence. The factor 10 is a margin for Gauss–Seidel coupling and for the stale thresholds described above (EST; the pilot records the distribution);
  - θ_L, σ_L and the R-side RSS_L at the final point are recorded, not checked: their consistency with RSS_L/(n + 2) and with (q + 1)/(2p + 1) depends on C-internal state (whether σ^2 was being estimated at that point, when θ was last updated) that the routine does not return.
  - The factor $10 \cdot$ eps above is a **project diagnostic tolerance** scaled with eps, not a derived error bound (Astra review 6).
- `converged` in the rule row is TRUE when the final point is not capped (§III.3.0).

(i) Verification (G3 unit tests and G6 pilot).

1. **Analytic lasso limit** (port of Oct-7 `SSL/tests.R`): $\lambda 0 = \lambda 1$, `penalty = "separable"`, `variance = "fixed"`, an orthogonal design with uncentred columns of unequal norms and a nonzero first observation. β must equal $\text{sign}(x^T y)(\|x^T y\| - \sigma^2\lambda)_+ / \|x\|^2$ to 1e-10; a shifted response must move β (no hidden centring in the native routine).
2. **Patched against upstream at p = 500:** install upstream 1.2.3 into a temporary library inside the G3 job; on one dev case both builds give `identical()` β paths, iterations, θ and σ (they run the same branch at p > n).
3. **Native against wrapper** (corrected after Astra review 2). Run the wrapper on the original data, `w <- SSLASSO::SSLASSO(X, y, penalty = "adaptive", variance = "unknown")`, and the native path on exactly the wrapper's internal inputs, `XX <- scale(X, center = TRUE, scale = FALSE)` and `yy <- y - mean(y)` (`SSLASSO.R` l. 123, 127): `a2_native_path(XX, yy, …)` with the default ladder. Require `identical(unname(w$beta), path$beta)`, `identical(w$iter, path$iter)`, `identical(w$thetas, path$theta)` and `identical(w$sigmas, path$sigma)`; the wrapper adds `dimnames` to β (l. 198), hence `unname`. Then compare `a2_native_path(X, yy, …)` (stored X, centred y) with the wrapper: same final support and max $\|\Delta\beta\| \le$ 1e-8 on the frozen dev fixture (dev case p = 500, S4, dev replicate 1, scale c100); this second comparison is a recorded tolerance check, run once in G3 before it is used as a gate.

4. **No-intercept canary:** with `variance = "fixed"` and the same explicit σ for both runs (`sigma0 = 1`, test-only argument), `a2_native_path` on y and on y + 3 gives β equal to 1e-10, because X is centred. With `variance = "unknown"` the two runs differ through σ^2 and are not compared.
5. **Audit code check on an analytic fixture** (changed after Astra review 6): `penalty = "separable"`, `variance = "fixed"`, `sigma0 = 1`, $\theta = 0.3$, $\lambda 1 = 0.5$, $\lambda 0 = 20$, `eps = 1e-12`, `max_iter = 100000`, on a frozen fixture (n = 50, p = 120, seed `TEST_SEED`). With θ and σ fixed the coordinate map has no stale hyperparameters, so `a2_fixed_point_audit` must report a fixed-point error ≤ 1e-9 and a support identical to the map's.
6. **Determinism:** two runs are `identical()`.
7. **Failure path:** `max_iter = 1` makes the final point capped and must give a `status = "failed"` row with the returned support kept in `support_audit`, never the null model.
8. **Orthogonal canary** of §IV.3 (p = 30) with the production settings: the five signals are recovered.

- **Pilot (G6) must measure:** elapsed time and MaxRSS at both p, for the default and the Ray–Szabó ladders; capped final points (expected 0) and capped intermediate points; sweeps per point; tail-instability and audit flag rates; the final σ^2 and k (descriptive); and, on the 60 pilot cases, the support agreement between the native route and the centred-y route (the size of the O5 choice).

(j) Compute (EST unless stated).

- **Basis.** Measured anchor: Oct-7, n = 100, p = 30: 0.12 s median per fit (Oct-7 ladder of 104 points, `max_iter = 5000`). Per sweep the C code costs about n × (number of ever-active coordinates) plus $n \cdot p$ for each screening scan.
- **Pilot hypothesis (not established; Astra finding 8).** The first ladder point ($\lambda 0 = \lambda 1 = 1$) is a lasso with penalty $\sigma_0^2 \cdot 1$ on the z scale, with σ^2 held at σ_0^2 (no estimation at the first point). Since $\sigma_0^2 \approx 0.12 \cdot \mathrm{var}(y)$, this penalty is small, so the first point may activate many coordinates and may run long. The pilot records the first point's support size, RSS, σ_0 and sweeps.
- **EST per fit:** 5 s at p = 500 and 10 s at p = 1000 (master figures, kept); SSL-RS about twice (200 points).
- **Memory:** the C code allocates a p × p Gram buffer even in the residual branch (`R_Calloc(p*p, double)`, l. 144; 8 MB at p = 1000) and a $p \cdot L$ β matrix (0.8 MB primary, 1.6 MB SSL-RS). EST peak below 200 MB.

(k) Code to write.

- File `code/R/methods/fit_A2_ssl.R`:
  - `fit_A2_ssl(X, y, foldid, seed, settings)`: the §III.3.0 interface; `settings$variant` ∈ {`"primary"`, `"RS"`, `"wrapper"`};
  - `a2_check_backend(settings)`: version and registration checks;
  - `a2_ladder(n, p, variant, settings)`: returns `list(lambda0, lambda1)`;
  - `a2_native_path(X, y, lambda0, lambda1, settings)`: item (e);
  - `a2_path_table(X, y, path, lambda0)`: per-point table with R-side RSS;

  - `a2_fixed_point_audit(X, y, beta, sigma2, theta, lambda0, lambda1)`: port of Oct-7 `.ssl_coordinate_audit`;
  - `a2_gates(path, audit, settings)` and `a2_rules(beta_final, variant)`.
- Shared helpers from `common.R`: `assert_inputs`, `assert_pkg_version`, `with_captured_conditions`, `support_key`, `make_rule_rows`, `fail_rows`, `elapsed`.
- Tests: `code/tests/test_A2_ssl.R` with `run_A2_tests()` (items i.1–i.8).

(l) Open items.

1. O5 (native call with uncentred y versus the wrapper) stays open; the pilot measures the difference (item i).
2. The fixed-point tolerance factor 10 (EST); the pilot records the audit distribution, and the factor may be fixed differently at G6 from diagnostics only.
3. The Moran, Ročková & George (2019) variance prior is taken from the Oct-7 audit, not re-read for this plan (UNVERIFIED here).
4. Runtimes at $p \geq 500$ (EST); G6.

III.3.8 A3 Horseshoe (`bayesreg`)

(a) Model, algorithm and the quantity read.

```
y_i = β0 + Σ_j x̃_ij b_j + ε_i,        ε_i ~ N(0, σ²)
x̃_j = (x_j - mean(x_j)) / ‖x_j - mean(x_j)‖      bayesreg's internal standardisation (unit length)
b_j | λ_j, τ, σ ~ N(0, σ² τ² λ_j²)
λ_j ~ C⁺(0, 1),  τ ~ C⁺(0, 1)                     half-Cauchy, via inverse-gamma auxiliaries ν_j, ξ
π(σ²) ∝ 1/σ²,  π(β0) ∝ 1                           flat intercept, always included
β_j = b_j / ‖x_j - mean(x_j)‖                     coefficients returned on the scale of the supplied X
```

- Priors and full conditionals read in the source [V-src `bayesreg.R` l. 865–1182]: b | · is a Gaussian draw (below); β0 | · ~ N(mean of y − Xb, σ^2/n) (l. 883–900); σ^2 | · ~ IG((n + p)/2, $\|e\|^2/2 + \Sigma$ b_j$^2/(2\tau^2\lambda_j^2)$) (l. 908–912), which is $\pi(\sigma^2) \propto 1/\sigma^2$ with the σ-scaled slab; τ^2 | ξ, · ~ IG((p + 1)/2, $1/\xi + \Sigma$ b_j$^2/(2\sigma^2\lambda_j^2)$) and ξ | τ^2 ~ IG(1, 1 + $1/\tau^2$) (l. 998–1007); λ_j^2 | ν_j, · ~ IG(1, $1/\nu_j$ + b_j$^2/(2\sigma^2\tau^2)$) and ν_j | λ_j^2 ~ IG(1, 1 + $1/\lambda_j^2$) (l. 1019–1030).
- Order per iteration: b, β0, σ^2, τ^2, ξ, λ^2, ν. Initial values b = 0, β0 = 0, $\sigma^2 = 1$, $\tau^2 = 1$, ξ = 1, $\lambda^2 = 1$, ν = 1 (l. 774–790).
- **Sampler for b (resolves a master UNVERIFIED item).** bayesreg uses Rue's algorithm when p/n < 2 and the algorithm of Bhattacharya et al. when p/n ≥ 2 (`if (p/n >= 2) mvnrue = F`, l. 416–423). At n = 200 both p = 500 (ratio 2.5) and p = 1000 (ratio 5) use **Bhattacharya et al.** [V-src]. That draw (l. 2299–2338) takes u ~ N(0, D), δ ~ N(0, I_n), solves $(\Phi D \Phi^T + I)w = \alpha - \Phi u - \delta$ by Cholesky, and returns $b = u + D\Phi^T w$, with $\Phi = X/\sigma$, $\alpha = (y - \beta 0)/\sigma$, $D = \sigma^2\tau^2$ diag(λ^2). Its cost is $O(n^2 p)$ per iteration. The manual's wording ("when the number of samples is twice the number of covariates") does not state the rule; the code does.
- Stored after `burnin` iterations, every `thin`-th iteration, until `n.samples` draws are kept; total iterations = burnin + n.samples · thin = 10,000 + 15,000 · 5 = 85,000 under the O10 re-plan (6,000 under the original 1,000 + 5,000, thin 1) (l. 865–1182).
- **Quantity read:** the n.samples draws of β_j (original scale). The selection reads their 2.5% and 97.5% quantiles.

- In the Bhattacharya branch, `fit$mu.beta` is the mean of the draws (the branch returns `r$m = r$x`, l. 2334–2335), not a Rao–Blackwellised mean.

(b) Software.

- Package `bayesreg` 1.3 (CRAN 2024-09-30), pure R (`NeedsCompilation: no`). Tarball SHA-256 `21e19a4b7314…` [V-src]. `Depends: pgdraw (>= 1.0), doParallel (>= 1.0.16), foreach (>= 1.5.1)`; `Imports: stats`. The NAMESPACE imports doParallel, foreach and pgdraw, so all three must be installed even with `n.cores = 1`.
- Dependencies (CRAN 2026-10-19, tarballs saved in `bench_scratch_20261019/bayes/dl/`):
  - `pgdraw` 1.1 (2019-02-19), SHA-256 `05b951b2ecc8…`; compiled C++, `Imports: Rcpp (>= 0.12.16)`. Used only by the logistic model;
  - `doParallel` 1.0.17 (2022-02-07), SHA-256 `b96a25ad105a…`; depends on foreach $\geq$ 1.2.0, iterators $\geq$ 1.0.0, parallel, utils;
  - on NOTS already: Rcpp 1.1.2, foreach 1.5.2, iterators 1.0.14, coda 0.19.4.1 [V-NOTS 2026-10-19]. bayesreg, pgdraw and doParallel are not installed.
- **Install route (G1):** with GCC/14.3.0 and R/4.5.2 loaded, install into `$STUDY/lib/R452` in dependency order from the saved tarballs (Rcpp, foreach, iterators, coda from their saved tarballs first if they are not yet in the study library): `R CMD INSTALL -l $STUDY/lib/R452 pgdraw_1.1.tar.gz doParallel_1.0.17.tar.gz bayesreg_1.3.tar.gz`. G1 then prints `formals(bayesreg::bayesreg)` and checks that the deparsed body contains `p/n >= 2` and `bayesreg.standardise`, so the sampler rule and standardisation read here are what runs.
- **Version assertions** in `fit_A3_horseshoe`: bayesreg "1.3", pgdraw "1.1", doParallel "1.0.17"; after the call, `fit$n.cores == 1`.
- Development (step 5d of §IV.4): no R is installed locally, so A3 is developed with single-seed runs on Owls or in the NOTS G3/G4 jobs.

(c) Inputs and preprocessing actually passed.

- `colnames(X) <- paste0("X", seq_len(p))`; `df <- data.frame(X, y = y)`; formula `y ~ .` y is the stored, uncentred column.
- **The formula must keep its intercept.** bayesreg builds `model.matrix(formula, data)` and then drops its **first column unconditionally** (`X = X[,-1,drop=FALSE]`, l. 342, 362). With `y ~ . - 1` or `y ~ 0 + .` it would drop the first predictor while still sampling an intercept (Astra finding 4, checked in the source). The wrapper asserts `fit$p == p` (l. 523) after the call.
- **Internal standardisation (resolves a master UNVERIFIED item).** `bayesreg.standardise` centres each column and divides it by $\mathrm{sd} \cdot \sqrt{(n-1)}$, its Euclidean length after centring (l. 389–390, 2145–2174). The coefficients are divided back by that length (l. 742–747). For our X (sd 1) every column is divided by $\sqrt{199} \approx 14.1$, so the master's statement "near zero impact since X already has sd 1" is wrong in its reasoning. Consequence (derived here): on the scale of our X the prior is $\beta_j \sim N(0, \sigma^2\tau^2\lambda_j^2/199)$, that is, the global scale has a $C^+(0, 1/\sqrt{(n-1)})$ prior. Any rescaling of the input is undone by the standardisation, so this cannot be neutralised; it is the package's fixed convention and the same for every user. It is not a defect: a global scale of order $1/\sqrt{n}$ is the usual order of magnitude for τ in sparse regression (the specific literature recommendation was not re-checked for this plan). Record it in the method description.

- **Intercept.** β0 is always sampled with a flat prior and cannot be disabled ("The intercept parameter is always included, and is never penalised", l. 21). It is a **method-specific difference** from the study's no-intercept likelihood, and is recorded as such (Astra finding 4; the master's "harmless" is too strong):
  - with centred X, integrating the flat β0 out gives the posterior of (b, σ^2) for the centred response $y - \bar{y}$ with $n - 1$ instead of n residual degrees of freedom (derived here). Compared with the no-intercept likelihood, the residual sum of squares loses the term $n\bar{y}^2$ (about 1 here, as $\bar{y} = \text{mean}(\varepsilon)$) and the σ^2 posterior loses one degree of freedom. The effect on the slopes runs only through σ^2 and is small, but it is not zero;
  - shift invariance (derived here; unit test 3): the b update depends on y only through $\alpha = (y - \beta 0)/\sigma$, and $(\Phi D \Phi^T + I)^{-1} 1 = 1$ with $\Phi^T 1 = 0$. So the same seed on y + a gives the same b draws up to floating-point rounding, and β0 absorbs a;
  - no response transformation is applied.
- **Orientation (resolves a master UNVERIFIED item).** `fit$beta` is p × n.samples: it is allocated `matrix(0, p, n.samples)` and filled column by column (`rv$beta[, k] = b`), and the back-transformation keeps that shape (l. 527, 854, 1134, 634, 742). The wrapper asserts `dim(fit$beta) == c(p, n.samples)`. `fit$tau2` and `fit$sigma2` are vectors of length n.samples after trimming (l. 640–641).

(d) Parameters. Signature `bayesreg(formula, data, model = 'normal', prior = 'ridge', n.samples = 1e3, burnin = 1e3, thin = 5, t.dof = 5, n.cores = Inf)` [V-src l. 252]; not yet checked on NOTS (not installed).

Argument or constant	Value	Source	Fixed or estimated	Depends on n, p	`config.R` name
`formula`, `data`	`y ~ .`, `data.frame(X, y = y)` with columns X1..Xp	project	—	p	—
`model`	`"normal"` (mapped to `"gaussian"`, l. 268–272)	project (default)	—	—	`A3$MODEL`
`prior`	`"hs"`	project	—	—	`A3$PRIOR`
`n.samples`	**15,000** kept draws (5,000 until the O10 re-plan of 2026-10-19)	O10 re-plan from dev diagnostics (fifth revision)	fixed	—	`A3$N_SAMPLES`
`burnin`	**10,000** (1,000 until the O10 re-plan)	O10 re-plan	fixed	—	`A3$BURNIN`
`thin`	**5** (1 until the O10 re-plan; = package default)	O10 re-plan; total 85,000 iterations	fixed	—	`A3$THIN`
`t.dof`	5, passed explicitly (unused by the normal model)	package default	—	—	`A3$T_DOF`
`n.cores`	1 (package default `Inf`)	project	fixed	—	`A3$N_CORES`
Sampler for b	Bhattacharya et al. (p/n ≥ 2)	package rule (l. 416–423)	—	p/n	—
Standardisation	unit length, back-transformed	package (l. 389, 742)	—	n	—
Priors	λ_j, $\tau \sim C^+(0, 1)$; $\pi(\sigma^2) \propto 1/\sigma^2$; flat β0	package	—	—	—

Argument or constant	Value	Source	Fixed or estimated	Depends on n, p	config.R name
Initial values	b = 0, β0 = 0, $\sigma^2 = \tau^2 = \xi = 1$, $\lambda^2 = \nu = 1$	package (l. 774–790)	—	—	—
Credible interval	equal-tailed 95%: `quantile(type = 7)` at 0.025 and 0.975	Porwal & Raftery 2022; equals `summary.bayesreg` (l. 2054)	fixed	—	`A3$CI_PROBS = c(0.025, 0.975)`, `A3$QTYPE = 7L`
ESS threshold of the flag	100	master (Oct-7 practice)	fixed	—	`A3$ESS_MIN`
2-means	`kmeans(abs(mu), centers = 2, nstart = 10, iter.max = 10, algorithm = "Hartigan-Wong")` (the last two are the `stats::kmeans` defaults, passed explicitly; formals [V-NOTS 2026-10-19])	master	fixed	—	`A3$KM_NSTART`, `A3$KM_ITER_MAX`, `A3$KM_ALGORITHM`
Package versions	`A3$PKG_VERSIONS = c(bayesreg = "1.3", pgdraw = "1.1", doParallel = "1.0.17")`	this study	fixed	—	`A3$PKG_VERSIONS`
Seed	method seed = data seed + 4,000,000, the same for all five scales	§II.7 proposal (O11)	fixed	—	`METHOD_SEED_OFFSET`

(e) Algorithm, step by step.

```
fit_A3_horseshoe <- function(X, y, foldid, seed, settings) {
  assert_inputs(X, y, foldid)
  for (pk in names(settings$A3$PKG_VERSIONS))                    # bayesreg, pgdraw, doParallel
    assert_pkg_version(pk, settings$A3$PKG_VERSIONS[[pk]])
  n <- nrow(X); p <- ncol(X)
  colnames(X) <- paste0("X", seq_len(p))
  df <- data.frame(X, y = y)
  set.seed(seed)                        # runner fixed RNGkind("Mersenne-Twister","Inversion","Rejection")
  res <- with_captured_conditions(bayesreg::bayesreg(y ~ ., data = df,
           model = settings$A3$MODEL, prior = settings$A3$PRIOR,
           n.samples = settings$A3$N_SAMPLES, burnin = settings$A3$BURNIN,
           thin = settings$A3$THIN, t.dof = settings$A3$T_DOF, n.cores = settings$A3$N_CORES))
  rng_after <- digest::digest(.Random.seed, algo = "sha256")   # CRN invariant (below)
  if (!is.null(res$error)) return(fail_rows(...))               # support = NA
  fit <- res$value; B <- fit$beta; M <- settings$A3$N_SAMPLES   # B: p x n.samples
  stopifnot(fit$p == p, identical(dim(B), c(p, as.integer(M))), all(is.finite(B)),
           length(fit$tau2) == M, all(is.finite(fit$tau2) & fit$tau2 > 0),
           length(fit$sigma2) == M, all(is.finite(fit$sigma2) & fit$sigma2 > 0),
           fit$n.cores == 1)                                     # a failed check fails the fit
  q  <- apply(B, 1, stats::quantile, probs = c(0.025, 0.5, 0.975), names = FALSE, type = 7)
  mu <- rowMeans(B); s <- apply(B, 1, stats::sd)
  ...                                  # rules, ESS, compact objects; then rm(B, fit)
}
```

1. Assert inputs and the three package versions.
2. Name the columns, build the data frame, set the seed and call `bayesreg::bayesreg` with every argument explicit. With `n.cores = 1` one chain runs in the R session and draws from R's global RNG (`stats::rnorm`, `rgamma`, `rexp`; l. 575–596), so `set.seed()` makes the fit reproducible.
   - `n.cores` is reset to min(detectCores() − 1, n.cores) (l. 543–561). If `detectCores()` returned 1 the effective count would be 0 and the call would fail (samples per core = n.samples/0); if it returned `NA`, so would the count (Astra finding 11). The G4 job records `parallel::detectCores()` on a compute node and requires it to be at least 2; the wrapper asserts `fit$n.cores == 1` after every call.
3. **Common random numbers across the five scales (derived here; conditions added after Astra finding 10).** The runner passes the same seed for the five scales of a dataset. Per iteration the draws are `rnorm(p)`, `rnorm(n)`, `rnorm(1)`, `rgamma` with shape (n + p)/2, `rgamma` with shape (p + 1)/2, `rexp(1)`, `rexp(p)`, `rexp(p)`.
   - R's normal generator with `normal.kind = "Inversion"` uses exactly two uniforms per draw (`snorm.c` l. 265–270); `rgamma`'s rejection loop depends only on the shape, the scale multiplying the result (`rgamma.c` l. 55–214); `exp_rand` has no parameter (`sexp.c`).
   - So, as long as every sampling parameter is finite and positive (a non-finite or zero parameter returns before drawing; `rgamma.c` l. 89–95, `rnorm.c` l. 35–38), the uniform stream is consumed identically for every y and the five scales use the same random numbers at every iteration. Floating-point results still differ across scales, because the data differ.
   - These routines were read in the R trunk source; that they are unchanged in R 4.5.2 is UNVERIFIED (an assumption). Unit test 6 and the per-dataset invariant are **empirical terminal-state checks**, not a proof of source identity or of alignment at every draw (Astra review 13): `rng_after` (the hash of `.Random.seed` taken right after the MCMC and before the randomised `km2` step) must be identical for the five scales of a dataset. A mismatch is flagged `crn_mismatch` on all five A3 fits of the dataset by the scoring step (`score.R`), not by the runner. `RNGkind()` is recorded in every RDS.
4. Assert dimensions, finiteness and `fit$n.cores == 1`; assert $\max|$`fit$mu.beta` − `rowMeans(B)`$| \le$ 1e-10 $\cdot (1 + \max|\mu|)$ (orientation and back-transformation check).
5. Compute per coefficient: mean, sd, the 2.5%, 50% and 97.5% type-7 quantiles, and the package's ESS `fit$ess`.
6. Apply the rules of item (f).
7. ESS with `coda` (Astra review 12 made this exact):
   - `coda::effectiveSize(log(fit$tau2))` and `coda::effectiveSize(log(fit$sigma2))`;
   - `coda::effectiveSize` of the draws of every coefficient selected by `ci95`, giving `min_ess_ci95`, and of every coefficient selected by `km2`, giving `min_ess_km2` (recorded only);
   - an error or a non-finite value in any of these is recorded as `NA`, and for the two flagged quantities it sets the flag (item h). Time this step separately as `diag_s`.
8. Keep the compact objects of item (g); drop the draws (`rm(B, fit)`).

(f) Selection rules.

- **Primary, `rule_id = "ci95"`:** {j : q_0.025,j > 0 or q_0.975,j < 0}, from the type-7 quantiles of the 5,000 draws of β_j (Porwal & Raftery 2022: "we calculated a 95% credible interval

and checked whether 0 was included"). An endpoint exactly 0 counts as including 0. No cap. `summary.bayesreg` computes the same interval (`stats::quantile(s, c(CI.low, CI.high))` on `object$beta[k, ]`, l. 2054), so the master's concern that its construction is undocumented is resolved; the wrapper still computes the quantiles itself, because `summary()` prints a table.

- **S0, `rule_id = "km2"` ("our 2-means implementation"):** `set.seed(seed); km <- stats::kmeans(abs(mu), centers = 2, nstart = settings$A3$KM_NSTART, iter.max = settings$A3$KM_ITER_MAX, algorithm = settings$A3$KM_ALGORITHM);` select the cluster with the larger centre. If `abs(mu)` has fewer than two distinct values, or `kmeans` errors, the rule fails. Kollipara et al.'s exact implementation (k = 2, 3, 4 on \|posterior means\|) is UNVERIFIED; only k = 2 is run.
- **S2 (not run unless approved):**
  - `method_id = "HS+"`: the primary call with `prior = "hs+"` (parameter-expanded horseshoe+, l. 1032–1052), rules `ci95`, `km2`;
  - `method_id = "HS-pkg"`: the archived `horseshoe` 0.2.0 (installed in `$HOME/R/library`), `horseshoe::horseshoe(y, X, burn = 1000, nmc = 5000, thin = 1, alpha = 0.05, …)` [V-NOTS formals: `y, X, method.tau, tau = 1, method.sigma, Sigma2 = 1, burn = 1000, nmc = 5000, thin = 1, alpha = 0.05`]. The values of `method.tau` and `method.sigma` and whether it fits an intercept are UNVERIFIED; read its source before any use.

(g) Outputs.

- **Rule rows:** two per fit (`ci95` primary, `km2` S0); `method_id = "A3"`, `package = "bayesreg"`, `package_version = "1.3"`, `method_seed` = the seed.
- **Saved in the RDS** (draws are never saved; 5,000 × 1,000 × 8 B = 40 MB per case):

Object	Type	EST size at p = 1000
`coef`: per coefficient mean, sd, q025, q50, q975, `ess_pkg` (bayesreg's estimator), `ess_coda` (coefficients selected by `ci95` or `km2`; NA otherwise)	data frame p × 7	56 KB
`trace_log_tau2`, `trace_sigma2`	double, length 5,000 each	80 KB
`ess_log_tau2`, `ess_log_sigma2`, `min_ess_ci95`, `min_ess_km2`	doubles	—
`rng_kind` (`RNGkind()`), `detect_cores`	character, integer	—
`beta0_mean`, `waic`, `n_cores`, `sampler` (`"bhattacharya"` when p/n ≥ 2), `rng_after`	scalars, character	—
`km2`: centres, cluster sizes, `tot.withinss`	list	< 1 KB
`diag_s`, warnings, settings echo		< 5 KB

Total EST 140 KB per case before compression.

- bayesreg's own ESS (`fit$ess`) uses an FFT autocorrelation without zero padding, which is circular (l. 1211–1276). It is saved for completeness only; the flag uses `coda`.

(h) Convergence, warnings and failures.

- **Failed fit** (both rules `status = "failed"`; not retried): the call errors; a version assertion fails; the dimension, finiteness, positivity, length, `n.cores` or posterior-mean checks of item (e) and step 4 fail.
- **Failed rule, `km2` only** (`ci95` unaffected): fewer than two distinct \|μ\| values, or `kmeans` errors. Failure takes precedence over any flag on that row.
- **Flagged fit** (`diag_flag = TRUE`; `status` is untouched; pre-declared; never changes the selection). The flag is set for both rules of the fit when any of these holds:
  - `ess_tau`: ESS(log τ^2) < 100 or non-finite;
  - `ess_ci95`: `min_ess_ci95` < 100 or non-finite (vacuous when `ci95` selects nothing);
  - `crn_mismatch`: set afterwards by `score.R` on all A3 rows of a dataset whose non-failed `rng_after` values differ (step 3). A flagged fit stays in the all-saved counts and is reported separately (Oct-7 practice).
- **What the flag does not show** (Astra finding 3). These are limited mixing diagnostics of one chain. ESS of log τ^2 and of selected coefficients does not establish the Monte Carlo precision of the 2.5% and 97.5% quantiles that the rule reads, and "no flag" does not mean the chain has converged. The pilot therefore adds a two-chain stability check (item i).
- **`converged = NA`:** the MCMC has no convergence certificate; the flag carries the diagnostics. The schema allows this (§III.3.0, §IV.1.2).
- There is one chain per production fit, so there is no between-chain R; the split-chain value is recorded only.

(i) Verification (G3 unit tests and G6 pilot). The master's test "ridge-prior posterior mean against its closed form" is not possible as written: `prior = "rr"` also puts a half-Cauchy prior on τ (l. 998–1007), so its posterior mean has no closed form. It is replaced by tests 1–2.

Fixtures are frozen in the test file (seed `TEST_SEED = 20261019L`; sizes and indices as stated). Tests 1, 2 and 5 are stochastic: each is run once in G3, its outcome recorded, and then used as a gate (Astra review 10).

1. **Bhattacharya kernel against the exact Gaussian:** fixture n = 20, p = 50, Φ with independent N(0, 1) entries, $\alpha \sim$ N(0, I), d_j $\sim$ Uniform(0.5, 2), all drawn with `TEST_SEED`. Call `bayesreg:::bayesreg.fastmvg_bhat(Phi, alpha, d)` M = 200,000 times. The exact law is N(m, Σ) with $\Sigma = (\Phi^{\mathrm{T}}\Phi + \mathrm{D}^{-1})^{-1}$ and $\mathrm{m} = \Sigma\Phi^{\mathrm{T}}\alpha$. Acceptance (calibrated after Astra review 3):
   - every coordinate of the sample mean within 5 standard errors $\sqrt{(\Sigma_jj/\mathrm{M})}$ of m_j (false-rejection probability below 3e-5 over 50 coordinates by a union bound);
   - relative Frobenius error $\|\mathrm{S} - \Sigma\|_\mathrm{F}/\|\Sigma\|_\mathrm{F} \le 4 \cdot \sqrt{(((\mathrm{tr}\ \Sigma)^2/\|\Sigma\|_\mathrm{F}^2 + 1)/(\mathrm{M} - 1))}$, four times its root-mean-square value under correct sampling (about 1.6% at most for this fixture).
2. **Rue kernel:** the same test for `bayesreg:::bayesreg.fastmvg2_rue.gaussian(Phi, alpha, d, PtP = crossprod(Phi), Ptalpha = crossprod(Phi, alpha), sigma2 = 1, PrecomputedXtX = TRUE)` at n = 50, p = 20. `PtP` and `Ptalpha` must be passed: with `PrecomputedXtX = FALSE` the function computes `PtP` but leaves `Ptalpha` as `NA` (l. 2272–2295).

3. **Shift canary (intercept):** the same seed on y and y + 5 (fixture of test 5, test budget). Expected: posterior means and the three quantiles of every β_j equal to 1e-6 (absolute), and the mean of β0 shifted by 5 to 1e-6. The largest difference over all draws is recorded, not gated: the identity is exact in real arithmetic, but rounding in the standardisation and the Cholesky solves can grow along the chain (Astra review 10).
4. **Scale canary (standardisation):** the same seed on X and 3X. Expected: posterior means and quantiles of β_j for 3X equal to those for X divided by 3, to 1e-6 relative; the largest draw-level difference recorded.
5. **Strong-signal canary at p > n:** n = 200, p = 500, independent N(0, 1) columns (seed `TEST_SEED`), centred and scaled; β_j = +3 at indices {7, 250, 499} and −3 at {123, 401}; σ = 1; test budget burnin 500 + 1,000 draws, labelled non-production. Expected: `ci95` selects exactly those five indices, and `rowMeans(fit$beta)` has the planted sign and lies within 0.5 of ±3 at each of them. The asymmetric indices and mixed signs make this a direct check of row order and orientation. `rownames(fit$beta)` is recorded (the source sets them from `labels(rv$mu.beta)`, l. 763; their exact form is not relied on).
6. **CRN invariant:** fits on two different y with the same seed leave identical `.Random.seed`.
7. **Determinism:** two runs with the same seed give `identical(fit$beta)`.
8. **Interval check:** the wrapper's type-7 intervals equal `summary(fit)$CI.coef[1:p, ]` to 1e-12 (output of `summary` suppressed with `capture.output`; `summary` hard-codes `PerSD = F`, l. 1814, so it uses the unscaled draws; its last row is the intercept).
9. **Rule tests:** hand-made quantile tables with endpoints just above, just below and exactly at 0; `km2` on a known two-cluster vector and on a constant vector (must fail).
10. **Orthogonal canary** of §IV.3 (p = 30; there bayesreg uses Rue's algorithm, so this test is not the production path; tests 1, 5 are).

- **Pilot (G6) must measure:** elapsed time and MaxRSS per fit at p = 500 and 1000; ESS(log τ^2), ESS(log σ^2), the smallest ESS over the selected set; the flag rate; the CRN invariant; selected sizes (descriptive).
- **Two-chain stability check (pilot only).** For each of the 60 pilot cases run a second chain with seed + 1,000,000 (not seed + 1, which is another dev dataset's method seed) and the production settings. Report the Jaccard similarity J of the two `ci95` supports and the number of coefficients whose `ci95` decision differs. This compares two runs of the same method with each other, never with the truth. Definitions (Astra review 11): J(∅, ∅) = 1; a pair in which either chain failed counts as J = 0 and is also reported as a failure; J is summarised separately at p = 500 and p = 1000 (median over the 30 cases of that p).
- The pilot result fixes O10 by diagnostics only (pre-declared rule in Appendix A, O10).

(j) Compute (EST unless stated).

- **Basis.** The matrix work of one Bhattacharya draw (X/σ, Φu, $\Phi D \Phi^T$ + I, Cholesky of n × n, two triangular solves, $D\Phi^T w$) was timed in numpy on one thread on the local PC: 2.4 ms at p = 500 and 3.4 ms at p = 1000 (measured 2026-10-19, `bench_scratch_20261019/bayes/scripts/time_bhat.py`, MKL BLAS). bayesreg adds an R-level loop over the p columns per draw (l. 2319–2323), a copy of X per draw (l. 2236) and the scalar and p-vector updates. EST 5–15 ms per iteration at p = 1000, so 30–90 s for 6,000 iterations, plus about 5–10 s of post-processing (feature ranking by 5,000 sorts, l. 652–681, and ESS). EST about half at p = 500.

- **Budget figures kept:** 150 s at p = 500 and 300 s at p = 1000 per fit (master §IV.2.3), as provisional planning values. The numpy basis uses another machine, BLAS (MKL, not OpenBLAS) and language, and omits the package overhead, so it does **not** show that the budget is conservative (Astra review 14); only the G6 pilot measurement replaces these figures.
- **Literature risk:** Kollipara et al. report HS 20–300 s with the `horseshoe` package and HS+ 250–17,000 s with bayesreg at n = 200, p = 1000 and 5,000 iterations; their bayesreg version and settings are not known.
- **Mitigation:** pilot first (G6); split A3 into its own array with `--methods A3` if the task total needs it; re-plan if any pilot fit exceeds 900 s at p = 1000; never shorten the chains after seeing recovery.
- **Pilot extra:** the two-chain stability check doubles A3's pilot cost (60 extra fits; EST about 3.8 core-hours at the budget figures, 30 × 150 s + 30 × 300 s). With it the pilot stays within the ≈ 20 core-hours of §IV.2.3 (derived here: the 60 pilot cases cost about 30 × 183 s + 30 × 367 s ≈ 4.6 core-hours of primary fits at the §IV.2.3 EST rates, plus S1 and this check).
- **Memory:** the draws are 40 MB at p = 1000, and the post-processing makes about three transient copies of that size (`t(rv$beta)`, the back-transformed matrix, the ranks matrix; l. 652–747). EST peak 200–300 MB; the per-task request of 4 GB (§IV.2.3) is ample.

(k) Code to write.

- File `code/R/methods/fit_A3_horseshoe.R`:
  - `fit_A3_horseshoe(X, y, foldid, seed, settings)`: the §III.3.0 interface; `settings$variant` ∈ `{"primary", "hsplus", "hspkg"}`;
  - `a3_check_backend(settings)`: version checks;
  - `a3_call(X, y, seed, settings)`: steps 2–3, returns `list(fit, rng_after)`;
  - `a3_summaries(fit, settings)`: step 5, the p × 7 table;
  - `a3_rules(coef, seed, settings)`: `ci95` and `km2`;
  - `a3_kmeans2(absmu, seed, settings)`;
  - `a3_ess(fit, selected)`: step 7;
  - `a3_gates(...)`.
- Each A3 fit writes its `rng_after` to its RDS and CSV immediately; the runner holds no state across fits. `score.R` compares the non-failed `rng_after` values of the five A3 fits of each dataset and sets `crn_mismatch` (and `diag_flag`) on all of them if they differ. Failed fits have no `rng_after` and are left out of the comparison.
- Shared helpers from `common.R`: `assert_inputs`, `assert_pkg_version`, `with_captured_conditions`, `support_key`, `make_rule_rows`, `fail_rows`, `elapsed`.
- Tests: `code/tests/test_A3_horseshoe.R` with `run_A3_tests()` (items i.1–i.10).

(l) Open items.

1. O10 (MCMC budget): fixed at G6 from diagnostics only; pre-declared rule in Appendix A, O10.
2. Runtime at p ≥ 500 (EST from a numpy timing); G6.
3. The R 4.5.2 RNG routines are assumed identical to the trunk source read here (UNVERIFIED). The CRN unit test (i.6) and the per-dataset invariant are empirical terminal-state

checks. To close the item, read `src/nmath/{rgamma,sexp,snorm}.c` of the R 4.5.2 release tarball at G1.

4. Kollipara et al.'s 2-means implementation (UNVERIFIED; ours is labelled ours).
5. The S2 `horseshoe` 0.2.0 arguments (UNVERIFIED; read before use).

III.3.9 No-oracle audit (acceptance item)

Each method receives only (X, y_c, foldid, seed, settings). The settings are constants in `config.R`, frozen before the pilot, and contain no truth-derived numbers.

Method	Inputs	Data-dependent tuning	Caps
F1	X, y, folds	CV on common folds; EBIC	EBIC cap 100
F2	X, y, folds	CV over (α, λ); EBIC as S0	EBIC candidates ≤ 100
F3	X, y, folds	CV ridge κ, CV λ; EBIC as S0	EBIC candidates ≤ 100
F4	X, y (folds only for the unrun S2)	EBIC on the path; BIC and EBIC(0.5) as S0	EBIC candidates ≤ 100
F5	X, y (internal seed 1, fixed)	EBIC	sizes 0..100
A1	X, y, seed (credible-set purity subsampling only)	empirical Bayes	L = 50 (S1: 100)
A2	X, y	adaptive $\theta \sim \text{Beta}(1, p)$; σ^2 estimated (S1: Ray–Szabó ladder to p)	none
A3	X, y, seed	MCMC	none

Checks:

- `truth/` is mode 000 during fitting waves (§II.10).
- A unit test runs every wrapper with `truth/` absent.
- A code grep of the fitting sources (`code/R/config.R`, `common.R`, `methods/*.R`) for the truth-bearing identifiers `truth`, `\$active`, `\$signal` and the path fragment `truth/` must find nothing (the noise vector exists only in the `truth/` files; a bare `eps` is not searched because solver tolerances use that name); wrappers must not use these identifiers. Coefficient vectors may be named `beta` or `b` (the interface itself returns `beta`, and KKT code speaks of active coordinates). The runner never passes the truth list to a fitting environment (Oct-14 `validate.R` used the pattern `\$truth|\$active|seed_data|scenario|replicate`).

III.4 Considered but not selected

The earlier candidate panel had 13 selection methods (F1–F7 + F6b, A1–A5) plus 4 samplers (B1–B4), at 9,000 cases each in L + H (the design before D1; the panel now runs 6,000 cases each, in H only). Trimming to a core set was the obvious lever; the final panel of eight (§III.1) was fixed by Felipe. Everything below was considered and is not in the panel. One line each; settings, quotes and sources are in `evidence/FREQUENTIST.md` and `evidence/BAYESIAN.md`. Items are not run unless the panel is extended.

III.4.1 Earlier candidate panel members not selected

Candidate	Planned setting (earlier list)	Reason not selected / design point
F6 Knockoffs	L: fixed-X, `knockoff` 0.3.6. H: model-X with Σ estimated from X, MVR construction, `knockpy` 1.3.5. Knockoff+ at q = 0.2 primary; q = 0.1 and offset 0 also reported.	Not in the final eight. Design points: knockoff+ cannot select fewer than $\lceil 1/q \rceil$ variables (10 at q = 0.1, 5 at q = 0.2), which is restrictive at $s_0 = 5$. Runtime of MVR knockoffs at p = 1000 is not measured. `evidence/FREQUENTIST.md` §1 slot 6.
F6b BH on OLS p-values	Regime L only; base R; q ∈ {.1, .2}	Not in the final eight; needs the OLS fit, which does not exist for p > n. `evidence/FREQUENTIST.md` §1 slot 6b.
F7 Stability selection (complementary pairs)	`stabs` 0.7-1; cutoff 0.75, PFER = 1	Not in the final eight. Design point: the PFER bound fixes the per-subsample model size, so the pilot would have to check that it allows at least 5 in L and at least 14 in H. `evidence/FREQUENTIST.md` §1 slot 7.
A4 BAS, hyper-g BMA	`BAS` 2.0.2, `method = "MCMC"`, HPM; in H the model prior is BB(1,1) truncated at n − 2	Not in the final eight. Runtime at p = 1000 not measured. `evidence/BAYESIAN.md` §1 A4.
A5 Nonlocal prior (pMOM)	`modelSelection` 1.0.7 (replaces the archived `mombf`), HPM	Not in the final eight. Runtime at p = 1000 not measured. `evidence/BAYESIAN.md` §1 A5.
B1 PARNI	Computational panel: samplers targeting the SMC's own posterior with the project's score l(S)	Not in the final eight. The authors' code hard-codes its own prior and centres y, so it must be re-implemented. `evidence/BAYESIAN.md` §2.
B2 Weighted tempered Gibbs (wTGS)	as B1	Not in the final eight. The official code hard-codes a g-prior with Bernoulli inclusion; re-implementation needed.
B3 Schäfer–Chopin binary SMC	as B1; L primary, H as a stress run	Not in the final eight. The Python `particles` package accepts a custom score and needs a BB(1,p) prior object; the only competitor that returns an evidence estimate.
B4 Add-delete-swap MCMC (long run)	as B1	Not in the final eight. Simple to re-implement.

For B1–B4, accuracy would be scored against exact enumeration in Regime L; Regime H has no exact reference, so accuracy there could only be agreement across independent runs. Runtimes at p = 1000 were not measured for `bayesreg`, BAS, `modelSelection` and MVR knockoffs. If BAS, `modelSelection` or MVR knockoffs were added, one pilot dataset each would have to be timed on NOTS before fixing budgets; for the panel, only `bayesreg` is piloted (§IV.2.2).

III.4.2 Sensitivity variants not adopted

The earlier list of sensitivities was: relaxed lasso, SCAD, L0Learn, ABESS in L, default knockoff constructions, and SIS/ISIS-SCAD in H. SCAD is adopted (S1 of §III.3.4). ABESS in L was adopted as an S2 of §III.3.5 and is now moot under D1 (§III.4.5); the rest are not adopted.

Variant	Applies to	Reason
Relaxed lasso (`cv.glmnet(relax=TRUE)`)	F1	Listed as a sensitivity in the earlier list; not adopted in §III.3.
L0Learn (L0L2, CDPSI; `L0Learn` 2.1.0, CRAN 2023-03-07)	F5	Listed as a sensitivity; not adopted. It is not installed on NOTS.
ABESS internal GIC (`tune.type="gic"`, the package default)	F5, H	Listed as a sensitivity; not adopted (the S0 rules are EBIC(0.5) and BIC).
Default knockoff constructions (ASDP/equi); fixed-X knockoffs with MVR	knockoffs	Attached to F6, not selected.
SIS / ISIS + SCAD	H only	Not needed: at $p \leq 1000$ the panel methods run on all p directly.
Stability selection cutoff 0.9 / `sampling.type="MB"`	F7	Attached to F7, not selected.
BAS (a) `prior = "g-prior", alpha = sqrt(n)`, (b) `"EB-local"`, (c) `tr.beta.binomial(1, p, trunc = 100)`, (d) MPM rule	A4	Attached to A4, not selected.
pMOM (a) MPM, (b) `momprior(tau = 0.348)`, (c) `BayesS5` 1.41	A5	Attached to A5, not selected.
Variational spike-and-slab: `varbvs` 2.6-10, `sparsevb` 0.1.1 (PIP > 0.5)	extra	Run in Kim 2024 and Ray & Szabó 2022; not selected.
`modelSelection` near-match external check (Zellner prior, BB(1,p), igprior(0.02, 0.02))	B extra	Not selected; exact Zellner parametrisation UNVERIFIED.
LIT-MH (Zhou et al. 2022, JRSSB 84(5):1751–1784, doi:10.1111/rssb.12546), re-implemented on l(S)	B extra	Not selected; authors' code URL timed out (UNVERIFIED).

III.4.3 Considered and rejected (frequentist)

Candidate	Reason
Fixed-X knockoffs in Regime H	Infeasible: `create.fixed` requires $n > p$.
BH on OLS p-values in Regime H	The OLS fit does not exist for $p > n$.
Exact best subset (leaps/MIO) in Regime H	Computationally infeasible at $p \geq 500$. The `bestsubset` (Gurobi MIO) package is not on CRAN and needs a Gurobi licence. ABESS fills the slot.
Forward stepwise	Dominated by exact best subset at $p = 30$; in H the same $\ell 0$ paradigm as ABESS. Optional, not recommended.
Debiased/desparsified lasso + BH	`hdi` archived on CRAN 2026-07-10.
Multi-split p-values + BH	Uses `hdi::multi.split` (archived).
Post-selection inference (`selectiveInference`)	1.2.5, no release since 2019-09-07.

Candidate	Reason
Data splitting (DS/MDS), Gaussian mirror	No CRAN package verified; run as comparators mainly within their authors' group.
T-Rex selector (`TRexSelector` 1.0.0, CRAN 2024-02-23)	Run mainly in its authors' own papers; journal venue UNVERIFIED.
Scaled / square-root lasso, Dantzig selector	Same convex $\ell 1$ paradigm as F1; `scalreg` archived 2026-07-10; `flare` 1.8 maintained.
SLOPE (`SLOPE` 2.1.1, CRAN 2026-08-29)	Not a comparator in any of the 13 p > n papers opened.
Group lasso / group knockoffs / sparse group lasso	Need group labels, which are generating-Σ information (oracle).
Random forest / boosting importance	Nonlinear learners in a correctly specified linear model.
uniLasso (2025)	Prior review reports failure on opposite-sign correlated pairs; not re-verified (UNVERIFIED).

III.4.4 Considered and rejected (Bayesian)

Candidate	Reason
EMVS	Archived from CRAN 2022-12-05; superseded by SSL from the same authors.
Skinny Gibbs / BASAD	Simulations are logistic; no CRAN package found (`skinnybasad` 404).
SSVS via `BoomSpikeSlab` 1.2.7 (`lm.spike`)	Default prior needs an expected model size, which Kollipara set to the truth; adds little beyond model-space methods. Optional only.
Regularized horseshoe (rstanarm)	HMC took 1400–3400 s per fit at n = 200, p = 1000; slab scale needs a guess at p0. Horseshoe+ is the cheaper variant (a sensitivity of A3, §III.3.8).
Dirichlet–Laplace, R2D2	"become infeasible and are excluded" at (p, n) = (1000, 200) in Das 2026.
Bayesian lasso	Does not select variables without an added rule; slow under MCMC (1150–2000 s); a weak foil per the 2026-10-03 review.
Mr.ASH	Empirical-Bayes prediction method; posterior mean not sparse; `mr.ash.alpha` not on CRAN.
BLiP (Spector & Janson 2025)	Post-processing of posterior samples rather than a selector.
Projection predictive (projpred), DSS	Decision rules on a reference posterior, not separate models.
ASI (Griffin et al. 2021)	Superseded by PARNI from the same group.
TGS (unweighted)	Dominated by wTGS; reported unreliable on PCR data.
Hamming ball sampler (Titsias & Yau 2017)	"required 34 times the run time" of ADS in Griffin et al. 2021.

Candidate	Reason
MAdaSub	R code is paper source files, not a package (CRAN 404); tied to g-prior/EBIC scores.
Fine-mapping tools (FINEMAP, DAP-G, CARMA, SuSiE-inf)	Genetics or summary-statistic specific; SuSiE is the general-purpose member.
`millipede`	Built for P $\gg 10^4$; hard-codes its own prior.

III.4.5 Dropped by decision D1 (2026-10-19)

D1 restricts the benchmark comparison to p > n (Regime H). Every Regime-L benchmark setting of the earlier plan is therefore dropped. They are recorded here so that nothing is lost if Felipe reverses D1; none of them is run.

Dropped item	Earlier setting	Reason
F5 exact best subset in Regime L (`leaps` 3.2 + BIC)	See the specification below.	D1: no benchmarks at p = 30. F5 is ABESS + EBIC(γ=1) only (§III.3.5).
F5 S2: ABESS in Regime L (sizes 0..30) as a check against the exact optimum	S2 (optional)	Moot: depends on the dropped exact arm and on fits at p = 30.
F4 primary rule in Regime L	BIC ($\gamma = 0$), cap 30; S0 EBIC(0.5) and EBIC(1)	D1. In H the rules are unchanged (§III.3.4).
A1 SuSiE in Regime L	L = 10 (2026-10-07 project decision); S1 L = 30	D1. In H, L = 50 and S1 L = 100 (§III.3.6).
EBIC candidate cap in Regime L	$\lvert S\rvert \leq 30$ (the full model)	D1. The H cap of 100 is unchanged (§III.3.0).
SSL patch as a Regime-L fix	The patched build was "mandatory in Regime L"	No SSL fit at p < n remains; the build is kept (§III.3.7).
Optional descriptive join of benchmark supports with the exact-posterior HPM and MPM in Regime L	§IV.1.4 (earlier)	No benchmark supports exist at p = 30. Regime L now compares only the SMC arm's $\hat{\mathbf{Z}}$ and PIPs with exact enumeration (§II.8).
Regime-L benchmark compute	p = 30 row of §IV.2.3: about 45 s per case, about 38 core-hours for 3,000 cases (EST); per-task request 1 h / 2 GB; W1 wave of 600 tasks	D1; removed from the budget (§IV.2.3).

Specification of the dropped exact F5 arm (kept for the record; not run).

- **References.** Branch and bound: Furnival & Wilson 1974, Technometrics 16(4) 499-511, doi:10.1080/00401706.1974.10489231. Use: Zhu et al. 2020, "Low-Dimensional Case", Table 1: "all-subsets regression (ASR), which exhaustively searches ... via an efficient branch-and-bound algorithm ... We use SIC (ASR-SIC)". The Regime-L rule was BIC, as in `SMC_Composite_Likelihood_Methodology.tex` §Benchmarks.

- **Software.** `leaps` 3.2 (CRAN 2024-06-10). Arguments [V-NOTS]; the call was never run in this project's harness (UNVERIFIED in harness).

```
rs <- leaps::regsubsets(x=X, y=y, nbest=1, nvmax=30, intercept=FALSE,
                        method="exhaustive", really.big=TRUE)
```

- Take the best support of each size 1..30 from `summary(rs)$which` and add the null model.
- Refit each with zero-intercept OLS and check that its RSS matches `summary(rs)$rss` to a relative 1e-8.
- Score with the common BIC, $\gamma = 0$. Do not use `summary()$bic`, because its intercept convention is not relied on.
- BIC is monotone in RSS for fixed k, so the minimum over the 31 best-of-size supports is the global BIC optimum over all 2^{30} subsets.
- **S0 rules:** EBIC(0.5), EBIC(1).
- **Save:** RSS per size (package and refit), the chosen size, and the leaps runtime.
- **Caveat:** exhaustive branch-and-bound cost at p = 30 depends on the design and is **unmeasured** (`evidence/FREQUENTIST.md`). It was the Regime-L runtime risk (S4, S5); the planned mitigation was an exact best-of-size search by a different exact algorithm, verified against leaps on fast cases, never with a size cap below 30.
- An earlier project implementation exists at `02_Experiments/C_2026-10-07_benchmarks_p30/Experiment_ 2/Benchmarks/Exact_best_sub/` (found by search; not read for this plan).

Part IV. Execution plan

IV.1 Outputs and metrics

IV.1.1 Per (case, method) files

Files are written atomically (temporary file, then rename, as in Oct-14): `results/<run_id>/p{pppp}/<case_id>/<method>.rds` (full compact diagnostics), `.csv` (rule rows), `.sha256`, `.done`.

IDs.

- `dataset_id = sprintf("p%04d_S%d_r%03d", p, s, r)`.
- `case_id = paste0(dataset_id, "_", scale_key)`, with `scale_key` ∈ {`c100`, `c075`, `c050`, `c035`, `c025`}.
- Dev cases are prefixed `dev_`.

IV.1.2 Rule-row schema

The CSV has one row per (case, method, rule). It is the same schema the SMC arm must emit.

Column	Content
`study`, `run_id`, `source_sha256`	provenance
`regime`, `p`, `scenario`, `replicate`, `scale_key`, `scale`, `dataset_id`, `case_id`	design
`data_seed`, `fold_seed`, `method_seed`	seeds
`inputs_sha256`, `x_sha256`, `y_sha256`, `fold_sha256`	input hashes, float64 little-endian
`method_id` (`F1`–`F5`, `A1`–`A3`; S1 ids `F4_SCAD`, `SuSiE-L100`, `SSL-RS`; reserved S2 ids `SuSiE-auto`, `SuSiE-refine`, `SSL-wrapper`, `HS+`, `HS-pkg`), `method_label`, `package`, `package_version`	method
`rule_id` (closed list: `cv_min`, `cv_1se`, `ebic1`, `ebic05`, `bic`, `pip05`, `cslead`, `bfdr10`, `ssl_final`, `ci95`, `km2`; reserved, not run: `cv_min_nested` (F3 S2) and `cv_min` of F4 (S2)), `role` (primary / S0 / S1 / S2)	rule
`status`	ok / warning / failed / timeout / not_run
`converged` (the solver's own criterion; NA for F5 and A3), `diag_flag`, `diag_codes`, `n_warnings`, `warnings`, `messages`, `error`	diagnostics (§III.3.0)
`k`, `support` (semicolon-separated sorted indices; NA for a failed rule), `support_audit` (the support a failed rule returned, for audit only)	selection (written at fit time)
`tp`, `fp`, `fn`, `exact`, `error_type` (exact / extra_only / missed_only / mixed / failed)	**filled by the scoring step only**
`sel_a01` … `sel_a14` (NA beyond s_0), `pip_a01` … `pip_a14` (where PIPs exist), `max_pip_null`	per-active-variable selection and PIPs (scoring step)
`runtime_s` (whole fit including tuning; shared by rules from the same fit), `host`, `slurm_job_id`, `slurm_array_task_id`, `started_utc`, `finished_utc`	compute

Method-specific extra columns (for example `lambda`, `alpha`, `kappa`, `cv_mean`, `cv_se`, `rss`, `crit_value`, `k_valid_max`, `k_path_max`) are listed under item g of each method. PIPs are available for A1 (and SuSiE-L100) only, as full vectors in the RDS. A3 stores per-coefficient posterior summaries (mean, sd, three quantiles, ESS); A2 stores the final β and the per-$\lambda 0$ path table; F1–F5 store the selected-λ β.

Error-type definitions:

Type	Condition
exact	FP = FN = 0
extra_only	FP > 0, FN = 0
missed_only	FP = 0, FN > 0
mixed	FP > 0 and FN > 0
failed	status $\in$ {failed, timeout, not_run}

Scoring is a separate script, run after fitting. It reads `truth/` and fills the truth columns. Fitting code never computes these columns.

IV.1.3 Aggregate tables

Use the primary rule unless stated otherwise.

- **T1:** exact recovery x/100 per (p, scenario, scale, method). There is also a pooled x/600 per (p, scale, method). For F1–F3 the `ebic1` rule is shown as a pre-declared companion column beside the CV-min primary, because the primary criteria differ across the panel (§III.3.1 a).
- **T2:** failed or timeout count /100 per cell, shown beside T1.
- **T3:** mean FP and mean FN; error-type counts.
- **T4:** selection frequency of each active variable, grouped by $|\beta|$ (0.8, 1.0, 1.2, 1.4, 1.6, 1.8) and by S6 ρ partner.
- **T5:** runtime median, p95 and maximum per (p, method).
- **T6:** convergence and diagnostic-flag counts.
- **S-tables:** the same as T1 for every S0 and S1 rule, labelled as sensitivities.

Denominators. The denominator is always 100 per cell (600 pooled).

- Failed, timed-out and not-run fits count as non-recovery in T1 and are reported in T2.
- "Exact among successful" is a secondary column only.
- Diagnostic-flagged fits get two count versions, all-saved and passing-only, as in Oct-7. The denominator stays 100 in both.

Paired analyses. Paired win/loss counts per (case) between methods are allowed. Scale-pooled totals must state that scales are paired.

IV.1.4 Pairing interface for the SMC-BVS arm

- The SMC arm reads the same `inputs/` files, keyed by `dataset_id`, with Y columns by `scale_key`.

- It writes rule rows with the §IV.1.2 schema: `method_id="SMCBVS"` and `rule_id` ∈ `{hpm, pip05}`. `hpm` uses the all-scored v3 HPM with exact-equality ties; `pip05` uses {j : primary RB PIP_j > 0.5} (DP2 D2). Stored input hashes refer to the shared original arrays; the arm standardises both X and y internally and retains the preprocessing map (M §4). Native v3 statuses and flags are retained alongside the study interface; its adapter mapping remains an integration item (§I.7).
- It writes to its own `results/<run_id>/` tree.
- The join key is (`case_id`, `method_id`, `rule_id`). The join asserts equal `x_sha256` and `y_sha256` on every row.
- The join with benchmark rows exists in Regime H only (D1). In Regime L there are no benchmark rows: the SMC arm's log $\hat{Z}$ and PIPs are compared with the exact values from its enumeration of all 2^{30} models (§II.8). This external study enumeration is separate from v3's automatic small-space exact branch: at n = 200, p = 30 the deployed fit takes the SMC branch (M §6.2). Compare the same standardised working target and primary RB PIPs. The study storage format for the paired exact/estimated accuracy results remains unspecified; v3's fit outputs are specified in Part I. (The earlier optional join of benchmark supports with the exact-posterior HPM and MPM is dropped; §III.4.5.)

IV.2 Compute plan on NOTS

IV.2.1 Job structure

Benchmark arrays (Regime H only, D1). Unit = one dataset (p, s, r) with p ∈ {500, 1000}. One array task loads one `inputs/` file and runs, for each of the 5 scales:

- the 8 primary methods;
- the Tier-S1 sensitivities: SCAD; SuSiE L = 100; SSL Ray–Szabó ladder.

That is 40 primary (case, method) fits plus 15 S1 fits per task.

- **Arrays:** one array per p (p = 500 and p = 1000), `--array=1-600%C`. Task t maps to s = ⌈t/100⌉ and r = (t−1) mod 100 + 1.
- **Size:** 1,200 benchmark tasks (1,200 datasets), under the per-user `MaxSubmitPU=5000` and `MaxArraySize=100001` (checked). With the 600 Regime-L tasks below, the study has 1,800 tasks in total, still under the limit.
- **Accounting:** 1,200 tasks × 5 scales × 8 methods = **48,000 primary fits = 6,000 cases × 8 methods** (2 p values × 6 scenarios × 5 scales × 100 = 6,000 cases per method). S1 adds 6,000 (SCAD) + 6,000 (SuSiE) + 6,000 (SSL) = 18,000 fits (derived here).

Regime L tasks (SMC arm, D1). One task per p = 30 dataset (600 tasks), each running SMC-BVS and exact enumeration of all 2^{30} models for each of the 5 scales: 3,000 SMC-BVS runs and 3,000 enumerations. No benchmark method runs in these tasks. Their study runner, resource requests, gates and aggregate compute budget belong to the SMC arm; the per-fit algorithm and particle/stage budgets are specified in Part I; they are listed here so the study's task count is complete.

- **Idempotent:** the runner skips any (case, method) that already has a valid `.done` and hash. The `--methods` and `--cases` filters let retries and an optional split of A3 into its own array run without code changes.

- **Partition:** `commons` (QoS `nots_commons`: max wall 1 day, 1,536 CPUs and 2,304 GB per user). Use 1 CPU per task.
- **Queue at check time:** 476 pending and 283 running jobs on commons, 0 idle nodes. Expect queue waits.

IV.2.2 Runtime pilot (gate G6) before any budget is fixed

Dev seeds only, excluded from counts:

- for each p ∈ {500, 1000}, all 6 scenarios × dev replicate 1 × 5 scales = 30 cases;
- 60 cases in total, with all 8 methods plus S1;
- 12 array tasks.

(Before D1 the pilot also covered p = 30: 90 cases and 18 tasks. The p = 30 dev datasets are still generated (§II.13, 90 dev rows) for the SMC arm's own development; no benchmark pilot runs on them.)

Measure the elapsed time and MaxRSS (`sacct`) of every (case, method). Then fix:

- `--time` = 3 × pilot p95 of the task total, rounded up;
- `--mem` = 2 × pilot maximum MaxRSS;
- the concurrency per wave.

Items flagged for the pilot:

- **bayesreg at p = 1000**: the dominant cost; unmeasured.
- SuSiE with L = 50 and 100 at p = 1000.
- the F2 loop of 55 fits at p = 1000.
- the horseshoe ESS diagnostics. These may trigger O10 **before** production. The decision may use only diagnostics, never recovery.
- A3 two-chain stability check: a second chain (seed + 1,000,000) on each of the 60 pilot cases; Jaccard of the two `ci95` supports (§III.3.8 i). About 3.8 extra core-hours (EST).
- A2: support agreement of the native route (uncentred y) with the centred-y route on the 60 pilot cases (the size of the O5 choice); first-ladder-point support size and sweeps (§III.3.7 i, j).
- A1: `niter`, saturation of L, `n_active`, `n_cs` and `pip_unreported_max` (§III.3.6 i).

Pre-declared frequentist actions (diagnostics only, fixed before G7):

- **Grid boundary:** if the smallest λ is chosen by `cv_min` in more than 5% of pilot cases for any of F1–F3, lower the grid ratio from 1e-4 to 1e-5 for F1–F4.
- **F3 κ boundary:** if κ_min is at a grid end in more than 5% of pilot cases, extend the κ grid by three decades at that end.
- **glmnet passes:** if any F1–F3 fit reaches `maxit` (non-zero `jerr`), raise `maxit` to 1e7 or, if runtime forbids, relax `thresh` to 1e-12 for F1–F3 together.
- (The exhaustive `leaps` timing at p = 30 is no longer a pilot item: the arm was dropped by D1, §III.4.5.)

IV.2.3 Estimates (EST; to be replaced by pilot measurements)

Anchors are measured single-thread NOTS medians:

- Oct-14, p = 500, n = 200: Lasso CV 0.78 s, MCP 1.5 s, ABESS 0.11 s per case.
- Oct-7, p = 30, n = 100: Lasso 1.3 s, EN 2.3 s, nested AL 1.5 s, ABESS 1.1 s, SSL 0.12 s, SuSiE 1.9 s.

Everything else is extrapolated:

- **Horseshoe:** about 6,000 Gibbs iterations with the Bhattacharya et al. sampler at $O(n^2p)$ per iteration. That is about 0.025 s per iteration at p = 500 and 0.05 s at p = 1000 (EST), giving 150 s and 300 s. The sampler choice is now confirmed from the source ($p/n \geq 2$). A numpy timing of the matrix work of one draw (2.4 ms at p = 500, 3.4 ms at p = 1000, one thread) plus bayesreg's R overhead suggests 5–15 ms per iteration at p = 1000, i.e. about 40–100 s per fit (EST, §III.3.8 j). That basis uses another machine, BLAS and language and omits package overhead, so it does not show the budget is conservative; 150 s and 300 s stay as provisional planning values until G6.
- **SuSiE:** $L \cdot n \cdot p$ per sweep.
- **EN:** about 7× Lasso.

Primary, seconds per case (EST)	**F1**	**F2**	**F3**	**F4**	**F5**	**A1**	**A2**	**A3**	**Sum**	**× 3,000 cases**
p = 500	1–2	8–15	2–3	2	0.2	15	5	150	≈ 183 s	≈ 153 h
p = 1000	2–4	16–30	3–5	4	0.5	30	10	300	≈ 367 s	≈ 306 h

F1–F3 now use `thresh = 1e-16` (Oct-14 F1 used 1e-9) and full-path KKT checks; the ranges allow for both (basis in item j of each method). The sums use the lower ends. The upper ends add about 9 s per case at p = 500 and 17 s at p = 1000, about 22 core-hours over the 6,000 cases (derived here), well inside the 2× contingency.

The p = 30 row of the earlier plan (≈ 45 s per case, ≈ 38 h, with `leaps` at 10 s unmeasured) is removed by D1 (§III.4.5).

Item	**Core-hours (EST)**
Primary (48,000 fits)	≈ 460 (153 + 306; derived here)
Tier S1 (SCAD ≈ MCP; SuSiE ≈ 2× A1; SSL wide ≈ 2× A2), H only	≈ 105 (SCAD ≈ 5, SuSiE ≈ 75, SSL ≈ 25; derived here with the same rules as before, which gave ≈ 110 with p = 30)
Validation, single-seed tests, pilot	≈ 20 (kept from the earlier plan; the pilot is now smaller)
Total	≈ **585** (derived here; was ≈ 630 with Regime-L benchmarks)
Budget ceiling with 2× contingency	≈ **1,170 core-hours** (derived here: 2 × 585; was 1,250)

Regime L (SMC arm; not in this budget). SMC-BVS v3 at p = 30: study runtime not estimated. External exact enumeration: not estimated. Its size is $2^{30} \approx 1.07 \times 10^9$ models per case, about 3.2×10^{12} model scores over the 3,000 cases if each case is enumerated separately

(derived here). The SMC arm's Regime-H cost is also outside this benchmark budget. The former round-1 B = 5 timings are historical and do not estimate v3 with B = 14, candidate-grid rebuilds and possible tier-1 escalation; use the v3 resource preflight (M §8.6, Part I).

Assumptions:

- one core per task;
- no reruns other than infrastructure retries;
- the horseshoe at 5,000 + 1,000 draws.

If the pilot shows the horseshoe above 900 s per case at p = 1000, re-plan instead of extending silently.

Per-task requests until the pilot (EST):

p	Time	Memory
500	2 h	4 GB
1000	4 h	4 GB

(The p = 30 benchmark request, 1 h / 2 GB, is removed by D1. Requests for the Regime-L SMC and enumeration tasks are not estimated.)

The largest objects are 40 MB of horseshoe draws in memory, 8 MB for X'X at p = 1000, and EN CV prediction arrays.

Wall clock. At 200 concurrent tasks, about 3–6 h of compute per wave if nodes are free. Queue waits dominate.

Output volume (EST). About 50 KB per (case, method) on average. By method (before compression): F1 50–90 KB, F2 150–300 KB, F3 70–110 KB, F4 30–60 KB, F5 40–80 KB, A1 30–60 KB, A2 about 50 KB, A3 about 140 KB (item g of each method); the RDS files are written with `compress = "xz"`. For 66,000 benchmark fits (48,000 primary + 18,000 S1) that is about 3.3 GB and about 260k small files on `$STUDY` (derived here at 4 files per (case, method); the earlier plan's 96,000 fits gave about 5 GB and about 400k files). Regime-L SMC and enumeration outputs are not estimated.

IV.2.4 Waves, monitoring and retries

- **Waves.**
  - W1: p = 500 (600 tasks).
  - Integrity audit, then W2: p = 1000 (600 tasks).
  - (The earlier W1 at p = 30 is removed by D1; the Regime-L SMC and enumeration tasks are scheduled by the SMC arm.)
  - Concurrency starts at 200 tasks (`%200`) and is adjusted from the measured failure rate of the previous wave.
  - Each wave needs the previous wave's audit to pass (§IV.3, G8).
- **Monitoring.**
  - Adapt the Oct-14 `cluster/monitor.py`: counts of `.done`, failed and missing files per (p, method); `sacct` states; runtime drift against the pilot.
  - A submission-intent file (Oct-14 `submit.py open('x')` pattern) prevents duplicate arrays.

  - Every `.started`, `.finished` and `.exit` file goes under `status/`.

- **Retry policy.**
  - **Infrastructure failures** (node failure, preemption, OOM, timeout, or a missing output without a method error) get **one** automatic resubmission of only the affected (case, method) pairs. It uses the same frozen snapshot, the same seeds and `--methods/--cases`. For OOM or timeout, memory or time is doubled. A second failure is final, and the pair stays in the denominator.
  - **Method failures** (a package error, a non-convergence gate, a failed consistency check) are **not retried**. They are counted and reported.
  - If a failure exposes a bug in *our* code, fix it under a new snapshot and rerun that method for **all** cases of the affected p, so one method × p never mixes code versions. Log this in FAILED_ATTEMPTS.md and RUNS.md.

IV.3 Gates and verification

Gate	What	Who	Acceptance
G0	Felipe confirms or overrides O1–O3 (others default)	Felipe / lead	written answer
GA	Fable review of this plan (policy Gate A)	lead	PLAN_REVIEW.md saved; accept/ reject log
G1	Study library built and sealed (`$STUDY/lib/ R452/`). Versions as in §III.3.0, from saved CRAN source tarballs with SHA-256. SSLASSO from the Oct-7 vendor tarball. bayesreg 1.3 + pgdraw + doParallel installed.	lead runs the install; a Sonnet agent writes `preflight.R` (Oct-14 pattern)	preflight prints exact versions; per-file package hashes in `environment_ manifest.json`; the library is built from the tarballs with the SHA-256 values in `evidence/FREQUENTIST.md` and `evidence/BAYESIAN.md` (glmnet and abess are byte-identical to the Oct-7 vendor copies); bayesreg standardisation (unit length), `beta` orientation (p × n.samples) and sampler switch (Bhattacharya et al. when p/n ≥ 2) were read from the source on 2026-10-19 (§III.3.8), and G1 checks that the installed `bayesreg` body contains `p/n >= 2` and `bayesreg.standardise`, prints its formals, and records `parallel::detectCores()`; the R 4.5.2 sources `src/nmath/ {rgamma,sexp,snorm}.c` are compared with the trunk routines used in §III.3.8 e
G2	Data generated and verified (§II.13)	an Opus agent writes the generator and verifier; lead runs them on NOTS	all six checks pass. In particular 600/600 identical X and ε hashes for p = 500 against Oct-14, and truth indices equal `scenario_check.out`
G3	Method unit tests on NOTS (one Slurm job)	wrapper authors write the tests; lead runs them	all pass; see below

Gate	What	Who	Acceptance
G4	**Single-seed NOTS test** before any multi-seed job: one dev dataset per benchmark p, run as two separate one-dataset jobs (p = 500, 1000; S4, dev r1). No benchmark job at p = 30 (D1).	lead	40 primary + S1 (case, method) outputs per job; schema validates; runtimes recorded; scoring script runs; `parallel::detectCores()` on the compute node recorded and at least 2 (bayesreg sets its core count to min(detectCores() − 1, n.cores))
G5	**Independent verification of core code** (CODE_LEDGER)	one Opus agent who did not write the code, using different methods	ledger status set to `verified` for every unit in §IV.3.2; results stay labelled provisional until then
G6	Pilot (60 dev cases, p = 500 and 1000)	lead launches (multi-seed)	no infrastructure failures; budgets fixed (§IV.2.2); diagnostics reviewed; O10 settled
G7	Freeze, then production go-ahead	lead freezes; **Felipe gives the go-ahead** (NOTS deploy-gate practice)	`runs/<ts>_bench20261019_prod/code/` read-only snapshot + `manifest.json`; RUNS.md entry; submit script refuses a hash mismatch
G8	Per-wave integrity audit (two benchmark waves, p = 500 and 1000)	a Sonnet agent with an independent count script	600 tasks; 5 × 11 `.done` per task (8 primary + 3 S1 method ids); reconciliation rows written for missing outputs (§III.3.0); hashes verify; `x_sha256` matches the manifest; failure rate reported
G9	Final aggregation and **independent recount**	aggregator (Sonnet) and recount by a different agent	T1–T6 recount identical; provisional label removed only after G5

Unit tests (G3), one set per method, on small known cases.

1. **Orthogonal design canary** (n = 200, p = 30, X'X = (n−1)I, strong signal on 5 variables; a unit-test design, not a Regime-L run, so it is unaffected by D1). Lasso, EN and MCP must match their closed-form soft-thresholding or firm-thresholding solutions. Every method must recover the 5 exactly.
2. **Null canary** ($\beta = 0$). The primary rules return small supports. Any rule returning more than 30% of p is flagged for review.
3. **No-intercept canary** (y with mean 3 and X centred). This exposes hidden intercepts.
   - (a) Full-data fits on a fixed grid must be identical to fits on $y - \bar{y}$: F1–F3 coefficient paths at the same λ grid (the grid depends only on X'y, which is unchanged), F4 paths (ncvreg centres y internally), F5 supports and slopes (the wrapper centres y), and the A2 native route with `variance = "fixed"` (with estimated variance σ^2 absorbs $n\bar{y}^2$).
   - (b) Every per-support RSS must satisfy $\mathrm{RSS}(y) = \mathrm{RSS}(y - \bar{y}) + n\bar{y}^2$.
   - (c) CV curves and CV choices (F1–F3) are **not** expected to be identical, because training folds are not centred; EBIC choices are **not** expected to be identical, because $n\bar{y}^2$ enters inside log(RSS/n). These differences are reported, not treated as failures.

- A1: with fixed V and σ^2 the single-effect fits on y and y + 3 are identical; with σ^2 estimated the intercept stays 0. A3 always fits an intercept: the same seed on y and y + 5 gives slope posterior means and quantiles equal to 1e-6.

4. **Method-specific checks:**
   - F2 against the generic `family=gaussian()` backend (Oct-7 test) at p = 500;
   - F3 closed-form ridge against the normal equations;
   - F4 KKT on the original scale;
   - F5 ABESS consistency checks (the `leaps` RSS check is dropped with the exact arm, D1);
   - A1 single-effect closed form (Oct-7 test);
   - A2 patched against unpatched at p = 500 (identical); the native call on the wrapper's internal inputs (`scale(X, center = TRUE, scale = FALSE)`, `y - mean(y)`) identical to the wrapper on (X, y) after `unname`; the native call on (stored X, centred y) with the same final support as the wrapper and β within 1e-8 on a frozen dev fixture;
   - A3 Bhattacharya and Rue kernels against the exact Gaussian moments (200,000 draws, calibrated acceptance), shift and scale canaries, a strong-signal p > n canary, the CRN invariant, and dimension assertions (§III.3.8 i). The earlier ridge-prior closed-form test is dropped: `prior = "rr"` has a half-Cauchy τ, so no closed form exists.
5. **Determinism.** Two runs on one case give bit-identical supports; A3 is identical with the same seed. A runner test runs one case in the order F5 → A3 and A3 alone; the A3 outputs must be bit-identical (§III.3.0, RNG).
6. **Failure path.** Forced errors produce `status=failed` rows, never null-model successes.
7. **No-oracle test** (§III.3.9).

Production canaries. Each wave re-runs 3 pilot dev cases inside the production array. They must reproduce the pilot outputs bit-identically.

IV.3.1 Run records (policy §8)

- Before G4, G6 and each production wave, freeze the exact code into `runs/<YYYYMMDDTHHMMZ>_bench20261019_<piece>/code/` (read-only). This happens locally under `02_Experiments/F_2026-10-19_benchmark_study_n200/Benchmark_Study_20261019/runs/` and is mirrored to `$STUDY/runs/`.
- `manifest.json` records:
  - code SHA-256 (a per-file list plus a tree hash);
  - the exact `sbatch` command;
  - host, job ID and array ID;
  - seed ranges;
  - module list, R version, the package-manifest hash and BLAS;
  - input paths with manifest checksums;
  - output paths.
- On completion, add status, runtime and output checksums to both `manifest.json` and `RUNS.md`.

- `RUNS.md` lives in `02_Experiments/F_2026-10-19_benchmark_study_n200/Benchmark_Study_20261019/`, with one row per launch: run id, piece, job IDs, status and outputs.

IV.3.2 CODE_LEDGER.md entries (project root), all "AI, unreviewed" until G5

Unit	Implements
`code/gen_data.R`, `code/truth.R`, `code/verify_data.R`	generator, truth, sealing
`code/R/config.R`	all constants
`code/R/common.R`	input assertions, OLS refit, EBIC/BIC scorer, support keys, rule-row builder, condition capture, IO, timing
`code/R/methods/glmnet_helpers.R`	CV engine, KKT and path checks shared by F1–F3
`code/R/methods/fit_{F1_lasso,F2_enet,F3_alasso,F4_mcp,F5_abess,A1_susie,A2_ssl,A3_horseshoe}.R`	method wrappers (F4 file holds MCP and SCAD)
`code/tests/test_common.R`, `code/tests/test_<method>.R`	G3 unit tests
`code/R/run_case.R`, `code/slurm/`	runner, I/O, timing; job scripts
`code/score.R`, `code/aggregate.R`	truth scoring and tables

Ported units keep a pointer to their Oct-7 or Oct-14 origin and the differences.
The G5 verifier independently:

- recomputes EBIC choices from saved candidate tables;
- recomputes the SuSiE PIP rule and the horseshoe interval rule from saved summaries;
- re-runs KKT checks;
- re-derives truth placement in its own code;
- recounts T1 for the pilot.

IV.4 Timeline and checklist (EST durations)

#	Step	Owner (tier)	Machine	Acceptance
1	G0 decisions; GA plan review	lead, Fable (1 call)	local	answers logged; PLAN_REVIEW.md
2	Seed and case-ID table (9,000 production + 90 dev rows, all p, since the SMC arm also uses the p = 30 cases; 6,000 production rows are benchmark cases; seeds, fold seeds, MCMC seeds) with 5 canary rows checked against §II.7	**Haiku**	local	canaries match; row count; no duplicate seeds

#	Step	Owner (tier)	Machine	Acceptance
3	Library preflight scripts; lead installs and seals the library (G1)	Sonnet writes, lead runs	NOTS	§IV.3 G1
4	Generator, truth and verifier (§II.9–§II.13)	Opus A	local single-seed, then NOTS by lead	G2
5a	F1, F4, F5 wrappers (port Oct-14 `methods.R`, add SCAD; no `leaps` after D1) + tests	Sonnet B	local single-seed	tests pass locally on 1 dev dataset
5b	F2, F3 wrappers (port Oct-7 EN/AL with changes) + tests	Opus C	local	as 5a
5c	A1, A2 wrappers (port Oct-7 SuSiE/SSL; L = 50; native SSL) + tests	Sonnet D	local	as 5a
5d	A3 wrapper (new) + tests	Opus E	local; bayesreg must be installed locally for development, otherwise single-seed development on Owls	as 5a
5e	Runner, Slurm script, submit/monitor (port Oct-14 `cluster/`), scoring and aggregation	Sonnet F	local	dry run on 1 dataset; schema validation
6	G3 unit tests and G4 single-seed tests on NOTS	lead	NOTS	§IV.3
7	G5 independent verification	Opus G (did not write any of 4–5e)	local + NOTS read-only	ledger updated
8	G6 pilot (60 dev cases, p = 500 and 1000); budgets fixed	lead	NOTS	§IV.2.2
9	G7 freeze and Felipe go-ahead	lead	—	snapshot, manifest, RUNS.md
10	W1 (p = 500) → G8 → W2 (p = 1000) → G8	lead launches; Sonnet audits	NOTS	§IV.3 G8
11	Scoring, T1–T6, independent recount (G9)	Sonnet H aggregates; Sonnet I recounts	NOTS → local	identical recount
12	Archive outputs locally with hash verification; final report	lead	local	SHA-256 match

- Steps 5a–5e are independent and can run in parallel, each on its own paths.
- Elapsed time (EST): steps 1–7 take about 2–3 working days; steps 8–9 take half a day; step 10 takes 1–2 days, dominated by the queue; steps 11–12 take 1 day. (These EST durations were set before D1 and are kept; dropping the p = 30 wave can only shorten step 10.)

- The Regime-L work under D1 (the SMC arm's v3 runs and the external exact enumeration of all 2^{30} models) is not in this timeline; it belongs to the SMC arm (Part I).

IV.5 Risks and mitigations

Risk	Effect	Mitigation
BB(1,p) prior in the SMC arm with 14 signals ($P(\|S\| \geq 14) \approx$ 6e-5; `evidence/BAYESIAN.md` §3.6)	Does not affect the benchmarks. It does bias the **SMC versus benchmark** comparison in H at weak scales towards SMC false negatives.	Report the comparison by c and by coefficient group (T4). Felipe's v3 decision is fixed: BB(1,p) only; no secondary prior arm (M requirements, §5).
bayesreg cost at p = 1000	It could dominate compute (EST 300 s per case; the literature reports a range up to hours for HS+)	Pilot first. Split A3 into its own array if needed. If it exceeds 900 s per case, re-plan. Never shorten the chains after seeing recovery.
v3 block start: B = 14 at n = 200, p = 500 or 1000	Each block requires 2^{size} table scores; actual total is the sum over packed blocks. Tables update HPM and candidate leaders but do not enter the score cache (M §6.3; CORE R2).	Resource preflight includes tables, grid rebuilds and escalation (M §8.6). Record and verify the SMC implementation separately; benchmark settings are unchanged.
Exact enumeration of 2^{30} models per Regime-L case (D1)	Cost not estimated (about 3.2 $\times$ 10^{12} model scores over 3,000 cases if enumerated separately; derived here)	Belongs to the SMC arm; needs a cost estimate before any Regime-L run. Whether all 3,000 cases are needed is O12.
CV with p > n (F1–F3)	λ_min at the grid boundary; near-saturated paths; KKT drift at small λ	Full grid (`fdev=0, devmax=1`); boundary flags; KKT gates; λ_1se and EBIC reported as S0
glmnet response normalisation (F2, ridge in F3)	Wrong objective if `cv.glmnet` is used for $\alpha < 1$	Oct-7 parameter map and closed-form ridge (§III.3.2–§III.3.3); tested against the generic backend
SSLASSO upstream bug (p < n)	Wrong SSL fits when p < n. After D1 no production fit has p < n; the p = 30 G3 canary does.	Patched 1.2.3.9001 build; a version assertion in the wrapper
ABESS `screening.num` bug	All-zero paths	`screening.num=NULL` plus the consistency checks
SuSiE cap binding (L = 50 saturated)	Recovery capped artificially	Flag all-L-active; S1 with L = 100
Oct-14 p = 500 inputs on scratch are purged before the G2 check	Loss of the pairing verification	Run the G2 p = 500 check first, or copy the 600 files to `$STUDY/oct14_p500_reference/` (read-only) as the first action
BLAS or `chol` non-determinism across nodes	Datasets not bit-identical	Same module stack; the G2 regeneration check on a different node; X hashes in the manifest checked in every task

Risk	Effect	Mitigation
Home quota (1.3 GB headroom)	Job failures from a full home	Nothing large in home. Logs, results and library go under `$STUDY`.
Busy queue (0 idle nodes at check time)	Long wall clock	Waves; moderate concurrency; no deadline-driven setting changes
Unverified group quota on `/projects/ml75`	Writes fail mid-wave	Check `quota/lfs` before G6; the benchmark output volume estimate is about 3.3 GB (§IV.2.3)
Horseshoe interval rule conservatism, SSL path cycling	Low recovery that reflects the rule, not the prior	Pre-declared S0 rules (2-means; CS-lead); flags reported, never used to reselect
bayesreg drops the first model-matrix column unconditionally	A formula without intercept (`y ~ . - 1`) would silently drop predictor 1 while still fitting an intercept	Use `y ~ .`; assert `fit$p == p` (§III.3.8 c)
SSLASSO leaves θ, σ as NaN and the loss uninitialised at capped ladder points	A whole-path finiteness check (as in Oct-7) would fail fits whose final point is valid; using the native loss would read garbage	Gates only on the final point; RSS recomputed in R; intermediate gaps stored as NA (§III.3.7 e, h)
abess re-seeds the session RNG from the clock after every fit	Any later random step in the same R process (A3 MCMC) would be irreproducible	The runner reseeds before every method; runner test F5 → A3 (§III.3.0, §IV.3 test 5)
glmnet fold fits with truncated paths still give finite CV predictions	A silently wrong CV curve if `cv.glmnet` were used	Shared CV engine checks every fold fit (§III.3.1 e)

Numbers behind the first row. These come from the benchmark summary and were recomputed by the lead:

- BB(1,p) puts the prior mean model size at about 1 at every p.
- P($|S| \geq 14$) = 8.4e-6 at p = 30, 5.6e-5 at p = 500 and 5.8e-5 at p = 1000.
- With 14 true signals, weak scales will likely lose the weakest signals through the prior, not through computation.
- This is the same mechanism that withdrew BB+HPM in the Oct-13 weak-signal phase.
- Fixed for v3 deployment: retain BB(1,p), with no Bernoulli(h) or second BB(a,b) arm (Felipe's requirements; M §5). The comparison caveat above remains.

IV.6 UNVERIFIED items (summary)

1. **bayesreg:** resolved from the CRAN 1.3 source (§III.3.8): standardisation to unit length (cannot be neutralised; implies a $C^+(0, 1/\sqrt{(n-1)})$ global scale on our X), `fit$beta` is p × n.samples, Bhattacharya et al. when $p/n \geq 2$ (both p at n = 200), a single chain on R's global RNG with `n.cores = 1`, intercept always fitted. Still open: the package is not installed on NOTS, so its formals are not [V-NOTS] until G1.
2. **SSL native route at p > n:** the code path is the residual branch in both builds at p ≥ 500 (source); the call reproduces the wrapper's initialisation and argument order (source; G3 test).

Still open: the size of the difference between uncentred and centred y on the final supports (pilot, O5).

3. **F2 response normalisation:** now read from the glmnet 5.1 source [V-src `standardize.hpp` l.40–47, `gaussian.hpp` l.430–451]. Still to run: the generic-backend comparison at $p > n$ (G3).
4. All runtimes except the Oct-14 and Oct-7 anchors, including SuSiE with $L = 50/100$ at $p = 1000$ and the horseshoe at $p \geq 500$. (`leaps` at $p = 30$ is no longer needed: dropped by D1.)
5. The group quota on `/projects/ml75`.
6. Collision checks for the proposed new seed bases: MCMC seeds and dev fold seeds (O11).
7. Kollipara's exact 2-means implementation (our version is labelled as ours).
8. Moot under D1: `leaps::regsubsets` with `intercept=FALSE, nvmax=30, really.big=TRUE` was never run in this harness, and the exact F5 arm is no longer run (§III.4.5).
9. SMC arm: deployment validation and study runtime for v3, including $B = 14$ at $p = 500$ and 1000, candidate-grid rebuilds and tier-1 resources. Historical $B = 5/B = 10$ engine timings are not a v3 cost estimate. Source/prototype disagreements and unresolved deployment details are recorded in `deployment_20261022_v3/CHANGES.md`.
10. SMC arm (D1): the cost of exact enumeration of all 2^{30} models per Regime-L case, and the SMC runtime at $p = 30$.
11. The R 4.5.2 RNG routines (`rgamma`, `exp_rand`, inversion `norm_rand`) are assumed identical to the R trunk source read for the A3 common-random-numbers derivation; a unit test and a per-dataset invariant check terminal RNG states empirically; reading the R 4.5.2 sources at G1 closes the item.
12. The Moran, Ročková & George (2019) unknown-variance prior of A2 is taken from the Oct-7 audit, not re-read.
13. The S2 `horseshoe` 0.2.0 arguments (`method.tau`, `method.sigma`, intercept handling).
14. SuSiE `pip_unreported_max`: whether weak active effects lift correlated proxies above 0.5 (pilot).
15. Whether every F1–F3 fit passes KKT at `thresh = 1e-16` at $p > n$, and the runtime cost of that threshold (G3, G6).
16. ABESS splicing convergence cannot be observed (no iteration count is returned; `max.splicing.iter` is inert with `important.search = p`); wall time is the only guard (§III.3.5 h).

Appendix A. Open decisions

One consolidated list. Gate G0 (§IV.3) is the point at which Felipe confirms or overrides O1–O3; the others default. O12–O14 were created by D1 and D2 (2026-10-19) and concern the SMC arm. O12 remains the study-design question; O13 and O14 are superseded by the fixed v3 deployment rules, with their historical purpose retained below.

Made moot by D1 (no longer decisions): the Regime-L benchmark settings, namely F5 exact `leaps` + BIC, F4 BIC in L, SuSiE L = 10 and its S1 L = 30, the EBIC cap of 30, ABESS in L as an S2, and the optional benchmark-versus-exact-posterior join (§III.4.5). None of them was an open decision O1–O11; O2 and O9 below are updated for the new counts. The A.3 question "whether the SMC arm enumerates at p = 30" is settled by D1 (yes, all 2^{30} models).

A.1 Study-design and execution decisions (O1–O14)

Open decisions (the default is applied unless Felipe overrides it):

#	Decision	Default used in this plan	Alternative	Cost of changing later
O1	Signals in Regime H	14 = $\lfloor\sqrt{200}\rfloor$	15 (round up)	Data regeneration (cheap). All H fits rerun.
O2	Signal scales	**Decided (D3): c = 1 only for the benchmark production run**; data keep 5 scales	c = 1 only (1,200 benchmark cases per method after D1; 1,800 before)	Cutting scales later only drops cases. Adding them later needs new runs.
O3	Replicates per cell	100	fewer	—
O4	Data and output storage	`/projects/ml75/fb31/BVS/bench20261019` (§II.12)	`/scratch/fb31/...` (14-day purge)	Moving later is cheap if done before any purge.
O5	SSL call route	Native `.Call("SSL_gaussian", …)` with uncentred y, patched build 1.2.3.9001 (as Oct-7)	Public `SSLASSO()` wrapper, which centres y (an implicit intercept, which `evidence/BAYESIAN.md` would accept). The pilot reports how often the two routes give different final supports (§III.3.7 i).	All A2 fits rerun.
O6	SuSiE `refine`	`FALSE`, the package default as in `evidence/BAYESIAN.md`. `TRUE` is a sensitivity.	`TRUE`, as frozen in the Oct-7 campaign	All A1 fits rerun.
O7	Adaptive-lasso weights	Full-data ridge pilot, then CV over λ (`evidence/FREQUENTIST.md` slot 3)	Leakage-free nested CV (Oct-7 `AdaptiveLasso/fit.R`)	All F3 fits rerun.

#	Decision	Default used in this plan	Alternative	Cost of changing later
O8	Elastic-net α grid	{.1, .25, .5, .75, .9} (§III.1)	Oct-7 grid {.1, .3, .5, .7, .9}	All F2 fits rerun.
O9	Separately fitted sensitivities run in production	Tier S1 only (§III.3.0), all in Regime H: SCAD; SuSiE L = 100; SSL wide ladder. (The Regime-L S1, SuSiE L = 30, is moot under D1.)	none, or add Tier S2 items	Each one is an independent add-on.

#	Decision	Default used in this plan	Alternative	Cost of changing later
O10	Horseshoe MCMC budget	1000 burn-in + 5000 kept, thin 1 (`evidence/BAYESIAN.md`)	Pre-declared rule (proposal, to be confirmed at G0/G6): if at either p more than 10% of the pilot A3 fits are ESS-flagged, or the median two-chain Jaccard of the `ci95` supports over the pilot cases at c = 1 (6 per p) is below 0.9 (weaker scales are excluded because borderline decisions there flip for Monte Carlo reasons a longer chain cannot remove), double the budget (2,000 burn-in + 10,000 kept, thin 1) and re-pilot A3 only; otherwise keep 1,000 + 5,000. The rule is applied separately at each p. $J(\emptyset, \emptyset) = 1$; a pair with a failed chain counts as $J = 0$. If the doubled-budget re-pilot still triggers the rule, stop and re-plan with Felipe. **Outcome:** the doubled budget still flagged 12/12; superseded by the O10 re-plan (10,000 + 15,000, thin 5; see the revision log). The 10% and 0.9 thresholds are project choices, not literature values. Fixed before production from diagnostics, never from recovery.	All A3 fits rerun.
O11	New seed ranges (MCMC seed, pilot fold seeds; §II.7)	Proposed in §II.7	—	Collision check against earlier campaigns is **pending**.

#	Decision	Default used in this plan	Alternative	Cost of changing later
O12	Regime-L design for the $\hat{Z}$ and PIP accuracy assessment (D1)	Keep the design unchanged: 6 scenarios × 5 scales × 100 replicates = 3,000 cases, each with an SMC-BVS run and an exact enumeration; p = 30 folds and MCMC seeds kept in the tables though unused (§II.6, §II.7)	Fewer scales (for example c = 1 only: 600 cases) and/or fewer replicates; stop generating p = 30 folds	Cutting later only drops cases; the enumeration cost (§IV.2.3, not estimated) scales with the number of cases.
O13	Neighbour-move settings (historical D2 question)	**Closed by v3:** B neighbour-swap steps, at most B directed partners before symmetric union, threshold $\sqrt{(2 \log p/n)}$, and p + B + 2 labels per random-order round (M §§6.4–6.5).	No deployment override	Any later change is a new method version.
O14	Block size for the historical round-1 regression test	**Superseded for deployment:** B = min($\lfloor\sqrt{n}\rfloor$, $\lfloor\sqrt{p}\rfloor$, 16); B = 14 at n = 200, p = 500 (M §3).	Historical reproduction, if separately commissioned, keeps its original version and labels.	No B override is part of the deployed method.

A.2 Decisions recorded with the scenario specification

1. **Truth size by p** (decided by Felipe): 5 signals at p = 30 and $\lfloor\sqrt{200}\rfloor$ = 14 at p = 500 and 1000. "√n" is read as the number of signals, rounded down. Rounding up would give 15 (this is O1 above).
2. **p = 500 keeps the Oct-14 seeds** (reason in §II.7).
3. **Five signal scales** (O2 above). At p=500 and c=1, the best methods are already at 81–99/100 per scenario: BB+HPM scores 93–99, and ABESS and MCP with EBIC γ=1 score 81–90 (`02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14_Mixture/comparative/comparison_2219840/recovery_c100_10methods_6scenarios.csv`). Those numbers are for the old 6-signal truth. With 14 signals, c=1 is a new test, and the weaker scales are what separate the leading methods. The alternative is c=1 only, which needs 1,800 cases per method instead of 9,000 (before D1); after D1 the benchmark counts are 1,200 instead of 6,000 per method, and the reason for the weaker scales applies to Regime H, where the benchmarks run. Whether Regime L still needs them is O12.
4. **100 replicates per cell** (O3 above). All multi-seed generation and fitting runs on NOTS.

A.3 Items that belong to the SMC-BVS arm and are not settled here

- The deployed rules are now Part I (METHODOLOGY_v3 + CORE rulings + DP2 D2/D4). The round-1 gaps are historical; they do not supply defaults. B, the neighbour move, and the single BB(1,p) prior arm are fixed.
- The former D4 and R16 source-level OPEN items are resolved by the lead ruling 2026-10-22, DP2_DECISIONS addendum (§I.2–§I.3, §I.9). CHANGES.md records the uncapped threshold-graph reading and the labelled heuristic certificate, plus the remaining prototype disagreements; the separate study-integration items below are unchanged.
- The implementation and aggregate cost of the separate study enumeration of all 2^{30} models, and the storage/summarisation of Regime-L accuracy errors, remain SMC study-integration items (§II.8, §IV.1.4).
- The study role mapping (`hpm` primary, `pip05` S0) and the adapter from native v3 statuses/flags to `converged` and `diag_flag` remain integration items (§I.7). They do not change the fit's HPM, primary RB PIP, evidence or diagnostics.

A.4 Unverified items to close before production

See §IV.6. Two of them are time-critical: the Oct-14 p = 500 inputs on scratch must be checked or copied before they are purged (§II.12, §IV.5), and the collision check for the new seed bases (O11) is pending.

Appendix B. Reconciled differences

Where the superseded documents (and the two evidence files they rest on) disagreed, the newer or more specific statement was kept. The list is complete for conflicts found; mere overlaps were merged into one version.

#	Topic	What the documents said	Kept in this master, and why
1	Truth size, status	The scenario specification calls 5 / 14 signals "decided by Felipe"; the plan lists "signals in Regime H" as open decision O1 (14 vs 15).	Both: 5 at p = 30 and 14 at p $\geq$ 500 are the working design (§II.3); the rounding reading (14 vs 15) stays open as O1.
2	Panel	The panel summary lists 13 selection methods plus 4 samplers, "proposed, awaiting Felipe's choice". The plan says the panel of eight is "fixed by Felipe".	The panel of eight (§III.1). The rest is §III.4.
3	SSL software and call	The evidence files give `SSLASSO` 1.2.3 and the public wrapper `SSLASSO(X, y, penalty = "adaptive", variance = "unknown")`. The plan uses the patched 1.2.3.9001 and the native `.Call` route (O5).	The plan: the upstream p < n branch drops observation 1, and the wrapper centres y (§III.3.7). The wrapper is the O5 alternative.
4	Elastic net call	The evidence files say `cv.glmnet` for each α. The plan uses an explicit 10-fold loop with the Oct-7 parameter map.	The plan: with `intercept=FALSE` and $\alpha < 1$, glmnet's native backend uses a fold-dependent $\ell 2$ weight (§III.3.2).
5	Adaptive-lasso ridge pilot	The evidence files say `cv.glmnet(alpha=0)`. The plan solves ridge in closed form from `svd(X)`.	The plan, for the same normalisation reason (§III.3.3).
6	Sensitivity labelling	The evidence files call EBIC(γ=0.5), `cv.ncvreg`, EN-EBIC, L = 100, `susie_auto`, `refine`, CS-lead, horseshoe+ and others "sensitivities". The plan sorts them into tiers S0 (free), S1 (budgeted), S2 (optional).	The plan's tiers (§III.3.0 and each method). Sensitivities attached to non-selected methods or not adopted are in §III.4.2.
7	SuSiE `refine`	The Oct-7 campaign froze `refine=TRUE`; `evidence/BAYESIAN.md` uses the package default `FALSE`.	`FALSE` as default, `TRUE` as sensitivity; O6 holds the alternative.
8	Adaptive-lasso weights	`evidence/FREQUENTIST.md`: full-data ridge pilot. Oct-7 code: leakage-free nested CV.	Full-data pilot as default, nested as S2; O7 holds the alternative.
9	Elastic-net α grid	BENCHMARKS.md: {.1, .25, .5, .75, .9}. Oct-7: {.1, .3, .5, .7, .9}.	The first; O8 holds the alternative.

#	Topic	What the documents said	Kept in this master, and why
10	New seed ranges	The scenario specification quotes the checked ranges as 1,320,001–1,370,100 and 1,510,001–1,570,100. The formula base + 10000 · s + r with s = 1..6, r = 1..100 and base 1,510,000 gives 1,520,001–1,570,100 for p = 1000 (arithmetic done for this master; not stated in the sources).	The quoted statement is kept (§II.7); the quoted range contains the used range, so the check claim stands.
11	Dev manifest rows	The verification check says the manifest holds "1,800 production and 90 dev dataset rows". The pilot design (6 scenarios × dev replicate 1 per p) gives 18 dev datasets and 90 dev cases (30 per p × 3); the seed-table step also counts "9,000 production + 90 dev rows" (cases).	Both statements are kept as written (§II.13, §IV.4). The meaning of "90 dev dataset rows" is not resolved in the sources: 90 is the number of dev cases. To be settled when the manifest is built.
12	Exact-posterior comparison in Regime L	The scenario specification lists PIP error, HPM and median-model agreement and log Z error as comparisons "for SMC-BVS and any same-posterior sampler". The plan calls the Regime-L join "optional" and "not a benchmark metric".	Both, as one statement (§II.8, §IV.1.4): an optional descriptive comparison for the SMC arm; no same-posterior sampler is in the panel (§III.4.1).
13	Status wording	The scenario specification says "proposed, awaiting Felipe's sign-off"; the panel summary says "proposed, awaiting Felipe's choice"; the plan says plan only, with G0 for O1–O3.	The plan's wording: plan only, nothing run, G0 for O1–O3 (header and §IV.3).
14	Section numbering	The methodology section of the plan was numbered 1A so that later numbers would not change, and said so in a paragraph.	Dropped as obsolete in the original merge. Part I was replaced and renumbered for v3 on 2026-10-22; use the current contents rather than the historical 1A.n mapping.
15	Cross-reference "§6, step 1"	The plan's library-pinning sentence points to "§6, step 1".	Written as "§IV.3, gate G1", the gate that builds and seals the library.
16	Earlier scenario draft	The scenario specification notes that its previous version (6 signals at every p) was saved separately.	Superseded by the 5 / 14 design (§II.3); not carried into the master.

#	Topic	What the documents said	Kept in this master, and why
17	Role of Regime L (revision of 2026-10-19, D1)	The earlier master ran all eight benchmarks in L and H (9,000 cases per method, 72,000 primary fits) and treated the exact-posterior comparison in L as optional and descriptive.	D1: Regime L only assesses SMC-BVS $\hat{\mathbf{Z}}$ and PIP accuracy against exact enumeration; benchmarks run in H only (6,000 cases per method, 48,000 primary fits). Entry 12 is superseded for Regime L (§II.1, §II.6, §II.8, §III.4.5).
18	Exact enumeration at p = 30	The superseded round-1 R1 said "small p" only; the earlier master left p = 30 unspecified.	The study decision D1 retains external enumeration of all 2^{30} models for every Regime-L case; its implementation and cost remain unspecified (§IV.2.3). Separately, v3 chooses its production branch by M §6.2 and takes SMC at n = 200, p = 30.
19	Sampler block size	R1 §4, §8 and DEC: B = 5 at every p (about 100 / 200 blocks; 3,200 / 6,400 block scores at p = 500 / 1000).	Historical D2 was superseded on 2026-10-22 by M §3: B = min($\lfloor\sqrt{n}\rfloor$, $\lfloor\sqrt{p}\rfloor$, 16), giving 5 / 14 / 14 at n = 200, p = 30 / 500 / 1000. Actual block count depends on whole-cluster packing. The population blocks of five in S1–S6 are unchanged.
20	Settings written as functions of B	R1 writes R_nb = B = 5 and m = 2B = 10; D2 changes B at p $\geq$ 500 but speaks of blocks only.	Closed for v3: B neighbour steps per round; at most B directed partners before symmetric union; m = p + B + 2 labelled steps (M §§6.4–6.5). O13/O14 are historical, not deployment choices.
21	SMC arm and the compute plan	Part I says the SMC arm is not in the budget, gates or timeline; after D1, Regime L has no benchmark tasks and exists only for the SMC arm.	Both: Regime-L tasks are listed in §IV.2.1 for completeness, but their cost, gates and timeline stay with the SMC arm (Part I preamble, §IV.2.3, §IV.4).
22	Pilot and dev counts after D1	§II.13 and §IV.4 count 90 dev rows (all p); the benchmark pilot covered 90 dev cases in 18 tasks.	Data generation keeps the 90 dev rows (the SMC arm can use the p = 30 dev data); the benchmark pilot is 60 dev cases in 12 tasks (p = 500, 1000; §IV.2.2). Entry 11 is unchanged.

Renumbering of the superseded plan: §1.1 → §II.6; §1.2 → §II.7, §II.8 and Appendix A; §1A.n → the historical Part I (replaced by v3 on 2026-10-22); §2.1 to §2.5 → §II.9 to §II.13; §3.x → §III.3.x; §4.x → §IV.1.x; §5.x → §IV.2.x; §6, §6.1, §6.2 → §IV.3, §IV.3.1, §IV.3.2; §7 → §IV.4; §8 → §IV.5; §9 → §IV.6. The scenario specification: §1 → §II.1; §2 → §II.2 and §II.3; §3 → §II.4 and §II.5; §4 → §II.6; §5 → §II.7; §6 → §II.8; §7 → Appendix A.2. The panel summary: §1 to §3 → §III.1, §III.2 and §III.4; §4 → §IV.5 and §III.4. The line-by-line account is in `evidence/MERGE_COVERAGE.md`.

Appendix C. Evidence and tools index

Files kept in the study folder (links are relative to this file).

Evidence (`evidence/`):

File	Content
`evidence/FREQUENTIST.md`	Frequentist evidence: 7-slot recommendation, references with verified ids, software versions, usage quotes with locations, sensitivities, rejected methods, sources (51 verified, 2 unverified).
`evidence/BAYESIAN.md`	Bayesian evidence: selection panel A1–A5, computational panel B1–B4, p > n caveats, sensitivities, rejected methods, sources (28 of 31 references and 16 of 17 software pages verified).
`evidence/notes_frequentist/`	Verbatim quotes with file and section (`EVIDENCE_LOG.md`), `check_quotes.py` (re-matches the 23 quotes; run it from inside `sources/`), the as-delivered original (`FREQUENTIST_as_delivered.md`), and saved sources (CRAN pages, Crossref records, arXiv abstracts, package source files).
`evidence/notes_bayesian/`	Evidence log (`evidence_log.md`), CRAN snapshot, Crossref check, package manuals (PDF and text), saved code pages (PARNI, TGS, scaleBVS, millipede, particles, SSLASSO) and the prior-review paper dump.
`evidence/methodology_traceability.md`	Historical trace of the superseded round-1 Part I; preserved, not the source map for v3.
`deployment_20261022_v3/CHANGES.md`	Current v3 changes, complete source fidelity map, exclusions, prototype disagreements and OPEN items; build verification record.
`evidence/nots_checks_20261019.txt`	Raw read-only NOTS checks of 2026-10-19: modules, installed package versions and signatures, storage, Slurm limits.
`evidence/MERGE_COVERAGE.md`	Coverage check of this merge: where every heading, table and item of the three superseded documents now lives, and the mechanical number check.

Deployment study 2026-10-19 (`deployment_20261019/`): the working files behind §III.3.1–§III.3.8. `FREQ/` (F1–F5) and `BAYES/` (A1–A3) each hold the agent's drop-in text (`DEPLOY_F.md`, `DEPLOY_A.md`; the merged versions with harmonised helper and constant names are `*_harmonised.md`), `CHANGELOG_*.md` (every departure from the earlier §III.3 text, with evidence), `SHARED_PROPOSALS.md` (the shared-section changes applied in this revision), and the GPT-6 Astra packet, raw logs and `ASTRA_RESOLUTION.md`. `integrate.py` performs the merge; `FINAL_REVIEW/` holds the closing Fable review. The source facts are appended to `evidence/FREQUENTIST.md` and `evidence/BAYESIAN.md` (sections "Deployment study 2026-10-19").

Tools (`tools/`):

File	Content
`tools/scenario_check.py`	Population-level checks (no random draws): block, position and S6 ρ of each active index, positive definiteness of Σ, nominal SNR at each scale. Mirrors `scenario_covariance()` in `02_Experiments/E_2026-10-14_p500_studies/Experiments_October_14/scripts/design.R`.
`tools/scenario_check.out`	Its output, the reference for the truth placement (§II.3, §II.13 item 1) and the SNR table (§II.5).

Other study files. `FAILED_ATTEMPTS.md` stays at the study root. Files created during the study (not yet existing): `runs/<run id>/` snapshots and `RUNS.md` (§IV.3.1).

Sources outside the study folder (project paths, not linked): the authoritative `01_Methodology_and_Theory/Methodology/SMC_BVS_v3_20261019/METHODOLOGY_v3.md` (SHA-256 in Part I); DP2_DECISIONS.md, CORE_SPEC.md and the read-only prototype from `C:/Users/felip/AppData/Local/Temp/master_v3_packet/` (source hashes and provenance in `deployment_20261022_v3/CHANGES.md`); the historical round-1 files `01_Methodology_and_Theory/Methodology/Setup_20261019/` (R1, DEC, FB, FA, FBS), the Oct-7 campaign `02_Experiments/C_2026-10-07_benchmarks_p30/BVS_Benchmarks`, the Oct-14 campaign folders, and `CODE_LEDGER.md` at the project root.